Notes on BitVM3: Garbled Circuits, Extractable Signatures, and Adaptor Signatures

Cian Lalor

Important Note: This document (and corresponding codebase) is a work in progress. Sections may be incomplete or require further review.

Notation

$\{0,1\}^\ell$	set of bit-strings of length ℓ; $\lvert\{0,1\}^\ell\rvert = 2^\ell$.
$\{0,1\}^*$	set of bit-strings of arbitrary finite length (no fixed bound).
κ	security parameter (bit-length of labels/keys; typically 128).
$A \leftarrow B$	A is assigned the value B (or the output of B, if B is an algorithm).
$A \xleftarrow{\$} S$	A is sampled uniformly at random from the set S.
$A : B \to C$	A is a function with domain B and codomain C.
$L_i^0 \mapsto 0$	inside a table: "the entry L_i^0 maps to the value 0".
$[n]$	the index set $\{1, 2, \ldots, n\}$: a set of n elements.
$A \sqcup B$	disjoint union: $A \cup B$ with $A \cap B = \emptyset$ asserted, so $\lvert A \sqcup B\rvert = \lvert A\rvert + \lvert B\rvert$.
$\oplus$	bitwise XOR (`^` in Rust on `bool` values).
$\lVert$	string concatenation.
1^κ	the security parameter in unary; standard input format for crypto algorithms.
$\mathrm{negl}(\kappa)$	a function shrinking faster than every inverse polynomial (§2.2).
PPT	probabilistic polynomial-time algorithm.
$\perp$	"no valid decoding"; signals an honest evaluation did not produce this output.
π	overloaded: a SNARK *proof object* (§13) and a *permutation mapping* of garbled gates.

Contents

Notation **i**

Introduction & BitVM3 Overview **1**

I Garbling Foundations **2**

1 Boolean Circuits **2**

2 Garbling Schemes **3**
2.1 Syntactic Definition . . . 3
2.2 Correctness and Authenticity . . . 4
2.3 Privacy-Free Variant . . . 5

3 Yao's Construction **5**
3.1 Wire Labels . . . 5
3.2 Garbled Gate Tables . . . 6
3.3 Worked Example: 1-Bit AND Gate . . . 7
3.4 Chaining Gates: Multi-Gate Evaluation . . . 8

4 Optimized Garbling: Free-XOR and Half-Gates **9**
4.1 Point-and-Permute . . . 9
4.2 Garbled Row Reduction (GRR) . . . 10
4.3 Free-XOR . . . 11
4.4 Half-Gates . . . 12
4.5 The Privacy-Free Half-Gate . . . 13
4.6 The Hash Function . . . 14
4.7 Cost Summary . . . 16

II Committing and Revealing Labels **17**

5 Signature Primitives: What Is Used Where **17**

6 Lamport One-Time Signatures **18**
6.1 Construction and Unforgeability . . . 19
6.2 Non-Equivocation via Reused Keys . . . 21

7 Winternitz One-Time Signatures **22**
7.1 Construction . . . 22
7.2 Checksum and Unforgeability . . . 23

8 Garbled Encoding Extractable Signatures **24**
8.1 Syntax and Security . . . 24
8.2 Lamport-over-Input-Labels Instantiation . . . 25

9 Schnorr Adaptor Signatures **26**
9.1 Motivation: Scriptless Scripts . . . 27
9.2 Schnorr Recap . . . 27
9.3 Pre-Signature, Adapt, Extract . . . 28
9.4 Security Properties . . . 28

9.5 Bridge to GES: Two Realizations of Extractability . . . 29
9.6 Realizing GES with Adaptor Signatures . . . 29
9.7 Wide Labels: Mapping Bits to Digits . . . 32
9.8 Application: Atomic Swap . . . 35

III Protocol and On-Chain Realization **36**

10 Setup Verification **36**
10.1 SNARK or STARK of Correct Setup . . . 36
10.2 Multiple Garblings and Statistical Soundness . . . 37
10.2.1 Cut-and-Choose . . . 37
10.2.2 Verifiable Secret Sharing of Labels . . . 41
10.3 Trusted Execution Environment . . . 43
10.4 Comparison of the Three Routes . . . 43

11 Bitcoin Transactions and the BitVM3 Transaction Graph **44**
11.1 Transaction Anatomy . . . 44
11.2 Taproot . . . 45
11.3 Sighash: What a Signature Commits To . . . 47
11.4 Spending Conditions . . . 48
11.5 Signatures and Adaptors in the Witness . . . 50
11.6 The Assert, Disprove, and Timeout Templates . . . 50

12 The SNARK Being Garbled **51**
12.1 Components . . . 51
12.2 Garbled Verifier Overview . . . 52
12.3 Public Inputs and Binding . . . 52
12.4 Trusted Setup and Toxic Waste . . . 53
12.5 How the Proof Reaches the Chain . . . 53
12.6 Structuring the Verifier Circuit . . . 53
12.7 Hazards and Constraints . . . 54

13 BitVM3-core Protocol **55**
13.1 Role Assignments . . . 56
13.2 Setup, Proving, Disprove . . . 57
13.2.1 Setup (off-chain, once) . . . 57
13.2.2 Proving (on-chain, per claim) . . . 58
13.2.3 Disprove (on-chain, conditional, within the challenge window) . . . 59
13.2.4 Anatomy of the Assert Broadcast . . . 59
13.3 Why the Pieces Compose . . . 62

14 Related Designs and Security History **63**
14.1 The RSA Garbling Scheme and Its Break . . . 63
14.2 The GSV Lineage . . . 63
14.3 Lombard BSA: TEE-Attested, Sealed Labels . . . 63
14.4 Reality Check . . . 63

(Optional) Further Reading **64**

A Bitwise XOR as an Encryption Primitive **64**
A.1 Wide-label rows . . . 64

B Merkle Commitments **64**

Introduction & BitVM3 Overview

A simplified picture of the bridge scenario this document supports.

1. A burn event happens on the destination chain (e.g. Ethereum).
2. The operator collects the burn transaction, the relevant block header, and Merkle inclusion proofs.
3. The operator runs $\texttt{Groth16.Prove}(\mathcal{R}, \text{crs}, \varphi, w)$ and obtains a proof π; the witness w holds the destination-chain artefacts and the statement φ is the burn fact being claimed.
4. The operator publishes a signature σ_π on Bitcoin that commits to π and, by the extractability of §8, releases the garbled encoding L_π of π.
5. A challenger extracts L_π from σ_π, evaluates the garbled verifier, and checks whether the output decodes to False.

The challenger verifies; the operator generates. Knowledge soundness of the SNARK means an accepting π exists only for a true statement φ, that is, only if the burn happened.

BitVM2 settles a dispute by committing the verifier's execution trace on chain and re-executing one disputed step in Bitcoin Script. BitVM3 instead garbles the verifier: the challenger evaluates it off chain, and the only on-chain check is a hash equality, which reduces the on-chain cost of a dispute by roughly 1000× (from about $16,000 to about $9 in the paper's estimate). Because the garbled input is a public proof, BitVM3 uses a *privacy-free* garbling scheme: input privacy is not required, only authenticity.

Two secrets drive the on-chain mechanics: the input labels, released by completing adaptor signatures when the operator claims (§9), and the false-output label, whose hash preimage spends the Disprove transaction (§13).

The document has four parts. *Part I (Garbling Foundations)* builds Boolean circuits, the garbling-scheme abstraction, Yao's construction, and the optimized half-gate garbling the deployed systems use. *Part II (Committing and Revealing Labels)* covers the machinery that releases input labels: which signature is used where, Lamport and Winternitz one-time signatures, the Garbled Encoding Extractable Signature (GES) built on them, and the Schnorr adaptor signatures BitVM3 deploys on chain. *Part III (Protocol and On-Chain Realization)* covers setup verification (cut-and-choose, verifiable secret sharing of labels, a SNARK of setup, or a trusted execution environment), the Bitcoin transactions and the Assert/Disprove graph that carry the dispute, the Groth16 verifier being garbled, the BitVM3-core protocol that composes everything, and the related designs and security history that separate the paper's clean model from what is being built. *Part IV (From Boolean Garbling to Pairing-Based Garbled Verifiers)* covers the pairing-based line that replaces the Boolean verifier: pairing groups and Groth16 verification, elliptic-curve addition as low-degree maps, garbling over fields and decomposable randomized encodings, witness encryption, BABE, Argo MAC, Glock, Duty-Free Bits, adaptor-signature release for field-garbled inputs, and a comparison. Cross-references to the accompanying Rust crates are noted throughout.

Part I: Garbling Foundations

1 Boolean Circuits

Definition 1.1 (Boolean circuit)**.** A *Boolean circuit* computing $f : \{0,1\}^\ell \to \{0,1\}^m$ is a finite directed acyclic graph C. It contains ℓ vertices with no incoming edges, the *input wires* (indexed $1, \ldots, \ell$), and m vertices with no outgoing edges, the *output wires* (indexed $1, \ldots, m$). All other vertices are *gates* and are labelled with one element of a gate set $\mathcal{G}$; the label of gate v is written g_v. The gate set used throughout is

$$\mathcal{G} = \{\text{AND}, \text{XOR}, \text{NOT}\},$$

with fan-in 2 for AND and XOR and 1 for NOT. Every gate has one output wire, which may fan out to several downstream gates. The *size* $|C|$ of a circuit is its number of gates.

Remark 1.1 (Graph terminology)*.* The *in-degree* of a vertex is the number of edges into it and the *out-degree* the number out of it. An input wire has in-degree 0 because its value is supplied from outside, as the i-th component of x; an output wire has out-degree 0 because its value is read off as the i-th component of $f(x)$. For a gate the in-degree is the *fan-in*, the number of bits consumed, and the out-degree the *fan-out*, the number of downstream gates reading its result. Fan-in is restricted to 2 because a gate of fan-in k has a truth table of 2^k rows and the cost of garbling it grows with that count.

Example 1.2 (Half-adder)**.** The half-adder $f(a, b) = (a \oplus b,\ a \wedge b)$ is a circuit on six vertices: input wires a, b; an XOR gate producing the sum bit; an AND gate producing the carry bit; and output wires sum, carry. Both inputs have fan-out 2, since each feeds both gates. It is the running example of §3.

Remark 1.3 (Directed acyclic graphs and topological order)*.* The circuit graph is *directed*, since a wire carries a value from u to v and not the reverse, and *acyclic*, since no sequence of edges returns to its start; cyclic graphs would model stateful circuits with a different evaluation semantics.[1] A directed acyclic graph admits a *topological ordering*, a listing of its vertices in which every edge (u, v) has u before v. The graph fixes only a partial order, so several topological orderings exist; all produce the same $f(x)$, because each gate depends only on its predecessors. Following Bellare, Hoang, and Rogaway (ePrint 2012/265) we write $\texttt{ev}(C, x)$ for the function a circuit computes. The *depth* of a circuit is the length of its longest path; gates at the same depth are independent and may be evaluated in parallel. The implementation stores the gates in one chosen topological order and numbers them in that order, which also supplies the per-gate nonce ν of §3.2.

Definition 1.2 (Evaluation)**.** Given $x = (x_1, \ldots, x_\ell) \in \{0,1\}^\ell$, the *evaluation* of C on x assigns $\beta_v \in \{0,1\}$ to each vertex in topological order: $\beta_{v_i} = x_i$ for input wires; $\beta_v = g_v(\beta_{u_1}, \ldots, \beta_{u_k})$ for a gate v with label g_v and predecessors $u_1, \ldots, u_k$; and $\beta_o = \beta_u$ for an output wire o with predecessor u. The output is $f(x) = (\beta_{o_1}, \ldots, \beta_{o_m})$.[2]

[1] A program with a loop still yields an acyclic circuit: the loop is *unrolled*, each iteration becoming its own copy of the loop body with fresh wires, so the wires of iteration k feed the gates of iteration $k + 1$ and never return to their own iteration. A circuit has no notion of time or state, so the number of iterations must be fixed when the circuit is built. For the pairing this is automatic: the Miller loop runs over the bits of a fixed curve constant and the final exponentiation uses a fixed exponent, so the verifier circuit has the same shape for every proof. A loop whose count depends on the input is unrolled to its maximum, each iteration guarded by a selector bit that passes the state through unchanged once the loop has terminated.

[2] The half-adder has $m = 2$; the SNARK verifier circuit of §13 has $m = 1$, a single accept-or-reject bit, which is what the BitVM dispute turns on.

The full assignment $\{\beta_v\}$ over every vertex, not only the output wires, is the analogue of BitVM2's execution trace; $f(x)$ is its final output.

2 Garbling Schemes

A garbling scheme transforms a Boolean circuit f into a randomized object F that another party can evaluate on an input it does not know in cleartext. The *garbler* chooses, for each wire of f, a pair of random labels (L_w^0, L_w^1) standing for the bits 0 and 1. The *evaluator* sees only labels, never bits, runs the garbled object gate by gate to obtain output labels, and a separate decoding step converts those labels to a plaintext bit string. What makes the object useful is what survives a malicious counterparty: an honest evaluator learns output $f(x)$ for a given input x and, in the full scheme, nothing else; a malicious evaluator cannot produce labels that decode to anything but $f(x)$.

In BitVM3 the roles are fixed. The prover is the garbler and the verifier the evaluator (BABE, in Part **??**, inverts this: the verifier garbles and the prover evaluates); the input being garbled is a SNARK proof π, which is public at two layers. The proof itself appears in plaintext on chain, as the message the prover signs; its garbled encoding $L_\pi = \texttt{En}(e, \pi)$ is a representation of that proof under the encoding e fixed at setup and committed by the GES public key (Remark 2.2), released by the same signature through public encrypted rows (§9.7), after which anyone holding the public garbled tables F can evaluate. Privacy is therefore not required, but authenticity is: a malicious verifier must not be able to produce labels that decode to False when $\texttt{SNARK.Vrfy}(\pi) = \text{True}$, which is what protects an honest prover from being slashed. A malicious *prover*, who might garble the wrong circuit, is a separate problem, handled by setup verification (§10).

2.1 Syntactic Definition

Definition 2.1 (Garbling scheme)**.** Let $\mathcal{F}$ be a class of Boolean circuits computing functions $f : \{0,1\}^\ell \to \{0,1\}^m$. A *garbling scheme* for $\mathcal{F}$ is a tuple $\mathsf{G} = (\texttt{Gb}, \texttt{En}, \texttt{Ev}, \texttt{De})$ of polynomial-time algorithms:[3]

1. $(F, e, d) \leftarrow \texttt{Gb}(1^\kappa, f)$: a probabilistic *garbler* taking a security parameter 1^κ and a circuit $f \in \mathcal{F}$, returning a garbled circuit F, encoding information e, and decoding information d.
2. $L_x \leftarrow \texttt{En}(e, x)$: a deterministic *encoder* mapping an input $x \in \{0,1\}^\ell$ to encoded labels L_x.
3. $L_y \leftarrow \texttt{Ev}(F, L_x)$: a deterministic *evaluator* mapping the garbled circuit and an encoded input to encoded output labels L_y.
4. $y \leftarrow \texttt{De}(d, L_y)$: a deterministic *decoder* recovering $y \in \{0,1\}^m \cup \{\perp\}$, where $\perp$ signals that L_y encodes no output.

Remark 2.1 (Probabilism and the security parameter). Gb draws internal randomness, the wire labels, so the same circuit yields different (F, e, d) on each run; the other three algorithms are deterministic. The security parameter κ is the label length in bits of the schemes of §3 and §4, $\kappa = 128$ throughout; the unary form 1^κ is chosen so that κ can be measured in polynomial time.[4] The baseline implementation uses 256-bit labels for a reason unrelated to garbling security (Remark 3.5).

[3] Bellare, Hoang, and Rogaway (ePrint 2012/265) include a fifth component, the plaintext evaluation function ev of Remark 1.3; the BitVM3 paper uses the four-algorithm form.

[4] Polynomial time is measured against the length of the input. Representing the security parameter κ in unary ensures that input length scales linearly with the work. Written in binary, $\kappa = 128$ is 10000000, eight symbols; doubling the security to $\kappa = 256$ gives 100000000, nine symbols. Each added symbol doubles κ and with it the

Remark 2.2 (The three outputs of Gb). The garbled circuit F is the public artefact: anyone holding it can run Ev. The encoding information e translates a plaintext input into the labels Ev accepts, so whoever encodes inputs holds e. The decoding information d inverts the translation at the output. In BitVM3-core (§13) the prover holds all three at garbling time and publishes F and d; e is never published. Instead the public key of the Garbled Encoding Extractable Signature (GES) of §8 commits to all of e, and signing an input x exposes only $\mathtt{En}(e, x)$, one label per wire. Releasing a wire's complementary label is equivocation and is slashed.

Remark 2.3 (Decoding in practice). For the constructions of §3, d stores, for each output wire i, the hashes $H(L_i^0 \,\|\, \text{out})$ and $H(L_i^1 \,\|\, \text{out})$ of its two labels. The output label is the plaintext recovered from the last gate's row, not a hash; the fixed string out is a tag that keeps these decoding hashes distinct from the gate-table hashes of §3.2, which hash input labels under a gate nonce to form the row pads, so that no value published in d can equal a pad in F. De hashes each component of $L_y = (L_1, \ldots, L_m)$ and compares: if every hash matches one of the two stored values for its wire it returns the corresponding bit string, otherwise $\perp$. Storing hashes rather than labels is what lets d be public, as it is on chain, without handing out the labels themselves: in BitVM3 the hash of the output wire's False label is the Disprove hashlock (§13). e, by contrast, holds both labels of every input wire, so whoever has it can encode a false input and obtain the False label by honest evaluation, which is why e must stay private. A label that is not the honest output matches a stored hash with probability $2^{-\kappa}$ per wire, which is the authenticity guarantee below.

2.2 Correctness and Authenticity

Remark 2.4 (PPT adversaries and negligible probability). A *probabilistic polynomial-time* (PPT) algorithm is a randomized algorithm whose running time is bounded by a polynomial in κ; brute-forcing a κ-bit key takes 2^κ steps and is not PPT. A function $\mu : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is *negligible* if for every constant $c > 0$ there is κ_0 with $\mu(\kappa) < \kappa^{-c}$ for all $\kappa \geq \kappa_0$; we write $\text{negl}(\kappa)$ for any such function. "Pr[bad event] $\leq \text{negl}(\kappa)$ for every PPT adversary" therefore says that no efficient attacker, even with polynomially many attempts, makes the bad event happen with non-vanishing probability. The *union bound*, $\Pr[A_1 \cup \cdots \cup A_n] \leq \sum_i \Pr[A_i]$, turns a bound on one bad event into a bound over all of an adversary's attempts.

Definition 2.2 (Correctness)**.** A garbling scheme is *correct* if for every circuit $f \in \mathcal{F}$ and every input $x \in \{0,1\}^\ell$,

$$\Pr\big[\, \mathtt{De}(d, \mathtt{Ev}(F, \mathtt{En}(e, x))) = f(x) \,\big] \geq 1 - \text{negl}(\kappa),$$

the probability being over the randomness of $\mathtt{Gb}(1^\kappa, f) = (F, e, d)$. The schemes of §4 achieve probability 1 at the evaluation step, since no row is found by trial; decoding through the hashes of Remark 2.3 fails only on a hash collision, a $2^{-\kappa}$ event per output wire. The trial-decryption construction of §3 achieves $1 - \text{negl}(\kappa)$ (Theorem 3.11).

Definition 2.3 (Authenticity)**.** A garbling scheme is *authentic* if for every PPT adversary $\mathcal{A}$, every $f \in \mathcal{F}$, and every $x \in \{0,1\}^\ell$, with

$$(F, e, d) \leftarrow \mathtt{Gb}(1^\kappa, f), \qquad L_x := \mathtt{En}(e, x), \qquad L'_y \leftarrow \mathcal{A}(1^\kappa, f, F, L_x, d),$$

$$\Pr\big[\, L'_y \neq \mathtt{Ev}(F, L_x) \wedge \mathtt{De}(d, L'_y) \neq \perp \,\big] \leq \text{negl}(\kappa).$$

The adversary receives what a BitVM3 challenger receives, including d, which the game of Bellare, Hoang, and Rogaway (ePrint 2012/265) withholds; the stronger form is needed because d is public on chain (Remark 2.3).

work of Gb, which writes κ-bit labels, so the work is 2^n on an n-symbol input, which no polynomial bounds. In unary the input is the string 1^κ of κ ones, of length κ, so work proportional to κ is polynomial in the input length.

The bounded event is that $\mathcal{A}$ produces output labels that differ from the honest evaluation yet still decode. A challenger holding the labels of a valid proof therefore cannot fabricate the False label, and an honest prover cannot be slashed.

2.3 Privacy-Free Variant

The framework of Bellare, Hoang, and Rogaway[5] defines, beyond correctness and authenticity, *privacy* (the evaluator learns nothing about x beyond $f(x)$ and a fixed side-information function of f, such as its topology) and *obliviousness*[6] (the evaluator learns nothing about x, not even $f(x)$). A scheme required to satisfy only correctness and authenticity is *privacy-free*, a notion introduced by Frederiksen, Nielsen, and Orlandi (EUROCRYPT 2015, ePrint 2014/598), who also showed that dropping privacy halves the cost of an AND gate.

In BitVM3-core the garbled input is a SNARK proof π that the prover publishes on chain, so there is nothing for privacy to protect. Dropping it admits the privacy-free half-gate of §4.5: one ciphertext per AND gate instead of two, with XOR gates free. Against plain Yao's four ciphertexts per gate this is a 4× reduction on AND gates, 2× from dropping privacy and 2× from the other optimizations of §4, and XOR gates go from four ciphertexts to none.

Remark 2.5 (Why the labels survive)*.* Privacy is the only property dropped; the label pairs are kept for authenticity. Evaluation still yields only the genuine output label, and forging the label of a false output is infeasible (Definition 2.3). The unforgeable output label, obtainable only by evaluating on the labels the prover released, is what the Disprove transaction binds (§13).

Remark 2.6 (What BitVM3 skips)*.* Three parts of the general scheme fall away. The bit-hiding machinery of privacy and obliviousness is dropped. The per-gate ciphertext shuffle of §3.2 is replaced, in the deployed scheme of §4, by permute bits that point the evaluator at its row; the baseline implementation of §3 keeps the shuffle. And the input labels are not delivered by oblivious transfer (Remark 3.9); they are released by signing the proof (§8).

3 Yao's Construction

3.1 Wire Labels

Section 2 gave the four-algorithm interface but not a construction. Yao's construction is the first; its building block is the *wire label*, a random κ-bit string standing for the bit 0 or 1 on a given wire, and the evaluator sees only labels until the final decoding. The trial decryption, padding and row shuffle introduced here are removed one by one by the optimizations of §4; the implementation uses the final form, the privacy-free half-gate. The written source for the construction and its proof is Lindell and Pinkas (ePrint 2004/175).

Definition 3.1 (Wire labels)**.** For a Boolean circuit C with wires indexed $1, \ldots, W$, the garbler samples, independently for each wire w and each bit $b \in \{0,1\}$, a label

$$L_w^b \xleftarrow{\$} \{0,1\}^\kappa.$$

The pair (L_w^0, L_w^1) is the *label pair* of wire w; every wire, input, intermediate, or output, has one.

At garble time the garbler holds all $2W$ labels. The input-wire pairs form the encoding information e; the hashes of the output-wire labels form the decoding information d (Remark 2.3). The evaluator receives one label per input wire, the one matching its bit, and d; no intermediate wire's pair is ever revealed.

[5] M. Bellare, V. T. Hoang, P. Rogaway, *Foundations of Garbled Circuits*, ACM CCS 2012. ePrint 2012/265, `https://eprint.iacr.org/2012/265`.

[6] A security property of the scheme, distinct from *oblivious transfer*, the protocol of Remark 3.9.

Remark 3.1 (Why intermediate wires are never revealed). A bit-to-label mapping exists only at the circuit's boundary, so an intermediate wire has no decoding; evaluation surfaces only its single active label (§3.2), and the complementary one stays locked in the garbled tables. In plain Yao this is what hides the intermediate bits. In the privacy-free setting the evaluator knows every intermediate bit anyway, by recomputing f on the public input, and each intermediate label is an authenticated token rather than a secret. A garbled circuit is for this reason a one-time object: evaluation reveals one label per wire, and labels released for a second input could be combined with the first set into the labels of an input that was never signed, whose evaluation yields the False label, the preimage of the Disprove hashlock. Releasing labels for two proofs is the equivocation that BitVM3 slashes (Remark 2.2), and a garbling is never reused for a second proof.

3.2 Garbled Gate Tables

A garbled gate is the encrypted analogue of the gate's *truth table*, the four rows $(i, j) \in \{0,1\}^2$ of a fan-in-2 gate with the output bit of each. Each row is encrypted under the two input labels for that row; the evaluator, holding the labels for the actual input, can decrypt exactly one row and recover the output label.

Definition 3.2 (Dual-key cipher)**.** Let $H : \{0,1\}^* \to \{0,1\}^{\kappa+t}$ be a hash function modelled as a random oracle,[7] and let t be a padding length. For keys $K_1, K_2 \in \{0,1\}^\kappa$, a message $M \in \{0,1\}^\kappa$, and a nonce ν, the *dual-key* encryption of M is

$$E^{\nu}_{K_1,K_2}(M) := H(K_1 \,\|\, K_2 \,\|\, \nu) \oplus (M \,\|\, 0^t),$$

with decryption $D^{\nu}_{K_1,K_2}(C) := H(K_1 \,\|\, K_2 \,\|\, \nu) \oplus C$, accepted if the last t bits are zero and $\perp$ otherwise.[8] For a fan-in-1 gate the single-key form $E^{\nu}_{K}(M) := H(K \,\|\, \nu) \oplus (M \,\|\, 0^t)$ is used. The term *dual-key cipher* is from Bellare, Hoang, and Rogaway (ePrint 2012/265); the 0^t marker, which makes a wrong-key decryption detectable, is the verifiable-range encryption of Lindell and Pinkas (ePrint 2004/175).

Remark 3.2 (Why padding). Decryption under the wrong keys yields a uniformly random $(\kappa+t)$-bit string, whose last t bits are zero with probability 2^{-t}. The pad is therefore a checkable witness that the keys were correct. With $t = 128$, as instantiated throughout, a false acceptance is a 2^{-128} event.

Remark 3.3 (Nonce choice). The nonce ν is the gate's index in the chosen topological order, public and derived identically by both parties; the implementation hashes it as a 4-byte little-endian integer after the keys. It is needed because two gates may share the same input wires, as the XOR and AND of the half-adder do (Example 1.2): without ν their rows for the same (i, j) would be masked by the same hash, the XOR of two such rows would cancel the mask and expose the XOR of two output labels, and an evaluator who knows one of them learns the other, including a label it was never given. Bellare, Hoang, and Rogaway call ν a tweak.

Remark 3.4 (Row size). A row is $\kappa + t$ bits, so H must emit $\kappa + t$ bits, by choosing a hash of that output length or truncating a longer one. A plain-Yao garbled circuit on $|C|$ fan-in-2 gates is $4|C|(\kappa + t)$ bits.

Remark 3.5 (The baseline implementation: 256-bit labels, SHA-384). The baseline crate uses each label itself as a secp256k1 scalar, an integer modulo the group order n serving as the adaptor witness t with statement $T = [t]G$, for the per-wire adaptor bridge of Remark 9.6,

[7] A standard idealization: H returns an independent uniformly random value on each new input and the same value on repeated inputs. Real hash functions are deterministic, and proofs in the random oracle model are accepted when no better assumption is available.

[8] XOR is self-inverse, so the second XOR cancels the first; Appendix A gives a bit-level example.

which forces $\kappa = 256$: a 128-bit label read as a scalar would leave a discrete log solvable in $\approx 2^{64}$ steps (Remark 9.7), while rejection sampling below the group order keeps the full $\approx 2^{128}$. With $t = 128$ a row is 384 bits, so the baseline uses SHA-384, whose output spans a row exactly. This is a property of that bridge, not of garbling: the half-gate scheme of §4 uses $\kappa = 128$ and releases labels through the wide-label rows of §9.7, where labels remain 128-bit strings and the adaptor witnesses are separate scalars (field elements) $W_{d,v}$ under which clusters of labels are encrypted.

Definition 3.3 (Garbled gate table)**.** Let g be a fan-in-2 gate with input wires a, b and output wire c. The *garbled gate table* for g is the set of four ciphertexts

$$\left\{ E^{\nu}_{L_a^i, L_b^j}\big(L_c^{g(i,j)}\big) \;\middle|\; (i,j) \in \{0,1\}^2 \right\},$$

shipped in the order given by a uniformly random permutation π_g of the four positions, a step needed only for the variant with privacy (Remark 3.7).

The evaluator, holding L_a^i and L_b^j, attempts $D^{\nu}_{L_a^i, L_b^j}$ on each ciphertext. The one encrypted under (L_a^i, L_b^j) decrypts to a string ending in 0^t; each of the other three does so with probability 2^{-t}, so, except with probability at most $3 \cdot 2^{-t}$, exactly one row passes and its leading κ bits are the output label $L_c^{g(i,j)}$.

Remark 3.6 (Truth-table size). A fan-in-k gate has 2^k rows: four for AND and XOR, two for NOT. The shuffle π_g is one of $4! = 24$ permutations for a fan-in-2 gate.

Remark 3.7 (Role of the shuffle). Without π_g the evaluator could read the input bits off the index of the row that decrypted, even though the labels themselves leak nothing. The shuffle is essential to privacy in plain Yao and unnecessary in the privacy-free setting, where the input is public; point-and-permute (§4.1) replaces it by permute bits, which keep privacy without trial decryption, and the privacy-free half-gate (§4.5) then sets those bits to zero.

3.3 Worked Example: 1-Bit AND Gate

One AND gate, end to end. Throughout, the evaluator handles only labels and recovers the plaintext of the output wire alone, through d; in plain Yao the input bits stay hidden.

1. *Setup.* The gate has input wires a, b and output wire c. The garbler samples six labels,

$$L_a^0, L_a^1, \quad L_b^0, L_b^1, \quad L_c^0, L_c^1 \xleftarrow{\$} \{0,1\}^{\kappa},$$

   sets $e = \{a \mapsto (L_a^0, L_a^1),\ b \mapsto (L_b^0, L_b^1)\}$, and sets d to the hashes of L_c^0 and L_c^1 (Remark 2.3).

2. *Garbling.* The four rows of the AND truth table in label form:

(i,j)	$i \wedge j$	row
$(0,0)$	0	$(L_a^0, L_b^0) \to L_c^0$
$(0,1)$	0	$(L_a^0, L_b^1) \to L_c^0$
$(1,0)$	0	$(L_a^1, L_b^0) \to L_c^0$
$(1,1)$	1	$(L_a^1, L_b^1) \to L_c^1$

   Each row is encrypted under its input labels with the gate's nonce ν:

$$\begin{aligned} C_{(0,0)} &= H(L_a^0 \,\|\, L_b^0 \,\|\, \nu) \oplus (L_c^0 \,\|\, 0^t), \\ C_{(0,1)} &= H(L_a^0 \,\|\, L_b^1 \,\|\, \nu) \oplus (L_c^0 \,\|\, 0^t), \\ C_{(1,0)} &= H(L_a^1 \,\|\, L_b^0 \,\|\, \nu) \oplus (L_c^0 \,\|\, 0^t), \\ C_{(1,1)} &= H(L_a^1 \,\|\, L_b^1 \,\|\, \nu) \oplus (L_c^1 \,\|\, 0^t). \end{aligned}$$

   The garbler ships the four ciphertexts in the order given by $\pi_g \xleftarrow{\$} S_4$.

3. *Encoding.* For input $x = (1, 0)$, meaning $a = 1$ and $b = 0$, $\texttt{En}(e, x) = (L_a^1, L_b^0) =: L_x$. The slot order, first a then b, is fixed by the public circuit description. In BitVM3 the input is the proof π, and the prover releases $L_\pi = \texttt{En}(e, \pi)$ by signing π (§8).

4. *Evaluation.* The evaluator computes $h := H(L_a^1 \,\|\, L_b^0 \,\|\, \nu)$ once and checks, for each shipped ciphertext C, whether $h \oplus C$ ends in 0^t. The ciphertext built as $C_{(1,0)}$ passes; the other three pass with probability 2^{-t} each (Remark 3.2). Stripping the pad gives $L_y = L_c^0$.

5. *Decoding.* $\texttt{De}(d, L_y)$ computes $H(L_c^0 \,\|\, \text{out})$, finds the stored hash for bit 0, and returns $y = 0 = 1 \wedge 0$.

The four algorithms, steps 2 to 5 above, are an abstraction; in a protocol several run on one party. In Yao's two-party protocol one party garbles, encodes its own input, and produces d, while the other evaluates and decodes, obtaining the labels for its own input by oblivious transfer. The BitVM3 role assignment is in §13.1.

Remark 3.8 (What the evaluator learned)*.* The evaluator holds L_a^1, L_b^0, L_c^0, three uniformly random strings, and learned the output bit through d. In plain Yao it learns nothing else: the labels look like fresh samples, and the position of the successful row is uniform in $\{1, 2, 3, 4\}$ by the shuffle. It never sees L_c^1, and that is what authenticity rests on (Theorem 3.11).

Remark 3.9 (Oblivious transfer and BitVM3)*.* In Yao's two-party protocol the evaluator must obtain the label matching each of its own input bits without revealing the bit to the garbler, which 1-out-of-2 *oblivious transfer* (OT) provides; it is the only interactive step. BitVM3 has no OT: the garbled input is a public proof, and the prover releases the active input labels by signing it under the Garbled Encoding Extractable Signature of §8, the complementary half of e never being exposed.

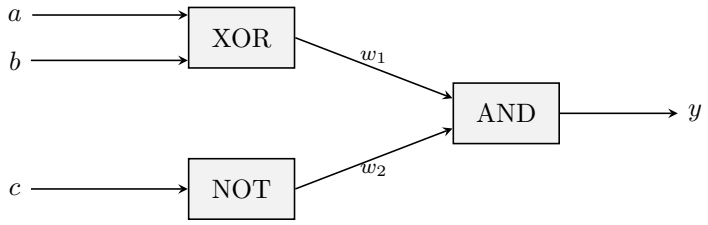
3.4 Chaining Gates: Multi-Gate Evaluation

No new construction is needed for a circuit of many gates. `Ev` iterates the single-gate procedure of §3.3 over the gates in the chosen topological order, so every gate runs after the gates feeding it.

Chaining needs no additional device: each wire has one label pair, sampled once. For an intermediate wire w that is the output of gate g and an input of gate g', so that w is the edge (g, g') and g is a predecessor of g' in the sense of Definition 1.2, the labels L_w^0, L_w^1 are the messages encrypted inside g's rows and the keys under which g''s rows are encrypted. The label recovered from the right row of g is therefore one of the keys for the right row of g': the wiring of the plaintext circuit is mirrored at the label level with no additional structure.

To evaluate a circuit with W wires and gates $g_1, \ldots, g_{|C|}$ in topological order, keep a buffer $B[1], \ldots, B[W]$ of recovered labels, initially unset. Set $B[i]$ to the label from L_x for each input wire i; then for each gate g_k in order, read its input labels from B, run the trial-decryption procedure on its table, and write the recovered output label to B at its output wire. The labels at the output-wire indices form L_y, and $\texttt{De}(d, L_y)$ gives the output. Intermediate labels serve as keys downstream and are otherwise discarded; only output labels are looked up in d.

Example 3.10 (A 3-gate circuit)**.** Consider $y = (a \oplus b) \wedge (\neg c)$ over $\{\text{AND}, \text{XOR}, \text{NOT}\}$:



Six wires: input wires a, b, c, intermediate wires $w_1 = a \oplus b$ and $w_2 = \neg c$, and output wire $y = w_1 \wedge w_2$. Topological order of gates: XOR, NOT, AND. Write $x = (x_a, x_b, x_c)$ for the input bits.

The garbler produces twelve labels, a pair per wire; a table of 4 ciphertexts for the XOR, 2 for the NOT (single-key form), and 4 for the AND, each shuffled independently. It ships F, the three tables (10 ciphertexts); e, the three input-wire pairs; and d, the two output-label hashes.

The evaluator, with $L_x = (L_a^{x_a}, L_b^{x_b}, L_c^{x_c})$ and the gate nonces ν_1, ν_2, ν_3, trial-decrypts the XOR table under $(L_a^{x_a}, L_b^{x_b})$ to obtain $L_{w_1}^{x_a \oplus x_b}$, the NOT table under $L_c^{x_c}$ to obtain $L_{w_2}^{\neg x_c}$, and the AND table under those two to obtain L_y^y; then $\mathtt{De}(d, L_y^y) = y$. The complementary label of every wire stays with the garbler.

Theorem 3.11 (Yao's construction is correct and authentic)**.** In the random oracle model, the construction of §3.2 with trial decryption satisfies Definition 2.2, with failure probability at most $3|C| \cdot 2^{-t}$, and Definition 2.3.

Proof sketch. Correctness: honest evaluation fails only if, at some gate, a wrong row passes the marker before the right one, a 2^{-t} event per wrong row (Remark 3.2); one such gate spoils the whole evaluation, since its wrong label is the key for every gate downstream, so the failure probability is at most the sum over the $3|C|$ wrong rows (union bound, Remark 2.4); with $t = \kappa$ it is negligible.

Authenticity: $\mathcal{A}$ holds F, L_x, d, and f. Every ciphertext in F is a random-oracle output masked onto a label, and during garbling the oracle was queried only on key tuples. Call a key tuple *inactive* if it contains a label $\mathcal{A}$ was not given. Unless $\mathcal{A}$ queries H on an inactive tuple, each inactive label is, in its view, a uniform κ-bit string masked by a fresh oracle value, and so is every output label other than the honest one. Querying an inactive tuple requires guessing an inactive label, which succeeds with probability at most $2^{-\kappa}$ per query, and producing an $L'_y \neq \mathtt{Ev}(F, L_x)$ that matches a hash in d without such a query likewise requires guessing a κ-bit label. Each hash query and each guess at an output label succeeds with probability at most $2^{-\kappa}$, and a PPT adversary makes polynomially many of each, so by the union bound its advantage is at most $\mathrm{poly}(\kappa) \cdot 2^{-\kappa}$. □

4 Optimized Garbling: Free-XOR and Half-Gates

Plain Yao (§3.2) ships $4(\kappa + t)$ bits per gate and finds the right row by trial decryption. The garbled object is the dominant off-chain storage cost of BitVM3, 16 bytes times the AND count (§4.7), distinct from the on-chain cost of a dispute, so each bit saved per gate matters. Five steps, applied in sequence, bring the cost to κ bits per AND gate and zero per XOR gate: four optimizations that keep privacy (point-and-permute, row reduction, Free-XOR, half-gates) and one relaxation that drops it (the privacy-free half-gate), which is the garbler the implementation uses. Each step states its garbler and evaluator equations, a proof sketch, its source, and its cost against the step before; percentages are for $\kappa = t = 128$. Throughout, (L_w^0, L_w^1) is the label pair of wire w (Definition 3.1), H is the hash of Definition 3.2 with its output truncated to κ bits, and ν is the public per-gate nonce (Remark 3.3).

4.1 Point-and-Permute

Trial decryption exists because the evaluator does not know which of the four rows is its own. Point-and-permute (Beaver, Micali, Rogaway, STOC 1990) lets the garbler tell it, without revealing the input bits.

Definition 4.1 (Permute bit, select bit)**.** For each wire w the garbler samples a *permute bit* $p_w \xleftarrow{\$} \{0,1\}$ and arranges the label pair so that $\mathrm{lsb}(L_w^b) = p_w \oplus b$ for $b \in \{0,1\}$. Write

L_w for whichever label of wire w the evaluator holds. Its *select bit* is its least significant bit, $s_w := \mathrm{lsb}(L_w)$, read directly off the label; by construction $s_w = p_w \oplus b$ when $L_w = L_w^b$, but the evaluator never learns b or p_w.

The garbler stores the row for inputs (i, j) at position $(p_a \oplus i,\ p_b \oplus j)$ of the table instead of shuffling. The evaluator holding L_a^i, L_b^j reads $(s_a, s_b) = (p_a \oplus i,\ p_b \oplus j)$ off its labels and decrypts that single row.

Proposition 4.1 (Point-and-permute)**.** The row at position (s_a, s_b) is the row for inputs (i, j), and the position reveals nothing about (i, j).

Proof sketch. The first claim holds by construction. For the second, (s_a, s_b) is (i, j) masked by the uniform bits (p_a, p_b), which the evaluator never learns, so the position is uniformly distributed whatever the inputs are. □

Two consequences follow. The marker is no longer needed: the row at position (s_a, s_b) always decrypts correctly, so the 0^t pad of Definition 3.2 is dropped and each row shrinks from $\kappa+t$ to κ bits. And the shuffle π_g of Definition 3.3 is replaced by the permute bits (Remark 2.6). Output decoding is unchanged from Remark 2.3: d stores the hashes $H(L_o^0 \,\|\, \mathrm{out})$ and $H(L_o^1 \,\|\, \mathrm{out})$ of each output wire's labels, and `De` compares; a forged label is rejected while the labels themselves stay secret until released.

Point-and-permute saves $t/(\kappa + t)$, 50% at $t = \kappa$, over plain Yao, the previous step, because the marker 0^t is no longer needed once the evaluator knows its row by position: a gate costs 4κ bits against $4(\kappa + t)$.

4.2 Garbled Row Reduction (GRR)

With rows at known positions the garbler may choose output labels so that one row need not be published (Naor, Pinkas, Sumner, ACM Conference on Electronic Commerce 1999). The row at position $(0, 0)$ is the one for inputs $(i, j) = (p_a, p_b)$; the garbler defines the output label of that row as the hash itself,

$$L_c^{g(p_a,p_b)} := H(L_a^{p_a} \,\|\, L_b^{p_b} \,\|\, \nu),$$

so that the row's ciphertext is 0^κ (distinct from the marker 0^t of Definition 3.2) and is omitted. In GRR3, three ciphertexts remain, and an evaluator who lands on position $(0, 0)$ outputs the hash directly, i.e., if both input labels have lsb 0, simply hash them with the gate number to get the output label. For example, with $p_a = 1$ and $p_b = 0$ the free row is position $(0, 0)$, serving inputs $(1, 0)$, with output label $H(L_a^1 \,\|\, L_b^0 \,\|\, \nu)$. Output labels are now derived rather than sampled, which is harmless, since a hash output is as unpredictable as a fresh label.

GRR3 reduces the cost by 25% over point-and-permute because one of the four rows is fixed to 0^κ and omitted: a gate costs 3κ bits against 4κ, and, at $t = \kappa$, 62.5% less than plain Yao.

Remark 4.2 (GRR2 and why it is not used). A two-ciphertext variant exists (Pinkas, Schneider, Smart, Williams, ASIACRYPT 2009, ePrint 2009/314). The four pads $H(L_a^i \,\|\, L_b^j \,\|\, \nu)$ are read as points of two polynomials over the finite field GF(2^κ):[9] the three pads of the rows with output

[9]The field structure is what interpolation needs: recovering a quadratic from three points divides by differences of the points' x-coordinates, and division requires every nonzero element to have an inverse, which holds in a field but not in a ring such as the integers modulo 2^κ. In GF(2^κ) the elements are κ-bit strings, addition is XOR, so pads and labels add as they already do, and multiplication is polynomial multiplication modulo a fixed irreducible polynomial. An element is a polynomial of degree below κ with coefficients in $\{0, 1\}$, the bit string listing its coefficients, so adding two elements adds coefficients modulo 2, which is bitwise XOR; a product has degree up to $2\kappa - 2$ and is reduced modulo the irreducible polynomial, which plays the role a prime modulus plays for the integers and guarantees that every nonzero element has an inverse. The scheme works over GF(2^κ)

0 determine a quadratic polynomial Q, the garbler sets $L_c^0 := Q(0)$ and publishes two further points of Q, so an evaluator holding any one of those pads reconstructs Q and reads off L_c^0; the row with output 1 uses a second quadratic polynomial through its pad and the same two points, giving L_c^1. Two values are published per gate, for every gate type. Both output labels are now outputs of interpolation, so their difference $L_c^0 \oplus L_c^1$ is not the garbler's to choose. In itself this is acceptable, since the garbler can commit to any final output label in the hash-preimage spending path of the Disprove script (§13); the obstacle is that Free-XOR (§4.3) requires that difference to be the global Δ on every wire. GRR2 therefore excludes Free-XOR and with it half-gates: it costs 2κ bits per gate, XOR gates included, against κ bits per AND gate and none per XOR gate for the final scheme of §4.5, and it needs a polynomial interpolation at every gate.

Example 4.3 (GRR2)**.** Take the field of integers modulo 7, GF(7),[10] in place of GF(2^κ), since interpolation uses only the field axioms and arithmetic modulo 7 can be checked by hand; positions $0, 1, 2$ for the rows with output 0 with pads $3, 5, 6$, and position 3 for the row with output 1 with pad 1. The quadratic polynomial through $(0, 3), (1, 5), (2, 6)$ is $Q(x) = 3x^2+6x+3$, so $L_c^0 = Q(0) = 3$, and the garbler publishes $Q(4) = 5$ and $Q(5) = 3$. The quadratic polynomial through $(3, 1), (4, 5), (5, 3)$ is $Q'(x) = 4x^2 + 4x + 2$, so $L_c^1 = Q'(0) = 2$. An evaluator at position 1 holds the point $(1, 5)$, the position read from its select bits and the pad $5 = H(L_a \,\|\, L_b \,\|\, \nu)$ computed from its two labels, exactly as the garbler computed it when building Q; as a quadratic polynomial is fixed by any three of its points, this pad together with the published points $(4, 5)$ and $(5, 3)$ gives back Q, so it outputs $Q(0) = 3$, while one at position 3 holds $(3, 1)$, which with the same two points gives back Q', so it outputs $Q'(0) = 2$.

4.3 Free-XOR

Definition 4.2 (Distribution)**.** A *distribution* $\mathcal{R}$ on $\{0,1\}^\kappa$ assigns each string a probability of being drawn, the probabilities summing to 1; $\Delta \leftarrow \mathcal{R}$ denotes drawing Δ by that rule, and $\Delta \xleftarrow{\$} \{0,1\}^\kappa$ is the uniform case.

Definition 4.3 (Global offset)**.** The garbler samples one secret $\Delta \xleftarrow{\$} \{0,1\}^\kappa$ for the whole circuit, with $\mathrm{lsb}(\Delta) = 1$, and, for every wire w, samples only L_w^0 and sets $L_w^1 := L_w^0 \oplus \Delta$ (Kolesnikov and Schneider, ICALP 2008, who write R for Δ). For an XOR gate $c = a \oplus b$ the garbler sets $L_c^0 := L_a^0 \oplus L_b^0$ and publishes nothing for XOR gates. The garbler therefore stores W labels and Δ in place of $2W$ labels.

Proposition 4.4 (XOR gates are free)**.** The evaluator holding L_a^i and L_b^j outputs $L_a^i \oplus L_b^j = L_c^{i \oplus j}$.

Proof. Writing $i\Delta$ for Δ if $i = 1$ and 0^κ otherwise,

$$
\begin{aligned}
L_a^i \oplus L_b^j &= (L_a^0 \oplus i\Delta) \oplus (L_b^0 \oplus j\Delta) && \text{Definition 4.3} \\
&= L_a^0 \oplus i\Delta \oplus L_b^0 \oplus j\Delta && \text{XOR associative} \\
&= L_a^0 \oplus L_b^0 \oplus i\Delta \oplus j\Delta && \text{XOR commutative} \\
&= (L_a^0 \oplus L_b^0) \oplus (i \oplus j)\Delta && i\Delta \oplus j\Delta = (i \oplus j)\Delta, \text{ as } \Delta \oplus \Delta = 0^\kappa \\
&= L_c^0 \oplus (i \oplus j)\Delta = L_c^{i \oplus j} && \text{Definition 4.3.}
\end{aligned}
$$

□

because it requires three things at once: (1) a field, since interpolation divides and every nonzero element must have an inverse; (2) the κ-bit strings as its elements, since pads and labels are such strings; (3) XOR as its addition, so that pads and labels enter and leave the interpolation as the bit strings they are, with no conversion to another representation. The integers modulo 2^κ fail (1), the integers modulo a prime fail (2) and (3), and the field of 2^κ elements, unique up to isomorphism, satisfies all three in its polynomial representation.

[10] Example 4.11 takes a characteristic-2 field.

The condition $\mathrm{lsb}(\Delta) = 1$ makes the two labels of every wire differ in their select bit, so Definition 4.1 holds with $p_w = \mathrm{lsb}(L_w^0)$ and no further arrangement. A NOT gate is also free: the garbler sets $L_c^0 := L_a^1$, so that $L_c^1 = L_a^0$, and the evaluator passes its label through unchanged.

Free-XOR saves 100% on XOR gates over row reduction because the output label is the XOR of the input labels and no table is sent: an XOR gate costs 0 bits against 3κ; AND gates are unchanged at 3κ.

Remark 4.5 (What Free-XOR costs in assumptions). Free-XOR has two costs. (1) Assumption. In plain Yao every label is independent and a random-oracle hash suffices. Under Free-XOR the evaluator sees, for one secret Δ, many values of the form $H(X \oplus \Delta \,\|\, \nu) \oplus \Delta$, where X is a label the evaluator holds and $X \oplus \Delta$ the complementary label it does not: the hash input and the masked plaintext are offset by the same Δ, because the plaintext L_c^1 equals $L_c^0 \oplus \Delta$. Correlation robustness in the sense of Ishai, Kilian, Nissim, and Petrank (CRYPTO 2003) covers $H(X \oplus \Delta)$ alone; Choi, Katz, Kumaresan, and Zhou (TCC 2012, ePrint 2011/510) show that the original Free-XOR proof needs more and define *circular* correlation robustness, circular because the secret Δ appears both inside the hash input and XORed onto its output, under which the oracle $(X, \nu, b) \mapsto H(X \oplus \Delta \,\|\, \nu) \oplus b\Delta$ is indistinguishable from a random function on distinct queries. A random oracle has the property; §4.6 gives the standard-model instantiation. (2) Structure. One exposed label pair (L_w^0, L_w^1) reveals $\Delta = L_w^0 \oplus L_w^1$, and with Δ every held label yields its complement on every wire. Equivocation on a single wire therefore exposes the whole circuit, which is one reason the wide-label reveal of Remark 9.11 never exposes a pair.

4.4 Half-Gates

Free-XOR leaves the AND gates, at three ciphertexts each after row reduction. Half-gates (Zahur, Rosulek, Evans, EUROCRYPT 2015, ePrint 2014/756) bring an AND gate to two ciphertexts, which the same paper proves optimal among garbling schemes that use the hash only through linear operations. The idea is to split the AND by the permute bit of one input; the text takes wire b, an arbitrary choice, since splitting by the permute and select bits of a gives the mirror-image construction.

Write $A_i := L_a^i$, $B_j := L_b^j$, $C_k := L_c^k$ for the labels of an AND gate $c = a \wedge b$, let $r := p_b$ be the permute bit of wire b, known to the garbler, and recall that the evaluator holding B_b knows the select bit $s_b = b \oplus r$. Since $b = r \oplus s_b$ and AND distributes over XOR,

$$a \wedge b \;=\; (a \wedge r) \;\oplus\; (a \wedge s_b). \tag{4.1}$$

Each term is an AND with one operand known to one party, r to the garbler and s_b to the evaluator, and each such *half-gate* costs one ciphertext: a gate with one unknown input has only two rows, one per value of a, and row reduction makes one of them free. The two halves use different tweaks, ν and ν', for instance $2g$ and $2g+1$ for the gate index g; the reason follows the construction.

The generator half-gate (generator being the source's term for the garbler) computes $a \wedge r$: the garbler knows r and garbles the unary function $a \mapsto a \wedge r$. It computes

$$T_G := H(A_0 \,\|\, \nu) \oplus H(A_1 \,\|\, \nu) \oplus r\Delta, \qquad W_G^0 := H(A_0 \,\|\, \nu) \oplus p_a T_G,$$

publishes table entry T_G, and keeps wire label W_G^0 as the half-gate's false label.

The evaluator, holding A_a with select bit $s_a = p_a \oplus a$, computes

$$W_G := H(A_a \,\|\, \nu) \oplus s_a T_G.$$

The evaluator half-gate computes $a \wedge s_b$: the evaluator knows s_b, so the half-gate must fold in A_a exactly when $s_b = 1$.

The garbler computes,

$$T_E := H(B_0 \,\|\, \nu') \oplus H(B_1 \,\|\, \nu') \oplus A_0, \qquad W_E^0 := H(B_0 \,\|\, \nu') \oplus r\,(T_E \oplus A_0),$$

publishes T_E, and keeps W_E^0 as the half-gate's false label.

The evaluator computes

$$W_E := H(B_b \,\|\, \nu') \oplus s_b(T_E \oplus A_a).$$

Combining the two halves, the garbler sets $C_0 := W_G^0 \oplus W_E^0$, so that $C_1 = C_0 \oplus \Delta$; the evaluator outputs $W_G \oplus W_E$. The garbled AND gate is the pair (T_G, T_E): two ciphertexts of κ bits. The garbler keeps W_G^0 and W_E^0 only until the gates fed by c are garbled, since C_0 and $C_1 = C_0 \oplus \Delta$ are the keys for those gates. Two tweaks per gate are needed because they keep the two hash calls distinct even when wires a and b carry the same labels. No sensible circuit contains the gate $a \wedge a$, but under Free-XOR two different wires can share labels: the wire $d = (a \oplus b) \oplus b$ has $L_d^0 = L_a^0 \oplus L_b^0 \oplus L_b^0 = L_a^0$, so a gate $d \wedge a$ hashes the same labels in both halves. With a shared tweak the two published ciphertexts would cancel to $T_G \oplus T_E = r\Delta \oplus A_0$, handing Δ to an evaluator holding A_0 whenever $r = 1$.

Proposition 4.6 (Half-gate correctness)**.** For all $a, b \in \{0,1\}$ the evaluator's output equals $C_{a \wedge b}$.

Proof sketch. Generator half. If $a = 0$ then $s_a = p_a \oplus 0 = p_a$, so the evaluator computes $W_G = H(A_0 \,\|\, \nu) \oplus p_a T_G$, which is the definition of W_G^0. If $a = 1$ then $s_a = p_a \oplus 1$, so $s_a T_G = p_a T_G \oplus T_G$ and

$$W_G = H(A_1 \,\|\, \nu) \oplus p_a T_G \oplus H(A_0 \,\|\, \nu) \oplus H(A_1 \,\|\, \nu) \oplus r\Delta = H(A_0 \,\|\, \nu) \oplus p_a T_G \oplus r\Delta = W_G^0 \oplus r\Delta,$$

the two $H(A_1 \,\|\, \nu)$ terms cancelling. In both cases $W_G = W_G^0 \oplus (a \wedge r)\Delta$.

Evaluator half. First, $W_E^0 = H(B_r \,\|\, \nu')$: for $r = 0$ this is the definition, and for $r = 1$ expanding T_E gives $H(B_0 \,\|\, \nu') \oplus H(B_0 \,\|\, \nu') \oplus H(B_1 \,\|\, \nu') \oplus A_0 \oplus A_0 = H(B_1 \,\|\, \nu')$. If $s_b = 0$ then $b = r$ and $W_E = H(B_r \,\|\, \nu') = W_E^0$. If $s_b = 1$ then $b = r \oplus 1$ and $W_E = H(B_{r\oplus 1} \,\|\, \nu') \oplus H(B_0 \,\|\, \nu') \oplus H(B_1 \,\|\, \nu') \oplus A_0 \oplus A_a = H(B_r \,\|\, \nu') \oplus a\Delta = W_E^0 \oplus a\Delta$, using $A_0 \oplus A_a = A_0 \oplus A_0 \oplus a\Delta = a\Delta$, which is Δ if $a = 1$ and 0^κ if $a = 0$. In both cases $W_E = W_E^0 \oplus (a \wedge s_b)\Delta$.

Summing, $W_G \oplus W_E = (W_G^0 \oplus W_E^0) \oplus \big((a \wedge r) \oplus (a \wedge s_b)\big)\Delta = C_0 \oplus \big((a \wedge r) \oplus (a \wedge s_b)\big)\Delta$. Since AND distributes over XOR, $(a \wedge r) \oplus (a \wedge s_b) = a \wedge (r \oplus s_b)$: if $a = 0$ both sides are 0, and if $a = 1$ both equal $r \oplus s_b$. And $r \oplus s_b = b$, since $s_b = r \oplus b$ by Definition 4.1. Hence $W_G \oplus W_E = C_0 \oplus (a \wedge b)\Delta = C_{a \wedge b}$, which is (4.1) read right to left. □

Authenticity (Definition 2.3) no longer rests on the 0^t marker. The complementary output label is $C_{a\wedge b} \oplus \Delta$, so forging it means learning Δ. Every published value carries the hash of a label the evaluator does not hold, $H(A_{1-a} \,\|\, \nu)$ inside T_G and $H(B_{1-b} \,\|\, \nu')$ inside T_E; under circular correlation robustness those hashes are indistinguishable from random, so the $r\Delta$ and A_0 they cover are masked as by a one-time pad, and the distinct tweaks remove the one cancellation that could strip a mask. Any route to Δ is therefore a guess at a κ-bit label. The full argument is the security proof of ePrint 2014/756.

Half-gates save a third on AND gates over row reduction with Free-XOR because the AND is split into two halves of one ciphertext each: an AND gate costs 2κ bits against 3κ, and, at $t = \kappa$, 75% less than plain Yao.

4.5 The Privacy-Free Half-Gate

In the privacy-free setting of §2.3 the evaluator knows every plaintext wire value, so the permute and select bits are dropped: the evaluator uses the plaintext bit b itself where the half-gate used

s_b, which is the case $r = 0$ of the split, and the condition $\mathrm{lsb}(\Delta) = 1$ is no longer needed. The split (4.1) collapses to $a \wedge b = a \wedge s_b$, the generator half-gate computes the constant 0 and is deleted, and the evaluator half-gate alone remains. Privacy-free garbling is due to Frederiksen, Nielsen, and Orlandi (EUROCRYPT 2015, ePrint 2014/598); the one-ciphertext half-gate form is given in ePrint 2014/756.

End to end, the privacy-free AND gate is as follows. The garbler sets $C_0 := H(B_0 \,\|\, \nu)$, so that $C_1 = C_0 \oplus \Delta$, and publishes the single ciphertext

$$T_E \;=\; H(B_0 \,\|\, \nu) \oplus H(B_1 \,\|\, \nu) \oplus A_0.$$

The evaluator, knowing b and holding A_a and B_b, outputs

$$\begin{cases} H(B_0 \,\|\, \nu) = C_0, & b = 0, \\ H(B_1 \,\|\, \nu) \oplus T_E \oplus A_a, & b = 1. \end{cases}$$

This is Proposition 4.6 with $r = 0$: for $b = 1$, $H(B_1 \,\|\, \nu) \oplus T_E \oplus A_a = H(B_0 \,\|\, \nu) \oplus A_0 \oplus A_a = C_0 \oplus a\Delta = C_a = C_{a \wedge 1}$, and for $b = 0$ the output is $C_0 = C_{a \wedge 0}$ for either a. T_E depends on both labels of b: the single ciphertext is not a stored truth-table row but a gadget correct for every (a, b), built before any input is fixed.

The nonce is not optional. Without ν, two AND gates sharing the input wire b would receive identical output label pairs, and an evaluator holding one output label from each, for gates whose plaintext outputs differ, would recover Δ as their XOR and with it every label in the circuit. This is why the hash must be *tweakable*: keyed by the gate as well as by the label.

The privacy-free half-gate saves 50% on AND gates over half-gates because the generator half computes the constant 0 and is deleted: an AND gate costs κ bits against 2κ, and, at $t = \kappa$, 87.5% less than plain Yao, the final figure of §4.7.

Example 4.7 (Free-XOR and a privacy-free AND with 4-bit labels)**.** Take $\kappa = 4$ and $\Delta = 1011$, so that $\mathrm{lsb}(\Delta) = 1$. The garbler samples $L_a^0 = 0110$ and $L_b^0 = 0010$, giving $L_a^1 = 0110 \oplus 1011 = 1101$ and $L_b^1 = 1001$.

XOR gate $c = a \oplus b$: the garbler sets $L_c^0 = 0110 \oplus 0010 = 0100$, so $L_c^1 = 1111$, and sends nothing. On input $(a, b) = (1, 0)$ the evaluator computes $1101 \oplus 0010 = 1111 = L_c^1$; on $(1, 1)$ it computes $1101 \oplus 1001 = 0100 = L_c^0$.

Privacy-free AND gate $c = a \wedge b$ with nonce ν: suppose the hash, truncated to 4 bits, returns $H(L_b^0 \,\|\, \nu) = 0011$ and $H(L_b^1 \,\|\, \nu) = 0100$ (a random oracle returns arbitrary values; these are chosen for the example). Then $C_0 = 0011$, $C_1 = 0011 \oplus 1011 = 1000$, and

$$T_E = 0011 \oplus 0100 \oplus 0110 = 0001.$$

On input $(1, 1)$ the evaluator holds $A_1 = 1101$ and $B_1 = 1001$, knows $b = 1$, and computes $0100 \oplus 0001 \oplus 1101 = 1000 = C_1$. On input $(0, 1)$ it computes $0100 \oplus 0001 \oplus 0110 = 0011 = C_0$. On $b = 0$ it outputs $H(L_b^0 \,\|\, \nu) = 0011 = C_0$ for either a. One 4-bit ciphertext served all four input pairs, and the XOR gate none.

4.6 The Hash Function

Every construction above uses H for one purpose: to turn a label the evaluator does not hold into a pad that looks random, so that a row or half-gate ciphertext hides what it masks. The proofs assumed a random oracle; a real H must have the property stated next. Since the verifier circuit needs billions of hash calls (§4.7), the second question is whether something cheaper than a hash, a block cipher under a fixed key at roughly a tenth of the cost, has that property.

Definition 4.4 (Tweakable circular correlation robustness)**.** Let $\mathcal{R}$ be a distribution on $\{0,1\}^\kappa$ (Definition 4.2) and $\Delta \leftarrow \mathcal{R}$ secret. H is *tweakable circular correlation robust* for $\mathcal{R}$ if no PPT distinguisher, given oracle access to $(X,\nu,b) \mapsto H(X \oplus \Delta \,\|\, \nu) \oplus b\Delta$ and never querying both $(X,\nu,0)$ and $(X,\nu,1)$, tells it from a random function with non-negligible advantage (Guo, Katz, Wang, Yu, ePrint 2019/074; the tweak ν is the per-gate nonce, the circular part $b\Delta$ is that of Remark 4.5). For the half-gates of §4.4, $\mathcal{R}$ is uniform on the strings with $\mathrm{lsb}(\Delta) = 1$, which Zahur, Rosulek, and Evans write $R \in \{0,1\}^{k-1}1$; for the privacy-free scheme of §4.5 it is uniform on $\{0,1\}^\kappa$.

Remark 4.8 (Why a distribution rather than uniform). What matters about $\mathcal{R}$ is that Δ be hard to guess: if some string had noticeable probability, a distinguisher could guess Δ, query the oracle, and check its guess. Guo et al. therefore state the property for every $\mathcal{R}$ in which no string has probability above $2^{-\rho}$ (min-entropy ρ), the guarantee weakening as ρ falls. Both distributions above are uniform on their support, with $\rho = \kappa$ and $\rho = \kappa - 1$ respectively.

A random oracle has the property, so SHA-256 truncated to κ bits, modelled as in Definition 3.2, is the instantiation the implementation starts from. The fast instantiation is a fixed-key block cipher: with π a fixed public permutation of $\{0,1\}^\kappa$, in practice AES under a fixed public key, whose security lies in the unknown input and in the feed-forward $\pi(K) \oplus K$ being one-way, Bellare, Hoang, Keelveedhi, and Rogaway (IEEE S&P 2013, ePrint 2013/426) garble with $H(X \,\|\, \nu) := \pi(K) \oplus K$ for $K = 2X \oplus \nu$, where $2X$ denotes doubling in $\mathrm{GF}(2^{128})$ (Example 4.11). Guo, Katz, Wang, and Yu (IEEE S&P 2020, ePrint 2019/074) show that this construction falls short of Definition 4.4 and replace it by $H(X \,\|\, \nu) := \pi(\sigma(X) \oplus \nu) \oplus \sigma(X)$, with σ a linear orthomorphism.

Definition 4.5 (Linear orthomorphism)**.** Let $\sigma : \{0,1\}^\kappa \to \{0,1\}^\kappa$. σ is *linear* if $\sigma(X \oplus Y) = \sigma(X) \oplus \sigma(Y)$ for all X, Y, that is, a homomorphism of the group $(\{0,1\}^\kappa, \oplus)$ to itself; a linear bijection is an automorphism of that group. σ is an *orthomorphism* if it is a bijection and the map $X \mapsto \sigma(X) \oplus X$, a second function built from σ, is also a bijection. A *linear orthomorphism* is both; Remark 4.10 shows why the construction needs each property.

Example 4.9 (A linear orthomorphism)**.** Split $X = X_L \,\|\, X_R$ into halves and set $\sigma(X_L \,\|\, X_R) := (X_L \oplus X_R) \,\|\, X_L$.

Linear: XOR acts bitwise, so $(A \,\|\, B) \oplus (C \,\|\, D) = (A \oplus C) \,\|\, (B \oplus D)$,[11] and with this rule and the regrouping of XOR terms, for $Y = Y_L \,\|\, Y_R$,

$$\sigma(X \oplus Y) = \big((X_L \oplus Y_L) \oplus (X_R \oplus Y_R)\big) \,\|\, (X_L \oplus Y_L) = \big((X_L \oplus X_R) \,\|\, X_L\big) \oplus \big((Y_L \oplus Y_R) \,\|\, Y_L\big) = \sigma(X) \oplus \sigma(Y).$$

Bijection: put $\tau(Y_L \,\|\, Y_R) := Y_R \,\|\, (Y_L \oplus Y_R)$. Then $\tau(\sigma(X)) = X_L \,\|\, \big((X_L \oplus X_R) \oplus X_L\big) = X$ and $\sigma(\tau(Y)) = \big(Y_R \oplus Y_L \oplus Y_R\big) \,\|\, Y_R = Y$, so $\tau = \sigma^{-1}$.

Orthomorphism: $\sigma(X) \oplus X = X_R \,\|\, (X_L \oplus X_R)$ is linear by the same computation, and injective, since $X_R = 0$ and $X_L \oplus X_R = 0$ force $X = 0$; an injective map of a finite set to itself is a bijection.

Guo et al. choose this σ because it costs a few CPU instructions, far fewer than a hash compression, saving compute and time.

Remark 4.10 (Why an orthomorphism). A circular query, $b = 1$ in Definition 4.4, is what a half-gate ciphertext with a complement plaintext looks like: in the privacy-free gate of §4.5 an evaluator holding B_1 and A_1 computes $T_E \oplus H(B_1 \,\|\, \nu) \oplus A_1 = H(B_0 \,\|\, \nu) \oplus A_0 \oplus A_1 = H(B_1 \oplus \Delta \,\|\, \nu) \oplus \Delta$, using $B_0 = B_1 \oplus \Delta$ and $A_0 \oplus A_1 = \Delta$, an oracle answer with $X = B_1$. The

[11] Bit i of $U \oplus V$ is $U_i \oplus V_i$. With $|A| = |C| = m$, for $i \le m$ bit i of both sides is $A_i \oplus C_i$, and for $i > m$ it is $B_{i-m} \oplus D_{i-m}$; the sides agree in every position.

question is whether such a value leaks our global wire secret Δ. Write the hash as $H(Y \,\|\, \nu) := \pi(\sigma(Y) \oplus \nu) \oplus \sigma(Y)$ and substitute $Y = X \oplus \Delta$:

$$\begin{aligned} H(X \oplus \Delta \,\|\, \nu) \oplus \Delta &= \pi\big(\sigma(X \oplus \Delta) \oplus \nu\big) \oplus \sigma(X \oplus \Delta) \oplus \Delta && \text{definition of } H \\ &= \pi\big(\sigma(X) \oplus \sigma(\Delta) \oplus \nu\big) \oplus \sigma(X) \oplus \sigma(\Delta) \oplus \Delta && \sigma \text{ linear} \\ &= \pi\big(\sigma(X) \oplus \nu \oplus \sigma(\Delta)\big) \oplus \sigma(X) \oplus \big(\sigma(\Delta) \oplus \Delta\big) && \text{regrouping.} \end{aligned}$$

The distinguisher chose X and ν, so $\sigma(X)$ is known to it; the unknowns are $\sigma(\Delta)$ inside π and the offset $\sigma(\Delta) \oplus \Delta$ on the output. If σ were the identity the offset would be $\Delta \oplus \Delta = 0^\kappa$, leaving $\pi(X \oplus \nu \oplus \Delta) \oplus X$; XORing off X, applying the public π^{-1}, and XORing off $X \oplus \nu$ then yields Δ from a single ciphertext. Guo et al. prove security for any linear permutation σ, with the distinguisher's advantage bounded by $pq/2^\rho + pq/2^{\rho'} + q^2/2^{\kappa+1}$ for p calls to π and q to the oracle, where ρ is the min-entropy of Δ and ρ' that of the offset $\sigma(\Delta) \oplus \Delta$; since $\rho' \le \rho$ with equality exactly when σ is an orthomorphism, the orthomorphism condition is the condition for losing no security, and the identity, with $\rho' = 0$, is the degenerate end. Many linear orthomorphisms exist, multiplication by any $\alpha \notin \{0, 1\}$ in $\mathrm{GF}(2^\kappa)$ among them; the half-swap of Example 4.9 is chosen for costing a few CPU instructions.

One block-cipher call per half-gate ciphertext, against a full hash compression per call, is a reduction in computation only: the same bits are published and the same proofs apply, and it is taken up in the implementation once hash counts matter.

Example 4.11 (Arithmetic in a characteristic-2 field)**.** A field has *characteristic* 2 when $1+1 = 0$ in it, so that every element is its own negative and addition coincides with subtraction; the finite fields of characteristic 2 are exactly the $\mathrm{GF}(2^k)$, and $\mathrm{GF}(7)$ of Example 4.3 is not one, since there $1 + 1 = 2$. In $\mathrm{GF}(8)$ built from the irreducible polynomial $\alpha^3 + \alpha + 1$, whose root α satisfies $\alpha^3 + \alpha + 1 = 0$, so that $\alpha^3 = -\alpha - 1 = \alpha + 1$ since $-1 = 1$ in characteristic 2, an element of $\mathrm{GF}(8)$ is a 3-bit string listing the coefficients of $b_2\alpha^2 + b_1\alpha + b_0$, as an element of $\mathrm{GF}(2^k)$ is a k-bit string. Addition is XOR: $110 + 011 = 101$, since $(\alpha^2 + \alpha) + (\alpha + 1) = \alpha^2 + 1$, the α terms cancelling because $1 + 1 = 0$. Multiplication multiplies the polynomials and reduces: $110 \cdot 011 = (\alpha^2 + \alpha)(\alpha + 1) = \alpha^3 + \alpha^2 + \alpha^2 + \alpha = \alpha^3 + \alpha = (\alpha + 1) + \alpha = 1$, so 110 and 011 are inverses. Doubling, multiplication by $\alpha = 010$, raises every term one degree, a left shift by one bit. If the bit shifted out is 0 the shifted string is the answer: $2 \cdot 011 = 110$. If it is 1 an α^3 term has appeared, which is replaced by $\alpha + 1 = 011$, so the shifted string is XORed with 011: $2 \cdot 110 = 100 \oplus 011 = 111$. In $\mathrm{GF}(2^{128})$ with the polynomial $\alpha^{128} + \alpha^7 + \alpha^2 + \alpha + 1$ the doubling $2X$ above is the same shift, the conditional XOR being with the constant 10000111.

4.7 Cost Summary

Ciphertext bits per fan-in-2 gate, with t the marker width of Definition 3.2:

	AND	XOR
plain Yao (§3.2)	$4(\kappa + t)$	$4(\kappa + t)$
point-and-permute	4κ	4κ
row reduction (GRR3)	3κ	3κ
Free-XOR with GRR3	3κ	0
half-gates	2κ	0
privacy-free half-gates	κ	0

NOT gates cost nothing from Free-XOR on. GRR2 (Remark 4.2) also reaches 2κ bits per AND gate, but charges 2κ per XOR gate as well; the Groth16 verifier circuit has about 10 billion gates of which 2.7 to 3 billion are AND gates (BitVM3 §8.2; BABE, ePrint 2026/065), so about seven in ten gates are free under Free-XOR, which is why GRR3 with Free-XOR and half-gates is used rather than GRR2, and the privacy-free step then halves the AND cost once more.

With $\kappa = t = 128$ a plain Yao gate is 128 bytes and a privacy-free AND gate is 16 bytes, so the garbled object is 16 bytes times the AND count, the figure every size estimate in this document uses (Remark 4.12, §14). Labels are never revealed in bulk: with Free-XOR a wire's pair is L_w^0 and $L_w^0 \oplus \Delta$, and only the one active label per input wire is released (§8).

Remark 4.12 (What BitVM3 actually garbles with, and the size frontier). The BitVM3 paper leaves the garbling scheme abstract: any privacy-free, correct, and authentic garbler suffices, and it fixes a concrete construction only for its cost estimates, namely a Yao-style Boolean garbled circuit for the Groth16 verifier whose non-free-gate count is read as half-gates ($\approx$ 40 GB, one 16-byte ciphertext per non-free gate). The half-gate scheme of this section is therefore what the paper benchmarks. The garbling primitive is, however, the least settled part of the design. An earlier BitVM3 whitepaper used an RSA-based garbling scheme that was broken: a malicious evaluator forges input and output wire labels, breaking authenticity (Futoransky, Larotonda, Barbàra, ePrint 2025/1291), which retracted it. The current size frontier among garbled verifiers is Argo MAC (Eagen and Lai, ePrint 2026/049), which encodes the bit decomposition of a curve point into a homomorphic elliptic-curve MAC, so that group additions become free; for a garbled Groth16 verifier the BitVM3 paper reports an estimated $\sim$ 25 MB (its §8.2), the verifier itself being deferred to later work. The smallest garbled object in the BitVM3 line is Duty-Free Bits (Khambhati, Bhattacharya, Heath, ePrint 2026/476), about 500 KiB against BABE's 22 MiB, but it garbles only the scalar multiplication of BABE's frame rather than the verifier, and is the end object of this document (Part **??**). We teach half-gates because it is what the reference paper and the public implementations use; §14 returns to the RSA break, and Part **??** treats Argo MAC, BABE, Glock, and Duty-Free Bits.

Part II: Committing and Revealing Labels

5 Signature Primitives: What Is Used Where

Three primitives recur, each doing a distinct job; this section aims to map their respective use cases, and subsequent sections aim to address each construction. To begin, a high level summary: *adaptor signatures reveal the proof's garbled labels, Winternitz commits values into script, and cut-and-choose or a SNARK certifies the garbling.*

Primitive	Description	Role
Lamport (§6)	simplest hash-OTS	clean-model GES; pedagogy
Winternitz (§7)	compact hash-OTS bit commitment	committing values to Bitcoin script
GES (§8)	a role: signature to release $\mathtt{En}(e, x)$	proof's input-label reveal
Schnorr adaptor (§9)	deployed realization of GES role	releasing proof labels on-chain

For a garbled SNARK-verifier circuit whose input is the proof π and whose output is the verdict, we can construct four basic categories to describe components:

1. The *input*, the proof labels $\mathtt{En}(e, \pi)$, is released by the GES: a Lamport hash-OTS in the clean model, a Schnorr adaptor signature in deployment (§9.6).
2. The *output*, the verdict, is evaluated off-chain; a wrong verdict is punished by revealing the false-output label through an on-chain *hashlock* (§11.4), not a signature: the label is the preimage of the hash baked into the Bitcoin locking script's hash check.
3. *Setup honesty*, that the garbling itself is faithful, is certified by cut-and-choose (§10.2.1) or a SNARK-of-setup (§10.1); the resulting one-versus-two-SNARK distinction is unpacked in Remark 5.1.

4. *Auxiliary values*, any other values which the protocol must pin to Bitcoin's stateless script, e.g., light-client epoch commitments, transaction-graph connectors, program inputs, are a Winternitz bit commitment (§7). Auxiliary values are any value beyond the input (proof labels revealed by adaptors) and the output (hashlock gated verdict).

Remark 5.1 (Two-SNARK distinction). The proof π is itself an application Groth16 proof, and its *verifier* is the circuit being garbled (§12); this SNARK is always present. It is distinct from the *SNARK-of-setup*, which proves the garbling was done correctly (§10.1). That second SNARK is optional: it appears only in the SNARK-of-setup variant, in lieu of the cut-and-choose option (§10.2.1) which certifies the garbling using a two-party (sub-)protocol with no second SNARK at all. Neither SNARK is the GC encoding information e: e is produced by garbling, and the setup-SNARK (or cut-and-choose) merely certifies it.

Each of the above signing primitives is *one-time*: equivocation (attesting to two separate conditions, revealing two separate preimages), i.e., revealing multiple conflicting commitments, leaks a slashable secret. This single idea is realized in two forms: hash-OTS signature preimage equivocation (Lamport, Winternitz, and the bit commitments built on them; §6.2) and Schnorr nonce reuse (adaptors; §9.4). That self-punishing equivocation is what binds each party to a single value across the transaction graph.

6 Lamport One-Time Signatures

Lamport's one-time signature is the clearest illustration of hash-based one-time signing, which is why we build the GES (§8) on it. Initial public testnet versions of BitVM3 implement *Schnorr adaptor signatures* as the on-chain mechanism to reveal GES labels instead of hash-based one-time signature; the reasons for that choice are discussed in (§9). Despite this, topics in this sections still carry over to the GES construction unchanged.

Lamport's one-time signature[12] is the simplest hash-based one-time signature, built from only a preimage-resistant hash function H, with fixed-length messages signed bit by bit. More compact hash-based generalizations exist, but Lamport is used here for its conceptual clarity. Its importance here is that Lamport *combines with* the garbled circuit of §3 to produce the *Garbled Encoding Extractable Signature* (GES) of §8: each garbled wire's label pair (L_w^0, L_w^1) serves directly as the Lamport secret pair for that input bit, so a Lamport signature on an input $x \in \{0,1\}^\ell$ *is* the garbled encoding $\mathtt{En}(e, x)$. In BitVM3 (§13) this lets the prover release the input-wire labels through a single on-chain signature on the SNARK proof, i.e., the operators (prover) can provide all inpute-wire labels to challenge by posting just a single signature. The Lamport construction below therefore carries over to §8 unchanged, with the wire labels themselves playing the role of Lamport secrets.

Remark 6.1 (Where hash-based one-time signatures still appear in BitVM3). The garbled-label reveal uses Schnorr adaptor signatures (§9), not a hash-based one-time signature.[13] But Lamport and its more compact generalization *Winternitz* (§7, which signs several bits per hash chain) are not absent from the deployed protocols: they remain the general on-chain *bit-commitment* primitive for parts outside the label reveal, for example a light client committing each epoch-boundary block hash and timestamp (roughly every 2016 blocks),[14] and program-input signing in the BitVMX line.[15] Winternitz trades signature size against hashing cost, but

[12] L. Lamport, *Constructing Digital Signatures from a One-Way Function*, SRI International CSL-98 (1979).

[13] The adaptor-signature encoding of the proof is BitVM3 §8.1 (`https://bitvm.org/bitvm3.pdf`); see also Glock (Eagen, Alpen Labs, 2025) and Mosaic (ePrint 2026/812, `https://github.com/alpenlabs/mosaic`). Reference implementation: `BitVM/garbled-snark-verifier` (`src/cut_and_choose/vsss/adaptor.rs`).

[14] The light-client epoch-boundary commitment via an on-chain Lamport/Winternitz signature is described in the BitVM3 paper's light-client construction (`https://bitvm.org/bitvm3.pdf`).

[15] BitVMX is a separate BitVM variant that verifies arbitrary program execution on an emulated

at roughly 25 witness bytes per signed bit[16] it is far heavier on-chain than the adaptor encoding, which is why the label reveal itself moved away from it.

6.1 Construction and Unforgeability

The scheme operates on fixed-length n-bit messages using a hash function $H : \{0,1\}^\kappa \to \{0,1\}^\kappa$, assumed preimage-resistant.

Definition 6.1 (Preimage resistance)**.** H is *preimage-resistant* (one-way) if, for $y = H(u)$ with $u \xleftarrow{\$} \{0,1\}^\kappa$, every PPT adversary given y finds some u' with $H(u') = y$ only with probability $\mathrm{negl}(\kappa)$. Every one-time-signature reduction in this part rests on this single assumption.

Definition 6.2 (Lamport one-time signature)**.** $\texttt{Lamport} = (\texttt{KeyGen}, \texttt{Sign}, \texttt{Vrfy})$ for n-bit messages:

1. $(sk, pk) \leftarrow \texttt{KeyGen}(1^\kappa, n)$: for n-bit message, sample $2n$ independent secrets $s_{i,b} \xleftarrow{\$} \{0,1\}^\kappa$ for $i \in \{0, \ldots, n-1\}$ and $b \in \{0,1\}$. Set $sk := (s_{i,b})_{i,b}$ and $pk := (H(s_{i,b}))_{i,b}$.
2. $\sigma \leftarrow \texttt{Sign}(sk, m)$: for $m = (m_0, \ldots, m_{n-1}) \in \{0,1\}^n$, output $\sigma := (s_{0,m_0}, s_{1,m_1}, \ldots, s_{n-1,m_{n-1}})$.
3. $b \leftarrow \texttt{Vrfy}(pk, m, \sigma)$: parse $\sigma = (\sigma_0, \ldots, \sigma_{n-1})$ and accept iff $H(\sigma_i) = pk_{i,m_i}$ for every $i \in \{0, \ldots, n-1\}$.

Remark 6.2 (Signing is selection, not computation)*.* A Lamport signature is literally a *subset* of the secret key: of the $2n$ secrets it reveals the n selected by the message, one per position (s_{i,m_i}). Nothing is computed at signing time; the signer just picks. Verification is equally cheap: hash each revealed secret and check it against the matching public-key commitment, $H(s_{i,m_i}) = pk_{i,m_i}$. The n unrevealed secrets (the complementary bits) stay secret, which is what keeps the scheme one-time.

Example 6.3 (Lamport on a 3-bit message)**.** Take $n = 3$ and message $m = 101$, so $m_0 = 1$, $m_1 = 0$, $m_2 = 1$.

$\texttt{KeyGen}$ samples six independent κ-bit secrets $s_{i,b}$ for $i \in \{0,1,2\}$, $b \in \{0,1\}$. The public key is the three pairs of their hashes,

$$pk = \big((H(s_{0,0}),\, H(s_{0,1})), \quad (H(s_{1,0}),\, H(s_{1,1})), \quad (H(s_{2,0}),\, H(s_{2,1})) \big).$$

$\texttt{Sign}(sk, 101)$ reveals one secret per position, selected by the bit value:

$$\sigma = (s_{0,1},\ s_{1,0},\ s_{2,1}).$$

$\texttt{Vrfy}(pk, 101, \sigma)$ then checks three hashes at the slots of pk indexed by m:

$$H(\sigma_0) = pk_{0,1}, \qquad H(\sigma_1) = pk_{1,0}, \qquad H(\sigma_2) = pk_{2,1}.$$

All three hold by construction of pk, and the verifier accepts.

CPU (RISC-V style) by optimistic challenge-response, narrowing a dispute to a single instruction step checked on-chain, rather than by garbled circuits; its Winternitz-signed program inputs are treated in ESSPI (Lerner, Jonas, Futoransky, `https://arxiv.org/abs/2503.02772`). For hash-based one-time signatures as stateful-script bit commitments more generally, see `https://www.rootstocklabs.com/blog/exploring-lamport-and-winternitz-signatures-for-stateful-bitcoin-scripts/`.

[16] ESSPI reports $\approx$ 25 witness bytes per signed bit for Winternitz (`https://arxiv.org/abs/2503.02772`); BitVM2 measurements give $\approx$ 26 bytes per input bit (Bitlayer, `https://medium.com/@Bitlayer/bitvm-and-its-optimization-considerations-007da599d8ac`).

Remark 6.4 (Structure of pk). pk is best viewed as an $n \times 2$ array of hashes: row i is the pair $(pk_{i,0}, pk_{i,1}) = (H(s_{i,0}), H(s_{i,1}))$, with $pk_{i,b}$ the hash committed for bit-value b at position i. Both columns are committed at keygen, *before any message is chosen*, because the signer cannot yet predict m_i. The bit m_i then acts at verification time as a column index or bit selector, i.e., denoting which of the two pre-committed hashes $H(\sigma_i)$ must match. The total size of pk is $2n\kappa$ bits.

Theorem 6.5 (Correctness)**.** For every $(sk, pk) \leftarrow \texttt{KeyGen}(1^\kappa, n)$ and every $m \in \{0,1\}^n$,

$$\texttt{Vrfy}(pk, m, \texttt{Sign}(sk, m)) = 1.$$

Proof. $\texttt{Sign}(sk, m)$ returns $\sigma_i = s_{i,m_i}$ for each i. Then $H(\sigma_i) = H(s_{i,m_i}) = pk_{i,m_i}$ by construction of pk, satisfying the verifier's check at every position. □

Theorem 6.6 (One-time unforgeability)**.** For every PPT adversary $\mathcal{A}$, let

$$(sk, pk) \leftarrow \texttt{KeyGen}(1^\kappa, n), \quad m^* \leftarrow \mathcal{A}(pk), \quad (m', \sigma') \leftarrow \mathcal{A}\big(pk, \texttt{Sign}(sk, m^*)\big).$$

Then

$$\Pr\big[\, m' \neq m^* \ \wedge\ \texttt{Vrfy}(pk, m', \sigma') = 1 \,\big] \leq \mathrm{negl}(\kappa).$$

Proof. We argue by contraposition ($P \Rightarrow Q \iff \neg Q \Rightarrow \neg P$, where P is "H is one-way" and Q is "Lamport is unforgeable"): rather than bound the forger directly, we show that a successful PPT forger yields a PPT algorithm inverting H. Since H is preimage-resistant (Definition 6.1), no such inverter exists, forcing the forging probability to be negligible.

Let $\mathcal{A}$ be a PPT forger with success probability

$$\varepsilon(\kappa) = \Pr\big[\, m' \neq m^* \ \wedge\ \texttt{Vrfy}(pk, m', \sigma') = 1 \,\big] \tag{6.1}$$

where κ is the security parameter, i.e. the bit-length of the secrets and the hash.

As $m' \neq m^*$, the messages differ in at least one position, i^*, where $m'_{i^*} = 1 - m^*_{i^*}$. The single sign-query on m^* revealed $s_{i^*, m^*_{i^*}}$ but not the complementary secret $s_{i^*, m'_{i^*}}$, so $\mathcal{A}$ has seen only its image $pk_{i^*, m'_{i^*}}$. Verification of the forgery nonetheless requires

$$H(\sigma'_{i^*}) = pk_{i^*, m'_{i^*}}, \tag{6.2}$$

so any accepting σ'_{i^*} is a preimage under H of a value whose preimage $\mathcal{A}$ was never given.

From the forger $\mathcal{A}$ we build a PPT algorithm $\mathcal{B}$ that inverts H: given a challenge image $y = H(u)$, $u \xleftarrow{\$} \{0,1\}^\kappa$, $\mathcal{B}$ must output a preimage of y. Treating $\mathcal{A}$ as a black-box subroutine (an algorithm it runs internally on inputs of its own choosing, reading off whatever $\mathcal{A}$ outputs), $\mathcal{B}$ draws $(i, b) \xleftarrow{\$} \{0, \ldots, n-1\} \times \{0,1\}$, samples the remaining $2n-1$ secrets and their images honestly, and sets $pk_{i,b} := y$; it then runs $\mathcal{A}$ on pk. To $\mathcal{A}$'s single query m^* it responds honestly when $m^*_i \neq b$, and aborts when $m^*_i = b$ (that query alone would require the unknown u). If $\mathcal{A}$ returns an accepting forgery with $(i^*, m'_{i^*}) = (i, b)$, then $H(\sigma'_i) = y$ and $\mathcal{B}$ outputs σ'_i.

The pair (i, b) is uniform and independent of $\mathcal{A}$'s view, so it coincides with the forgery position (i^*, m'_{i^*}) with probability $1/(2n)$. On that event $m^*_i = 1 - b \neq b$, so $\mathcal{B}$ does not abort and presents $\mathcal{A}$ with a perfectly simulated game. Hence

$$\Pr\big[\, \mathcal{B}(y) \in H^{-1}(y) \,\big] \ \geq\ \frac{\varepsilon(\kappa)}{2n}. \tag{6.3}$$

Preimage resistance bounds the left-hand side by $\mathrm{negl}(\kappa)$, and since $2n$ is polynomial in κ while a polynomial times a negligible function is again negligible,[17]

$$\varepsilon(\kappa) \ \leq\ 2n \cdot \mathrm{negl}(\kappa) \ =\ \mathrm{negl}(\kappa). \tag{6.4}$$

□

[17] Recall (Remark 2.4) that μ is *negligible* if for every constant $c > 0$ there is a κ_0 with $\mu(\kappa) < 1/\kappa^c$ for all $\kappa \geq \kappa_0$. Such a μ times a fixed polynomial p of degree d stays negligible: given c, invoke the bound at exponent $c + d + 1$; since $p(\kappa) \leq \kappa^{d+1}$ for large κ, we get $p(\kappa)\,\mu(\kappa) < \kappa^{d+1}/\kappa^{c+d+1} = 1/\kappa^c$.

Remark 6.7 (Why one-time). If the signer reuses pk to sign a second message $m'' \neq m^*$, every position i where $m_i^* \neq m_i''$ now exposes *both* preimages of pk_i: both $s_{i,0}$ and $s_{i,1}$ are public. With both preimages in hand for any position, an adversary can mint a valid signature on any third message that agrees with the union of revealed bits, breaking Theorem 6.6. The honest mitigation is one fresh keypair per signature; BitVM3 (§13) takes that route. §6.2 turns the same failure mode into a feature, via deliberately reused keys.

6.2 Non-Equivocation via Reused Keys

The same Lamport keypair used twice was the failure mode of §6.1. Reversing perspective turns it into a feature: a verifier who can produce two signatures under the same pk on contradictory messages $m \neq m'$ holds cryptographic evidence that the key was reused, since the two signatures jointly reveal both preimages of pk_i at every position i where $m_i \neq m_i'$. The leaked preimages can be replayed as a fraud-proof witness (see Remark 6.8); BitVM3 (§3.4, "Non-Equivocation Signatures") uses this to slash an operator who tries to claim two contradictory burn-events.

Definition 6.3 (Equivocation evidence)**.** Let $(sk, pk) \leftarrow \texttt{KeyGen}(1^\kappa, n)$, and let σ, σ' be valid Lamport signatures under pk on messages $m, m' \in \{0,1\}^n$ with $m \neq m'$. The tuple (m, σ, m', σ') is an *equivocation pair*[18]; for every position i with $m_i \neq m_i'$ it exposes the secret pair $\{s_{i,0}, s_{i,1}\} = \{\sigma_i, \sigma_i'\}$.

Remark 6.8 (One-bit equivocation suffices). A single position where the two messages disagree is enough to leak both preimages at that position, and one such leaked pair is enough to drive the slashing transaction: the fraud-proof demonstrates *some* reused pair, not a full key recovery. Larger Hamming distance between m and m' does not strengthen the proof, only its forensic value.

Remark 6.9 (Constant-size punishment). Verification of an equivocation pair is one hash equality per leaked position, so the on-chain fraud-proof is constant-size regardless of n. This is what makes the construction attractive when the off-chain commitment graph is large but the on-chain footprint of pk is fixed.

Remark 6.10 (Naive Lamport sizes in BitVM3). The Lamport public key is $2n\kappa$ bits in total: two κ-bit hashes per bit position, n positions. For BitVM3 with $\kappa = 128$ and n the bit-length of the 128-byte Groth16 proof ($n = 1024$), the BitVM3 paper's evaluation prices the Lamport reveal at 74 leaf-script bytes plus a 20-byte witness preimage per bit, roughly 25 kvB on-chain against 2.4 kvB for the adaptor encoding (BitVM3 §8.1). The much-quoted 7.6 MB reveal arises in a different setting: Citrea's post-quantum analysis garbles a verifier for a hash-based proof of ∼14 kB at 80-bit security, and Lamport-signing every bit of that proof yields the 7.6 MB figure.[19] This is large but Taproot-feasible: SegWit/Taproot witness elements carry the bulk, sidestepping both the legacy 520-byte single-push limit and the 10,000-byte legacy script cap that would block a pre-SegWit scriptSig from holding the key.

Remark 6.11 (Discrete-log analogue: extractable one-time signatures). The equivocation-to-slashing mechanism has a discrete-log *analogue*: the same structural idea (equivocation yields an extractable secret, which is the fraud proof) realized with a different primitive. Where a reused Lamport key leaks both hash *preimages*, a Schnorr key reused across two messages under one nonce leaks the *private key* itself (the nonce-reuse of §9.4); this is an *extractable one-time signature* (EOTS). Babylon's Bitcoin Staking uses EOTS for this: a finality provider that

[18] The canonical treatment is Ruffing, Kate, and Schröder, *Liar, Liar, Coins on Fire! Penalizing Equivocation by Loss of Bitcoins*, ACM CCS 2015 (`https://dl.acm.org/doi/epdf/10.1145/2810103.2813686`), which introduces *accountable assertions* (a primitive whose equivocation reveals the party's secret credentials) and *non-equivocation contracts* that turn this into a Bitcoin penalty.

[19] Citrea engineering write-up, `https://www.blog.citrea.xyz/post-quantum-citrea/`; adaptor and Lamport costs from the BitVM3 paper, `https://bitvm.org/bitvm3.pdf`, §8.1.

double-signs two blocks at the same height must reuse its nonce, exposing its key and enabling slashing.[20]

7 Winternitz One-Time Signatures

Winternitz is the compact generalization of Lamport (§6). Lamport spends a secret *pair* per message bit and reveals one of the two; Winternitz instead signs w bits at a time using a single hash chain (Def. 7.1) per w-bit digit. The digit's *value* selects a *position* along that chain: to sign a digit d the signer reveals the secret sitting d steps in, $H^d(s)$, and the verifier hashes it the remaining $2^w - 1 - d$ times to reach the public chain tip $H^{2^w-1}(s)$. So with $w = 2$ the digit $d = 3$ (i.e. 11) is signed by revealing $H^3(s)$, which the verifier identifies as position 3 because 0 further hashes already land on the tip. One chain thus covers all 2^w values of the group, trading signature size against the cost of building and walking chains of length $2^w - 1$. Its role in BitVM3, the bit-commitment primitive rather than the label reveal, is introduced in §5; here we give the construction.

Definition 7.1 (Hash chain)**.** A *hash chain* of length ℓ from a seed $s \in \{0,1\}^\kappa$ is the sequence

$$s,\ H(s),\ H^2(s),\ \ldots,\ H^\ell(s),$$

where H^k denotes H applied k times (so $H^2(s) = H(H(s))$), each element the hash of the one before. Moving *forward* from $H^k(s)$ to $H^{k'}(s)$ costs $k' - k$ hash evaluations; moving *backward* would invert H and is infeasible, so $H^k(s)$ determines every later element while hiding every earlier one. The cost of walking a chain is thus the *number* of steps, each a single hash of a fixed κ-bit value, so it grows with the chain length (2^w here), not with any per-hash input size.[21]

7.1 Construction

Fix a Winternitz parameter $w \geq 1$ (the number of message bits signed per chain) and let H be a preimage-resistant hash. We work in base 2^w (i.e. each digit holds one of 2^w values, from 0 to $2^w - 1$). To sign a message m, hash it to a fixed digest $H(m)$ and split that digest into w-bit chunks, reading each chunk as a base-2^w digit. This gives ℓ_1 digits $d_1, \ldots, d_{\ell_1}$, where the count ℓ_1 is how many w-bit chunks the digest occupies ($\ell_1 = \lceil |H(m)|/w \rceil$); each digit $d_i \in \{0, \ldots, 2^w - 1\}$ is signed on its own chain.

Remark 7.1 (Two conventions for the Winternitz parameter)*.* The digit count admits two equivalent expressions. RFC 8391 (XMSS) takes w to be the number of values a digit may hold and sets $len_1 = \lceil 8n / \lg w \rceil$, where n is the digest length in bytes; thus $8n$ is the digest length in bits, $\lg w$ the number of bits per digit, and each chain has length $w - 1$. This section instead takes w to be the number of message bits signed per chain, so digits lie in base 2^w, $\ell_1 = \lceil |H(m)|/w \rceil = \lceil 8n/w \rceil$, and each chain has length $2^w - 1$. The two are related by substituting 2^w for w: RFC 8391's $w = 16$ is this section's $w = 4$. When comparing parameters across sources, the chain length, $w - 1$ versus $2^w - 1$, identifies which convention is in force.

Definition 7.2 (Winternitz one-time signature)**.**

1. `KeyGen`: for each chain i sample a secret $sk_i \xleftarrow{\$} \{0,1\}^\kappa$ and publish the chain end $pk_i = H^{2^w-1}(sk_i)$ (H applied $2^w - 1$ times).

[20] Babylon, *Bitcoin Staking: Unlocking 21M Bitcoins to Secure the Proof-of-Stake Economy* (litepaper), which specifies EOTS and the double-sign slashing mechanism: `https://docs.babylonlabs.io/papers/btc_staking_litepaper(EN).pdf`.

[21] Hash chains originate with Lamport, *Password Authentication with Insecure Communication*, CACM 24(11):770–772 (1981): `https://lamport.azurewebsites.net/pubs/password.pdf`.

2. `Sign`: for each digit d_i reveal $\sigma_i = H^{d_i}(sk_i)$ (walk the chain d_i steps from the secret).
3. `Vrfy`: accept iff $H^{2^w-1-d_i}(\sigma_i) = pk_i$ for every i (walk the remaining steps to the committed end).

Example 7.2 (Toy example: one chain)**.** Take $w = 3$, so one digit ranges over $\{0, \ldots, 7\}$ and a single chain has $2^3 = 8$ nodes joined by $2^3 - 1 = 7$ hashes: the secret $s = H^0(s)$ signs 000, its first hash $H^1(s)$ signs 001, and so on up to the tip $H^7(s) = pk$ signing 111. To sign 011 (digit 3) the signer reveals $H^3(s)$, and the verifier applies H the remaining $7 - 3 = 4$ times to recover pk; every one of the 8 combinations is thus a distinct position on the same chain.

7.2 Checksum and Unforgeability

Remark 7.3 (Checksum). A *checksum* is a small value computed from data so that any change in the data changes the value. It is usually a tool for *error detection*, flagging accidental corruption in storage or transmission. Winternitz instead uses it explicitly for *forgery prevention*: the checksum C below is a signed function of the digits whose value moves opposite to any message digit, so a forger cannot raise one digit without being forced to lower another.

The chains alone are forgeable: since $\sigma_i = H^{d_i}(sk_i)$, anyone can hash forward to $H^{d_i+k}(sk_i)$, a valid signature for the *larger* digit $d_i + k$. To block this the signer appends a *checksum*

$$C = \sum_{i=1}^{\ell_1} \big(2^w - 1 - d_i\big),$$

a single integer, then *positionally encoded* back into base 2^w as ℓ_2 checksum digits $c_1, \ldots, c_{\ell_2}$, where $C = \sum_{j=1}^{\ell_2} c_j \, (2^w)^{\ell_2 - j}$ and each $c_j \in \{0, \ldots, 2^w - 1\}$. The string actually signed is the concatenation of the message and checksum digits,

$$d_1, \ldots, d_{\ell_1} \,\|\, c_1, \ldots, c_{\ell_2},$$

one chain per digit, for $\ell_1 + \ell_2$ chains in total. The verifier does not receive C: it recomputes C from the message digits d_i, re-derives the c_j, and then checks *every* chain, message and checksum alike, by the `Vrfy` rule of Definition 7.2. Recomputation is what gives the checksum its force: the c_j are a fixed function of the message, not free input, so a forger cannot simply supply convenient checksum digits. Incrementing any message digit d_i *decreases* C, which forces *some* recomputed checksum digit c_j to *decrease*, and signing that lower digit means revealing an earlier chain node $H^{c_j - k}(sk)$, i.e. inverting H, which is infeasible. So forgery-by-chain-extension is blocked, and one-time unforgeability reduces to preimage resistance (Definition 6.1), as for Lamport (§6.1).

Example 7.4 (Worked example: message and checksum combined)**.** Take $w = 2$ (base 4, digits 0–3) and a toy 4-bit digest 1101. Split into 2-bit chunks: $11, 01$, so $d_1 = 3$, $d_2 = 1$ on $\ell_1 = 2$ chains. The checksum is $C = (3 - 3) + (3 - 1) = 2$; its maximum is $2 \cdot 3 = 6$, needing $\ell_2 = 2$ base-4 digits, and $C = 2$ encodes as 02_4, i.e. $c_1 = 0$, $c_2 = 2$. The signed string is $3, 1 \,\|\, 0, 2$: four digits, four chains, signature $\big(H^3(sk_1), H^1(sk_2), H^0(sk_3), H^2(sk_4)\big)$. Now attempt a forgery: bump d_2 from 1 to 2 by hashing σ_2 once forward, which is free. The verifier recomputes $C = (3 - 3) + (3 - 2) = 1 = 01_4$, so the forger must drop c_2 from 2 to 1, i.e. produce $H^1(sk_4)$ from $H^2(sk_4)$: a preimage, and the forgery is stuck.

Example 7.5 (Checksum sizing)**.** With $w = 4$ (base 16) and a 256-bit digest, the message occupies $\ell_1 = 64$ chains, each of length $2^4 - 1 = 15$. The checksum is at most $64 \cdot 15 = 960$, which fits in $\ell_2 = 3$ base-16 digits, giving 67 chains in total. Raising w to 8 halves the message chains to 32 (plus 2 checksum) but lengthens each to 255 hashes: the size-versus-time trade of Remark 7.6.

Remark 7.6 (Size vs. time, and the Lamport limit). Larger w packs more bits per chain, giving fewer chains and a shorter signature, at the cost of chains of length 2^w and thus more hashing per sign and verify. At $w = 1$ each chain has length 1 and one secret covers a single bit; this is the bit-granular regime of Lamport (§6), though not identical to it, since the checksum stands in for Lamport's second per-bit secret. The one-time restriction still binds: signing a second message reuses chain secrets and, as in Lamport (Remark 6.7), leaks enough to forge. The GES of §8 is instantiated over the Lamport case for clarity; it carries over to Winternitz with chains in place of secret pairs.

8 Garbled Encoding Extractable Signatures

A *garbled encoding extractable signature* (GES) keys a signature scheme to one specific garbled circuit's input encoding e, so that signing an input x and releasing its garbled encoding $\mathtt{En}(e, x)$ become a single act. It is the construction that *combines* the garbling scheme of §3 with the Lamport signature of §6: the label pair (L_i^0, L_i^1) that §3 assigns to the i-th input wire is reused as the Lamport secret pair for the i-th message bit, so a signature on x *is* the list of input labels $\mathtt{En}(e, x)$. A verifier holding such a signature can extract those labels and run $\mathtt{Ev}$ on them directly, with no separate label-delivery step. In BitVM3 (§13) this is what lets the operator release the input of a garbled SNARK-verifier circuit through a single on-chain signature on the proof π, revealing the labels for π and nothing else.[22]

8.1 Syntax and Security

The instantiation works because Yao's encoder is *projective*: the encoding information splits into one label pair per input wire, and $\mathtt{En}$ selects one label from each pair according to the corresponding input bit.

Definition 8.1 (Projective garbling scheme)**.** A garbling scheme (Definition 2.1) is *projective* if, for a circuit of input length ℓ, the encoding information has the form $e = \big((e_{i,0}, e_{i,1})\big)_{i=1}^{\ell}$ of one label pair per input wire, and the encoder selects componentwise:

$$\mathtt{En}(e, x) = (e_{1,x_1},\, e_{2,x_2},\, \ldots,\, e_{\ell,x_\ell}).$$

Remark 8.1 (Yao is projective). The wire-label construction of §3.1 is projective: e is the input-wire label pairs (Definition 3.1), and $\mathtt{En}$ returns the matching label of each pair (one per input bit). This per-bit structure is the hinge on which the GES instantiation rests, because it lines up one-to-one with Lamport's per-bit secrets (2 committed values per position, one revealed). A non-projective scheme, whose encoding mixes input bits together, would have no such alignment.

Definition 8.2 (Garbled encoding extractable signature)**.** Let $\mathsf{G} = (\mathtt{Gb}, \mathtt{En}, \mathtt{Ev}, \mathtt{De})$ be a projective garbling scheme, f a circuit of input length ℓ, and fix an honestly generated $(F, e, d) \leftarrow \mathtt{Gb}(1^\kappa, f)$. A *garbled encoding extractable signature scheme* for G is a tuple $\mathtt{GS} = (\mathtt{KeyGen}, \mathtt{Sign}, \mathtt{Vrfy}, \mathtt{Extract})$ of deterministic polynomial-time algorithms:

1. $(pk, sk) \leftarrow \mathtt{KeyGen}(1^\kappa, e)$: from the encoding information e, output a key pair. The secret key sk holds the input labels of e; the public key pk commits to them. Determinism is well-defined because e already fixes the labels (the randomness lived in $\mathtt{Gb}$).
2. $\sigma \leftarrow \mathtt{Sign}(sk, x)$: for an input $x \in \{0,1\}^\ell$, output a signature σ.

[22] Definitions in this section follow the BitVM3 paper, §3.3 and Appendix B.4 (Linus Woll et al.), `https://bitvm.org/bitvm3.pdf`.

3. $b \leftarrow \texttt{Vrfy}(pk, \sigma)$: output $b \in \{\text{True}, \text{False}\}$, equal to True iff σ is a valid signature under pk. There is no message argument.

4. $L_x \leftarrow \texttt{Extract}(pk, \sigma)$: from a valid signature σ, output the encoded input L_x that σ reveals.

Remark 8.2 (Why `Vrfy` takes no message). Plain Lamport verifies against a supplied message: $\texttt{Vrfy}(pk, m, \sigma)$ (Definition 6.2). A GES signature is instead *self-describing*. Each component reveals a preimage of one of the two committed hashes at its position, and which of the two it matches *is* the corresponding input bit. Validity is therefore checkable from (pk, σ) alone, and the message is not an input to verification but an output, recovered by `Extract` followed by `De`. The signature does not attest to an externally chosen m; it *is* the encoding of whichever x it commits to.

A secure GES satisfies three properties, stated for an honestly generated e and a fixed input length ℓ.

Theorem 8.3 (Correctness)**.** For every $x \in \{0,1\}^\ell$, with $(pk, sk) \leftarrow \texttt{KeyGen}(1^\kappa, e)$ and $\sigma \leftarrow \texttt{Sign}(sk, x)$,

$$\Pr\big[\,\texttt{Vrfy}(pk, \sigma) = \text{True}\,\big] = 1.$$

Theorem 8.4 (Extractability)**.** For every $x \in \{0,1\}^\ell$, with $(pk, sk) \leftarrow \texttt{KeyGen}(1^\kappa, e)$ and $\sigma \leftarrow \texttt{Sign}(sk, x)$,

$$\Pr\big[\,\texttt{Extract}(pk, \sigma) = \texttt{En}(e, x)\,\big] = 1.$$

That is, every honestly generated signature reveals exactly the garbled encoding of the signed input.

Theorem 8.5 (Unforgeability)**.** For every PPT adversary $\mathcal{A}$, with $(pk, sk) \leftarrow \texttt{KeyGen}(1^\kappa, e)$ and $\sigma^* \leftarrow \mathcal{A}(1^\kappa, pk)$,

$$\Pr\big[\,\texttt{Vrfy}(pk, \sigma^*) = \text{True}\,\big] \le \text{negl}(\kappa).$$

Remark 8.6 (Reading GES unforgeability). The game hands $\mathcal{A}$ only pk, with no signing oracle, and asks for *any* verifying signature. Since each position of pk is a pair of hashes whose preimages are the secret labels, producing a valid σ requires inverting H at every position, which is infeasible under preimage resistance. This is weaker than the EUF-CMA notion of §6, but it is what BitVM3 needs: the encoding stays hidden behind pk until the operator chooses to sign. After one signature on x is public, the relevant guarantee is the one-time unforgeability of §6: a valid signature on $x' \neq x$ needs a label from the unrevealed half, a preimage of pk, and the proof there applies verbatim since the GES is Lamport over the labels. The one-time discipline of §6.1 (one fresh key per proof) then stops a second signature from leaking the complementary labels, and the equivocation view of §6.2 is the same fact read forward, namely that a second signature under one pk is what slashes a cheating operator.

8.2 Lamport-over-Input-Labels Instantiation

The canonical GES is a Lamport signature (§6) whose secrets are not freshly sampled but *taken to be* the input-wire labels of the garbled circuit. To align with Lamport's subscript notation $s_{i,b}$, write the label that §3.1 denoted L_i^b (the bit-b label of the i-th input wire) as $L_{i,b}$ throughout this subsection. Let $H : \{0,1\}^\kappa \to \{0,1\}^\kappa$ be the preimage-resistant hash of §6, which is *not* the dual-key hash $H : \{0,1\}^* \to \{0,1\}^{\kappa+t}$ of Definition 3.2. The scheme:

1. $\texttt{KeyGen}(1^\kappa, e)$: parse the projective encoding $e = \big((L_{i,0}, L_{i,1})\big)_{i=1}^{\ell}$. Set $sk := e$ and $pk := \big((H(L_{i,0}),\, H(L_{i,1}))\big)_{i=1}^{\ell}$.
2. $\texttt{Sign}(sk, x)$: output $\sigma := (L_{1,x_1},\, L_{2,x_2},\, \ldots,\, L_{\ell,x_\ell})$.

3. $\texttt{Vrfy}(pk, \sigma)$: parse $\sigma = (\sigma_1, \ldots, \sigma_\ell)$ and accept iff $H(\sigma_i) \in \{pk_{i,0}, pk_{i,1}\}$ for every $i \in \{1, \ldots, \ell\}$.

4. $\texttt{Extract}(pk, \sigma)$: output σ itself.

The three properties of §8.1 are now immediate. By projectivity (Definition 8.1), $\texttt{Sign}(sk, x) = (L_{i,x_i})_{i=1}^{\ell} = \texttt{En}(e, x)$, so $\texttt{Extract}$ returns $\texttt{En}(e, x)$ verbatim: extractability (Theorem 8.4) holds by construction, with $\texttt{Extract}$ doing no work beyond parsing. Correctness (Theorem 8.3) holds because $H(L_{i,x_i}) = pk_{i,x_i} \in \{pk_{i,0}, pk_{i,1}\}$ at every position. Unforgeability (Theorem 8.5) is Lamport's preimage argument (Theorem 6.6) read per position: a forger given only pk must exhibit a preimage of $pk_{i,0}$ or $pk_{i,1}$ to pass $\texttt{Vrfy}$ at position i, and both are images of secret labels under H.

Example 8.7 (GES over a 2-input AND gate)**.** Reuse the garbled AND gate of §3.3, with input wires a, b taken as positions $1, 2$. The encoding is $e = \big((L_{1,0}, L_{1,1}), (L_{2,0}, L_{2,1})\big)$, so

$$pk = \big((H(L_{1,0}), H(L_{1,1})), \quad (H(L_{2,0}), H(L_{2,1})) \big).$$

Signing the input $x = 10$ (that is $a = 1, b = 0$) reveals one label per position:

$$\sigma = (L_{1,1},\, L_{2,0}).$$

Verification checks $H(L_{1,1}) = pk_{1,1}$ and $H(L_{2,0}) = pk_{2,0}$, both committed slots, and accepts. Extraction returns

$$\texttt{Extract}(pk, \sigma) = (L_{1,1},\, L_{2,0}) = \texttt{En}(e, 10) = L_x,$$

which are the very labels the evaluator fed into $\texttt{Ev}$ in §3.3 (there written (L_a^1, L_b^0)). One signature, and the challenger holds the garbled input ready to evaluate.

Remark 8.8 (Instantiation in BitVM3). In BitVM3-core (§13) the signed input is the SNARK proof, $x = \pi$, so ℓ is the proof bit-length and pk_{GS} is the 2ℓ committed hashes of Remark 6.10. Posting $\sigma_\pi \leftarrow \texttt{Sign}(sk_{GS}, \pi)$ on-chain releases $L_\pi = \texttt{En}(e, \pi)$; any challenger extracts L_π, evaluates the garbled verifier $L_y \leftarrow \texttt{Ev}(GC, L_\pi)$, and decodes $y \leftarrow \texttt{De}(d, L_y)$, posting a fraud proof if $y =$ False. The on-chain predicate is written $\texttt{CheckGS}_{pk}(\sigma)$ for consistency with Bitcoin Script.

Remark 8.9 (Label width is forced by the adaptor realization, not this one). The two GES instantiations react differently to the label width κ. The *adaptor* GES (§9.6) reads a label as a scalar and commits it as $T = [\ell]G$, so a narrow label leaves a kangaroo-recoverable discrete log and κ must be pushed near the field width (Remark 9.6, Remark 9.7). The *Lamport* GES above is indifferent to κ: $\texttt{KeyGen}, \texttt{Sign}, \texttt{Vrfy}, \texttt{Extract}$ treat a label only as a preimage of $H : \{0,1\}^\kappa \to \{0,1\}^\kappa$, so widening κ merely lengthens each secret label and its committed hash, changing no step of the construction and costing nothing beyond the extra bytes. Its security rests on preimage resistance, which is already full at $\kappa = 128$ (recovering a label is a 2^{128} preimage search, with no interval structure for kangaroo to exploit, contrast Remark 9.5); any extra width it carries is defensive slack inherited for the sibling adaptor realization, not a demand of its own. This is why, in code, a Lamport-over-labels GES whose secret type *is* the garbling label needs no edit when that label grows: the hash absorbs any input length, and only the byte count changes.

9 Schnorr Adaptor Signatures

An *adaptor signature* is a Schnorr signature deliberately offset by a secret t: the signer publishes a *pre-signature* $\widetilde{\sigma}$ that does not yet verify under the standard Schnorr equation; whoever holds t can *adapt* it into a normal signature σ that does; and from the pair $(\widetilde{\sigma}, \sigma)$ anyone can *extract* t by subtraction. This turns "publishing a signature on chain" into "atomically releasing a secret

off chain", which is the workhorse of scriptless scripts, atomic swaps, and Lightning channels – without any extra on-chain script. It is also what BitVM3 actually deploys to reveal garbled input labels on-chain: for the 128-byte Groth16 proof, a Lamport reveal (§6) costs roughly 94 witness bytes per proof bit (about 24 kvB: 94×1024 bits of proof, discounted $4\times$ as witness data; the paper reports 25), Winternitz roughly 25, and the adaptor encoding about 8 (one 65-byte signature per 8-bit digit, about 2.4 kvB in total), a 3–10× saving that is why the deployed systems moved to it (BitVM3 §8.1, Remark 6.10). The reveal is not free: even the adaptor encoding is a 65:1 expansion over the raw proof.

The construction adds three algorithms on top of plain Schnorr (recapped below):

- $\widetilde{\sigma} \leftarrow \texttt{PreSign}(sk, m, T)$: pre-signature on message m bound to public statement $T = [t]G$.
- $\sigma \leftarrow \texttt{Adapt}(\widetilde{\sigma}, t)$: inject the witness, producing a real Schnorr signature.
- $t \leftarrow \texttt{Ext}(\widetilde{\sigma}, \sigma)$: recover the witness from any matching pre-/full-signature pair.

9.1 Motivation: Scriptless Scripts

A *scriptless script*[23] enforces a spending condition without expressing it in on-chain script. The carrier is an ordinary signature: the transaction looks like a plain payment, yet completing and publishing that signature also discloses a secret to a designated party. Adaptor signatures are the mechanism. Because the condition lives in the signature algebra rather than in Bitcoin Script, the on-chain footprint is a single signature and the logic (atomic swaps, payment channels, and the BitVM3 label reveal of §13) is not identifiable by observers.

9.2 Schnorr Recap

Fix a group of prime order n with generator G, written additively, so $[k]G$ denotes the k-fold sum of G. The signer holds a private key $x \in \{1, \ldots, n-1\}$ and publishes the public key $P = [x]G$. The construction below is the (R, s) form of Schnorr; it is the baseline that the adaptor variant of §9.3 modifies, and it is implemented from scratch in this repository's `schnorr` crate.

Signing (message m):

1. sample a nonce $k \xleftarrow{\$} \{1, \ldots, n-1\}$ and set $R = [k]G$;
2. compute the challenge $e = H(R \,\|\, P \,\|\, m) \pmod n$, the digest read as an integer before reduction;
3. set $s = k + ex \pmod n$ and output the signature (R, s).

Verification of (R, s) on m:

1. recompute $e = H(R \,\|\, P \,\|\, m)$;
2. accept iff $[s]G = R + [e]P$.

Correctness is the single identity the whole section rests on: expanding $s = k + ex$,

$$[s]G = [k + ex]G = [k]G + [e]\,[x]G = R + [e]P. \tag{9.1}$$

Remark 9.1 (The homomorphism behind the verification identity). Both non-trivial steps of (9.1), $[k+ex]G = [k]G + [ex]G$ and $[ex]G = [e]\,[x]G$, are the single fact that scalar multiplication is a group homomorphism. Let $\varphi : \mathbb{Z}_n \to \mathbb{G}$, $k \mapsto [k]G$, map the additive group of scalars mod the prime order n into the curve group $\mathbb{G}$; it is a homomorphism, and since G generates a prime-order group, an isomorphism onto $\mathbb{G} = \langle G \rangle$ (the whole group is cyclic, generated by G).

[23] The term and construction originate with Poelstra, *Scriptless Scripts* (2016–2017), `https://github.com/apoelstra/scriptless-scripts`.

The homomorphism property $\varphi(a+b) = \varphi(a) + \varphi(b)$ gives the first step, so a sum of scalars becomes a sum of points. The second step is a regrouping of repeated addition: ex copies of G sum as e batches of x copies each, so $[ex]G = [e]([x]G) = [e]P$. Here n is the group *order* (a prime), distinct from the base-field prime over which the curve is defined, and all scalar arithmetic ($k + ex$, ex) is mod n. This is the only structure the section uses; the adaptor of §9.3 and the Taproot tweak of §11.2 (Remark 11.2) reuse the same φ, so shifting or splitting a statement at the scalar level mirrors on the points.

Remark 9.2 (The nonce point is what the adaptor tweaks)*.* A Schnorr signature is checked against a single point: the challenge $e = H(R \,\|\, P \,\|\, m)$ and the equation $[s]G = R + [e]P$ both contain R, so (R, s) passes for that R and no other. The adaptor construction of §9.3 works by choosing *which* point plays this role: the pre-signature is built so that the completed signature is pinned to $R + T$ rather than R, and moving between the two costs exactly the witness t.

9.3 Pre-Signature, Adapt, Extract

An adaptor signature binds a signature to a *statement* $T = [t]G$, where the scalar t is the *witness*. The party producing the pre-signature holds its private key x and the public statement T, but not the witness t.

Definition 9.1 (Schnorr adaptor signature)**.** For a statement $T = [t]G$ and message m:

1. $\widetilde{\sigma} = (R, \widetilde{s}) \leftarrow \texttt{PreSign}(x, m, T)$: sample $k \xleftarrow{\$} \{1, \ldots, n-1\}$, set $R = [k]G$, compute the challenge against the *shifted* nonce $e = H(R+T \,\|\, P \,\|\, m)$, and output $\widetilde{s} = k + ex \pmod{n}$.
2. $\texttt{PreVrfy}(P, m, T, \widetilde{\sigma})$: with $e = H(R + T \,\|\, P \,\|\, m)$, accept iff $[\widetilde{s}]G = R + [e]P$.
3. $\sigma = (R + T,\, s) \leftarrow \texttt{Adapt}(\widetilde{\sigma}, t)$: set $s = \widetilde{s} + t \pmod{n}$.
4. $t \leftarrow \texttt{Ext}(\widetilde{\sigma}, \sigma)$: output $t = s - \widetilde{s} \pmod{n}$.

The subtlety is in `PreSign`: the challenge e is computed against the shifted nonce $R+T$, not R alone. The signer can do this without knowing t, since T is public and $R+T = [k]G+[t]G$. This is what makes the adapted signature verify. Pre-signature and completed signature therefore share the same challenge e; adapting changes only the scalar s, never the hashed point.

Proposition 9.3 (Adaptability and extractability)**.** If $\widetilde{\sigma} = (R, \widetilde{s})$ passes `PreVrfy` for the statement $T = [t]G$ on message m, then $\sigma = \texttt{Adapt}(\widetilde{\sigma}, t)$ is a valid Schnorr signature on m, and $\texttt{Ext}(\widetilde{\sigma}, \sigma) = t$.

Proof. The adapted signature is (R', s) with $R' = R+T$ and $s = \widetilde{s}+t$. Its verification challenge is $e' = H(R' \,\|\, P \,\|\, m) = H(R + T \,\|\, P \,\|\, m) = e$, the same challenge `PreSign` used. Then, using `PreVrfy` (i.e. $[\widetilde{s}]G = R + [e]P$) for the middle equality,

$$[s]G = [\widetilde{s}]G + [t]G = \big(R + [e]P\big) + T = (R + T) + [e]P = R' + [e]P,$$

so (R', s) verifies. Extraction is immediate: $s - \widetilde{s} = (\widetilde{s} + t) - \widetilde{s} = t$. □

9.4 Security Properties

Beyond correctness, a secure adaptor signature scheme satisfies three properties.

1. *Pre-signature adaptability.* Any pre-signature valid under `PreVrfy` for a statement T can be completed into a valid signature given the witness t (Proposition 9.3).
2. *Witness extractability.* From a valid pre-signature and a valid *adapted* signature for the same (m, T), the witness t is recovered (Proposition 9.3). Publishing the completed signature therefore leaks t to anyone holding the pre-signature.

3. *Pre-signature unforgeability.* No PPT adversary, even one that may first obtain pre-signatures on messages of its choice, can produce a pre-signature that passes `PreVrfy` without the secret key x. Since `PreVrfy` is the Schnorr verification equation with the challenge merely hashed over $R + T$ instead of R, forging a pre-signature is essentially forging a plain Schnorr signature; the intuition is that unforgeability rests on Schnorr's, hence on discrete-log hardness, while the statement T only adds the extractable witness. The full formal result also needs Schnorr's *strong* unforgeability and hardness of the statement relation.[24][25]

Remark 9.4 (One-time discipline). As with plain Schnorr, the nonce k must be fresh per signature: two signatures under the same R expose x. Explicitly, for a wire whose two pre-signatures share nonce k (so $R = [k]G$) under key x and message m, with statements T_0, T_1 and challenges $e_b = H(R + T_b \,\|\, P \,\|\, m)$ for $b \in \{0, 1\}$,

$$\widetilde{s}_0 = k + e_0 x, \qquad \widetilde{s}_1 = k + e_1 x \quad \Longrightarrow \quad x = \frac{\widetilde{s}_0 - \widetilde{s}_1}{e_0 - e_1},$$

since k cancels in the difference and both e_b are publicly computable (T_0, T_1, P, m are public). The two *pre-signatures* alone therefore hand any observer the key, before any completion is posted. A construction must consequently give every pre-signature its own nonce, as the wide-label construction of Remark 9.11 does, and punish equivocation through evaluation rather than through key extraction.

9.5 Bridge to GES: Two Realizations of Extractability

The GES of §8 and the adaptor signature above are the same idea in two settings: publishing a signature on-chain releases a payload off-chain. They differ in what the payload is and who can read it.

	payload released	**who extracts**
GES (§8)	$\mathtt{En}(e, x)$, the input labels	any observer of the chain
Adaptor (§9.3)	t, a discrete-log witness	the holder of the pre-signature

In GES the signature *is* the encoding, so extraction is a parse: `Extract` merely reads the labels off the signature, computing nothing; in the adaptor case the signature and its pre-signature jointly *determine* t by subtraction. BitVM3-core uses GES in the paper's clean model (the protocol as specified, instantiated with the hash-based GES of §8.2) but realizes the on-chain reveal with adaptor signatures (§13), so both realizations appear in the deployed protocol.

9.6 Realizing GES with Adaptor Signatures

The deployed reveal is the same GES instantiated differently, not a second scheme. The Lamport GES of §8 releases each input label as a hash preimage; the deployed GES keeps the interface (§8.1) and swaps the release mechanism to the adaptor signature of this section. Since an adaptor signature is a Schnorr signature whose nonce is shifted by a statement $T = [t]G$, and completing it releases the witness t (§9.3), taking each input label to *be* a witness turns "post the claim signature" into "release the encoding."

Definition 9.2 (Adaptor-GES over input labels)**.** Let G be a projective garbling scheme (Definition 8.1) with input-wire label pairs $e = \big((L_{i,0}, L_{i,1})\big)_{i=1}^{\ell}$ (each label read as a scalar), let x be the operator's Schnorr key (§9.2), let m be the Assert transaction, and let the signed input be the proof π.

[24] The statement relation is discrete log (recover t from $T = [t]G$): generically $O(\sqrt{n})$ group operations via Pollard's rho (n the group order), or $O(\sqrt{L})$ by the kangaroo variant when the witness t is known to lie in an interval of width L, so a b-bit witness gives only $\approx b/2$-bit security.

[25] Formal treatment in *Foundations of Adaptor Signatures*, ePrint 2024/1809.

1. `KeyGen`(e): for each input wire i and bit $b \in \{0,1\}$, publish the statement $T_{i,b} = [L_{i,b}]G$, committing *both* labels of the wire (as the Lamport pk_{GS} commits both hashes, §8.2).
2. `Sign`(x, m): pre-sign both statements of every wire, $\widetilde{\sigma}_{i,b} \leftarrow$ `PreSign`$(x, m, T_{i,b})$ (Definition 9.1), and publish them.
3. `Post`(x, m, π): to claim on π, complete only the selected pre-signatures, $\sigma_i \leftarrow$ `Adapt`$(\widetilde{\sigma}_{i,\pi_i}, L_{i,\pi_i})$, and place them in the Assert witness.
4. `Extract`: recover $L_{i,\pi_i} = s_i - \widetilde{s}_{i,\pi_i}$ for every i, i.e. the encoding `En`(e, π).

Remark 9.5 (Why a point, not a hash). Committing a label as $T = [\ell]G$ is a *group homomorphism* $\ell \mapsto [\ell]G$ from the scalar field into the elliptic-curve group, so the label's additive structure is preserved: statements can be shifted ($R + T$) and the witness recovered by scalar subtraction ($s - \widetilde{s} = \ell$). The discrete-logarithm assumption hides ℓ while leaving this arithmetic available. A hash commitment $H(\ell)$ is instead *one-way* with no algebraic structure: it supports only reveal-and-compare, which is why the Lamport GES (§8.2) can *open* a label but never *adapt* it.

Remark 9.6 (Encoding labels as witnesses). The construction reads each label as a scalar, so the map from label bytes to $\mathbb{Z}_n$ must be *injective and invertible*: injective so that an extracted scalar determines a unique label, invertible so that the label is efficiently computable from it, since `Extract` has to return the exact label the evaluator will feed to `Ev`. A hash fails the second condition: restricted to the label set it is collision-free, hence injective, yet one-way. A 128-bit label embeds injectively (it is below n), but then the witness lives in a 128-bit sub-range, and the discrete log of $T = [\ell]G$ is recoverable in $\approx 2^{64}$ by Pollard's kangaroo (Remark 9.7), well under the intended level. Full security therefore requires labels spanning nearly the full scalar range, while injectivity requires every label, read as an integer, to stay below the group order n: a value $\geq n$ would be reduced mod n onto a scalar some smaller label already occupies. Widening the label is a garbling-parameter choice; a cleverer byte-to-scalar map cannot substitute for it, since a deterministic map of the 2^{128} possible labels yields at most 2^{128} possible witnesses, leaving the attacker's search space unchanged. Two samplers meet both requirements; the accompanying implementation uses the second.

1. *Clamping*: force the top byte of each 32-byte label to zero, so every value is below $2^{248} < n$ by construction.
2. *Rejection sampling*: draw all 32 bytes and discard any value $\geq n$; for secp256k1 the discarded slice is a $\approx 2^{-127}$ fraction of the space, so kept labels range over essentially all of $\mathbb{Z}_n$.

Both exclusion rules are public,[26] and both preserve injectivity; they differ in how much range survives. Clamping excludes 255/256 of the space, leaving an interval of width 2^{248} that Pollard's kangaroo (Remark 9.7) searches in $\approx 2^{124}$ steps; rejection sampling leaves width $\approx 2^{256}$, retaining the full $\approx 2^{128}$. The four-bit gap is the price clamping would pay for its one-line simplicity.

Remark 9.7 (Pollard's kangaroo and interval discrete log). *Pollard's kangaroo* (or lambda) method solves discrete log when the exponent is known to lie in an interval of width L, in $\approx \sqrt{L}$ group operations, compared with the $\approx \sqrt{n}$ that a full-range attack (Pollard's rho) costs over a group of order n. Two pseudo-random walks, a *tame* kangaroo started from a known point and a *wild* one from the target $T = [\ell]G$, hop across the interval until their trajectories collide; the accumulated offset at collision then yields ℓ. The consequence used above: interval

[26] Kerckhoffs's principle: a cryptosystem must remain secure when everything about it except the key is public. Here the sampler is part of the published garbling code (challengers re-run it from revealed seeds in cut-and-choose), so only the label values themselves are secret, never the shape of the set they are drawn from. `https://en.wikipedia.org/wiki/Kerckhoffs%27s_principle`

width, not group order, sets a clamped label's security, so a b-bit interval gives only a $\approx b/2$-bit margin (security level in bits). Confining a witness to a narrow interval, as reading a 128-bit label directly as a scalar does, is therefore self-defeating; labels must span close to the full width (Remark 9.6).

Extractability (§8) holds verbatim: completing the selected pre-signatures to post the claim is the *same act* as releasing $\texttt{En}(e, \pi)$, and any challenger holding the public pre-signatures runs `Extract`. Completing *both* statements of a wire would reveal both its labels, letting any challenger encode a mutated π' differing in that wire, which by SNARK soundness is invalid and evaluates to the Disprove preimage (the mechanism developed for the deployed encoding in Remark 9.11); it is also equivocation, punished through evaluation as developed there; the two pre-signatures of a wire must carry distinct nonces, since a shared nonce would leak the operator's key from the published pre-signatures already at setup (Remark 9.4). Correctness is immediate from the adaptor construction; unforgeability rests on the pre-signature unforgeability of §9.4. In deployment the positions are carried under a single public key with `OP_CODESEPARATOR` (§11.4).

Remark 9.8 (Who must hold the signing key). Definition 9.2 signs under the operator's own key x, and this is a real gap, inherited from the paper's underspecified deployment sketch. GS soundness needs *every* valid signature to extract the encoding, even one produced by the holder of sk; the Lamport GES has this unconditionally (a valid signature *is* the preimages; Proposition 9.9), but an adaptor GES under an operator-known x does not: the operator can satisfy each `OP_CHECKSIGVERIFY` with a fresh vanilla BIP340 signature (witness data is covered by no sighash, §11.3), posting a valid Assert that reveals nothing. The repair is a slot key with no surviving signing exponent: generated jointly at setup (a distributed key generation, FROST-style Feldman-verified sharing, cf. Remark 10.12), used only to issue the pre-signatures, its shares then deleted, mirroring the covenant committee's deletion assumption, so that completing a pre-signature with a label is the only way any slot signature can ever exist. The BitVM3 paper asserts the extract-from-any-valid-signature property but does not specify this key arrangement for its adaptor encoding, and its formal extractability definition quantifies only over honestly generated signatures, weaker than what its own soundness proof uses.

Proposition 9.9 (Universal extractability, achieved by the Lamport GES)**.** Let $pk = \big((H(L_{i,0}), H(L_{i,1}))\big)_{i=1}^{\ell}$ be a Lamport-GES public key (§8.2) with H collision-resistant. Then every σ, however produced, with $\texttt{Vrfy}(pk, \sigma) = \text{True}$ satisfies $\texttt{Extract}(pk, \sigma) = \texttt{En}(e, x)$ for some $x \in \{0,1\}^{\ell}$, except with negligible probability.

Proof. Fix a position i. Acceptance requires $H(\sigma_i) = pk_{i,b_i} = H(L_{i,b_i})$ for some $b_i \in \{0,1\}$. If $\sigma_i \neq L_{i,b_i}$, the pair (σ_i, L_{i,b_i}) is a collision for H, contradicting collision resistance. Hence $\sigma_i = L_{i,b_i}$ at every position; setting $x := (b_1, \ldots, b_\ell)$ gives $\sigma = (L_{i,x_i})_{i=1}^{\ell} = \texttt{En}(e, x)$, which is what `Extract` returns. The acceptance predicate ranges over label tuples and nothing else: no signer, honest or malicious, has any satisfying assignment that is not an encoding. □

The adaptor variant fails at this point when a signing exponent survives setup. A slot's consensus check accepts *any* pair (R', s') with

$$[s']G = R' + [e']P, \qquad e' = H(R' \,\|\, P \,\|\, m),$$

and the operator, knowing x, mints one afresh: $k' \xleftarrow{\$} \{1, \ldots, n-1\}$, $R' = [k']G$, $s' = k' + e'x$. This verifies, yet is no completion of either pre-signature: $R' \neq R_i + T_{i,b}$ except with negligible probability, and the challenger's subtraction returns

$$s' - \tilde{s}_{i,b} = (k' - k_i) + (e' - e_{i,b})\, x,$$

a scalar carrying a fresh random nonce, not a label. The acceptance set is parameterized by the free nonce k' and strictly contains the two completions; extractability holds only on that two-element subset, while membership in the set at large requires only x. Deleting a share of x once the pre-signatures are issued (one honest deleter suffices under N-of-N sharing, the covenant committee's own assumption) collapses the *computable* acceptance set to exactly those completions, which restores extraction.

9.7 Wide Labels: Mapping Bits to Digits

Definition 9.2 releases one label per *wire*: 1024 signatures per claim (π 128 bytes, 1024 bits). To reduce the on chain assert transaction costs (data/witness size) deployment instead groups the 1024 input wires into 128 digits of 8 bits and spends one 65-byte completed signature (R', s) per digit (the script gadget carrying the 128 slots: §11.4, Remark 13.4). A signature releases a single scalar, but a digit's assignment selects 8 bit-labels; the gap is closed by releasing a *key* rather than the labels themselves: the labels sit published but encrypted in a lookup row, and the signature's scalar, the *wide label*, unlocks them.

Definition 9.3 (Wide-label reveal, single instance)**.** Fix the Assert transaction and write m_d for slot d's sighash: the BIP342 script-spend digest committing to the entire Assert (inputs and outputs, §11.3), the executing tapleaf, and the opcode position of slot d's preceding `OP_CODESEPARATOR`. The separator position is the only field varying across slots, so there are exactly 128 pairwise-distinct messages, each shared by its slot's 256 candidate pre-signatures (Remark 13.7). For each digit $d \in [128]$ and candidate value $v \in \{0, \ldots, 255\}$ (the 2^8 assignments of the digit's 8 bits) the garbler:

1. *samples the key*: $W_{d,v} \xleftarrow{\$} \mathbb{Z}_n$, with statement $T_{d,v} = [W_{d,v}]G$;
2. *pre-signs slot d against it*: with a fresh nonce $k_{d,v}$ (distinct for every pair (d, v)), $R_{d,v} = [k_{d,v}]G$ and
$$\widetilde{s}_{d,v} = k_{d,v} + e_{d,v}\, x, \qquad e_{d,v} = H\big(R_{d,v} + T_{d,v} \,\|\, P \,\|\, m_d\big),$$
the `PreSign` of Definition 9.1 under the slot key x, with $P = [x]G = pk_{slot}$ the one key every `CHECKSIGVERIFY` of the leaf checks; no operator signature gates the spend, and the slot key's shares are deleted after these pre-signatures are issued (Remark 9.8), so completions are the only signatures that can ever exist. The statement enters through the challenge, so this pre-signature completes only with $W_{d,v}$ itself (Remark 13.6) and verifies only against slot d's message;
3. *locks the labels behind the key*: having garbled the circuit, the garbler holds every bit-label; it derives the symmetric key $H(W_{d,v})$, a hash of the scalar's canonical serialization (any fixed deterministic derivation of the key from $W_{d,v}$ serves, the serialization itself included; the accompanying implementation uses the scalar's little-endian bytes directly), and publishes $\mathrm{row}_{d,v} = \mathtt{Enc}_{H(W_{d,v})}\big(L_{8(d-1)+1,\,v_1}, L_{8(d-1)+2,\,v_2}, \ldots, L_{8d,\,v_8}\big)$, the 8 bit-labels selected by v's bits under that key (worked byte-level in Appendix A.1, Eq. (A.1)); that the rows encrypt genuine labels is part of what setup verification must certify (§10).

The tuples $(T_{d,v}, R_{d,v}, \widetilde{s}_{d,v}, \mathrm{row}_{d,v})$ are published off-chain and are the entire reveal state. At claim time the operator completes one pre-signature per slot, $s = \widetilde{s}_{d,v_d} + W_{d,v_d}$; the on-chain witness carries only these 128 completed signatures (≈2.4 kvB). Each completed signature is itself the leak: any challenger computes $W = s - \widetilde{s}_{d,v_d}$, derives $H(W)$, and decrypts the row to the digit's 8 bit-labels, the flow Example 9.13 works step by step. Setup therefore issues $128 \times 256 = 32{,}768$ pre-signatures per Assert template (under cut-and-choose, one such set per evaluation instance; Remark 10.8). Only under the VSSS sharing of §10.2.2 do the statements not need publishing, being derivable from the Feldman coefficient commitments, so

the quadruple shrinks to the triple $(R, \widetilde{s}, \text{row})$; the single-instance certification routes publish all four.

Remark 9.10 (The slot equation, end to end). One slot, sighash m_d, one candidate. Secret: the slot exponent x (deleted after setup), the nonce k, the key W (operator only). Public: $pk_{slot} = [x]G$, $T = [W]G$, $R + T$ with $R = [k]G$, $\widetilde{s}$, row.

- *Setup publishes:* $e = H\big((R+T).x \,\|\, pk_{slot}.x \,\|\, m_d\big), \quad \widetilde{s} = k + ex.$
- *Script:* $\langle pk_{slot} \rangle$ `CHECKSIGVERIFY`.
- *Witness:* $\big((R+T).x,\ s\big)$.
- *Node:* lifts $(R+T).x$, recomputes the same e, accepts iff $[s]G = (R+T) + [e]\,pk_{slot}$.
- `PreVrfy` (public data): $[\widetilde{s}]G + T = (R+T) + [e]\,pk_{slot}$.
- *Attempts:* $s = \widetilde{s}$ fails, short by T; $s = \widetilde{s} + W$ passes, $[W]G = T$; $R.x$ in the witness hashes a different challenge; a fresh signature needs the deleted x. The accepted scalar without W is the discrete logarithm of T.
- *Reveal:* $W = s - \widetilde{s}$; $H(W)$ opens row (Example 9.13).

Remark 9.11 (Why full-width sampling is what makes this secure). $W_{d,v}$ is sampled uniformly over all $\sim 2^{256} = \big(2^{3.33}\big)^{76.9} \approx 10^{77}$ scalars, represented by 64 hex characters. The discrete log of $T_{d,v}$ is not known to lie in any interval smaller than the full range $[0, n)$, so Pollard's kangaroo (Remark 9.7), whose cost is the square root of the *interval width*, degenerates to the generic $\sim\sqrt{n} \approx 2^{128}$ bound; contrast the clamped per-wire witness of Remark 9.6.

Conversely, committing a scalar to a label is never an issue: W is never reduced or truncated into a label; it enters only as the preimage of the symmetric key $H(W)$ for a 256-bit (or more) hash H, and the bit-labels are sampled independently of W (then concatenated and XORed with the keystream of this hash digest, see App. A.1). Consequently no scalar-to-label map exists whose non-injectivity could induce collisions; the injectivity requirement on a label-to-scalar encoding arises only in the per-wire construction (Remark 9.6), where the label itself is the witness.

Before a claim each $W_{d,v}$ is hidden by discrete log and each row by the cipher; at the claim, a slot signature is consensus-valid only if it completes a pre-signature with exactly the committed scalar (Remark 13.6), so a valid Assert necessarily releases genuine keys. Revealing two values for one digit is equivocation, and it is punished through evaluation rather than key extraction: the two revealed keys decrypt two rows, giving that digit's labels under two bytes, and challengers splice them, encoding a mutated π' differing from π in that digit alone. By SNARK soundness π' is invalid bar negligible probability, so evaluation hands the challenger the false-output label, the Disprove preimage, and the bond is slashed; the sibling completion need never confirm on-chain (the bonded output is spent once), possession of the material alone sufficing, and the nonce-reuse leak of Remark 9.4 plays no role since each of a slot's 256 pre-signatures carries a distinct nonce, a shared nonce leaking the signing key from the published $\widetilde{s}$ already at setup (§9.4).

Remark 9.12 (Statement independence). In a group of prime order every nonzero element is a generator, so any two nonzero elements are related by a unique scalar; in particular, for any pair of statements there is exactly one $\alpha \in [1, n)$ with $T_{d',v'} = [\alpha]\,T_{d,v}$. The relation is exploitable only when α is known, and recovering it from honestly sampled statements is a discrete logarithm. Suppose α is known for some pair. A completion for candidate (d, v) publishes $s = \widetilde{s}_{d,v} + W_{d,v}$, and $\widetilde{s}_{d,v}$ is public from setup, so any observer computes $W_{d,v} = s - \widetilde{s}_{d,v}$ and with it the key of every candidate whose statement bears a known relation to $T_{d,v}$, decrypting each such row. A single signature thus opens, beyond its own row, any related candidate's

labels: splice material (Remark 9.11) without equivocation. With $w = 8$ and 128 slots the setup publishes $N = 128 \cdot 2^8 = 2^{15}$ statements, one per candidate key. A fixed α relates a given pair with probability $1/n \approx 2^{-256}$, so over all $\binom{N}{2} \approx 2^{29}$ pairs the honest collision probability per screened relation is $\binom{N}{2}/n \approx 2^{29-256} = 2^{-227}$. A verifier can screen only relations it can compute: negation ($\alpha = n-1$) by comparing x-coordinates, since T and $-T$ share one; the curve endomorphism $(x, y) \mapsto (\beta x, y)$, which realizes $\alpha = \lambda$ for a known cube root of unity λ modulo n, at one field multiplication per statement; small multiples up to a bound B by double-and-add at BN group operations; and sums, a class beyond scalar multiples, since $T_3 = T_1 + T_2$ implies $W_3 = W_1 + W_2$, with all pairwise sums checked in $N^2 \approx 2^{30}$ operations. Even taken together these cover fewer than $\sim 2^{43}$ of the $n \approx 2^{256}$ possible relations, and under honest sampling the probability that any cheaply recognizable relation exists at all is of the same vanishing order. The exposed party is moreover the garbler alone: a related pair would let a single honest completion open a second row and an honest claim be disproved, while challenger soundness never rests on statement independence.

Example 9.13 (Challenger decoding, one digit)**.** The Assert witness holds 128 signatures. For slot d, from public data alone:

1. read (R', s) at position d (position identifies d: slot sighashes are pairwise distinct, Remark 13.7);
2. identify the value: test $R' = R_{d,v} + T_{d,v}$ over the 256 published candidates; the unique match is v_d, the digit's revealed value;
3. extract the key: $W = s - \widetilde{s}_{d,v_d}$;
4. decrypt: derive the symmetric key $H(W)$ from the extracted scalar's canonical serialization, as the garbler did at setup; $\mathtt{Dec}_{H(W)}(\mathrm{row}_{d,v_d})$ yields the 8 bit-labels of wires $8(d-1)+1, 8(d-1)+2, \ldots, 8d$. The row already *contains* the labels, concatenated at setup; $H(W)$ merely unlocks them (keystream mechanics worked byte-level in Appendix A).

Concatenating over all 128 slots gives exactly $\mathtt{En}(e, \pi)$. The value v_d is revealed *positionally*, by which statement matches; no arithmetic on the scalar encodes it. This flow serves any single-instance setup certification (§10.1, §10.3); under cut-and-choose the extracted scalar is additionally a Shamir share, and interpolation precedes step (iv) per kept instance (§10.2.2).

Remark 9.14 (Anatomy of the 65 bytes). Each completed signature is $32 + 32 + 1$ bytes: the nonce point serialized *x-only*, the scalar s in full, and a sighash-type flag (omitted under `SIGHASH_DEFAULT`, giving 64). X-only serialization is sound because the x-coordinate determines the point up to negation: dropping y loses only its sign. The curve is defined over the prime field $\mathbb{F}_p$, $p = 2^{256} - 2^{32} - 977$ (the base-field prime, distinct from the group order n), by the short Weierstrass equation $y^2 = x^3 + 7$; for any valid x this has exactly the two mirror solutions y and $p - y \equiv -y \pmod{p}$, negatives of one another: $(x, p-y) = -(x, y) = [-1](x, y)$, group negation reflecting a point over the x-axis, so dropping y loses only this one sign bit, which the parity convention restores. As p is odd, y and $p - y$ have opposite parity: for suitable $n, k, \tilde{k} \in \mathbb{Z}_{\geq 0}$, let $p = 2n + 1$, if $y = 2k$ then $p - y = 2(n - k) + 1$, and if $y = 2\tilde{k} + 1$, then $p - y = 2(n - \tilde{k})$. BIP340 fixes the convention of taking the even root y; the verifier (every validating node's script interpreter,[27] and any challenger re-checking off-chain) recomputes y from x and selects it: derive y from x and return point (x, y) if y even, else $(x, p - y)$. The convention is always well defined: it could fail only if the two roots coincided, $y \equiv p - y$, which forces $2y \equiv 0$ and hence $y = 0$ since p is odd; such a point is a $g \in G$ with $g = -g$, so $2g = e$ and

[27] Bitcoin Core v28.0: `CheckSchnorrSignature`, `https://github.com/bitcoin/bitcoin/blob/v28.0/src/script/interpreter.cpp#L1673`, which also handles the 64-versus-65-byte sighash-flag cases, calls `XOnlyPubKey::VerifySchnorr`, `https://github.com/bitcoin/bitcoin/blob/v28.0/src/pubkey.cpp#L232`, whose x-only key parse performs the even-y lift inside libsecp256k1.

g has order two; however, via Lagrange, element orders divide the group order, and secp256k1's order n is a large odd prime, so no such g exists.

The convention also constrains the *signer*: since the verifier reads the stored x-coordinate as the even-y point, the signer must use secrets consistent with that reading. If $R = [k]G$ has odd y it signs with $-k$, whose point is the mirror $(x, p-y)$, and it likewise replaces its key x by $-x$ when $P = [x]G$ has odd y: negating a scalar reflects the point, flipping the parity of y while fixing the x-coordinate. For an adaptor pre-signature the point serialized on-chain is $R + T$, and negating k moves R but not T; the signer instead resamples k until $R+T$ has even y. The accompanying `schnorr`, `adaptor-sig`, and `ges-via-adaptor` crates apply these rules.

Remark 9.15 (One signature, eight labels). A completed adaptor signature releases one scalar of at most 256 bits, but a digit's bit-labels total $8 \times 256 = 2048$ bits: no scalar can carry them. The wide label resolves this by indirection: the label material sits published but encrypted in the row, and the released scalar is only its key.

Remark 9.16 (The digit-width dial). Byte granularity is a tuning choice: w-bit digits need $1024/w$ on-chain signatures but $(1024/w)\,2^w$ wide labels, pre-signatures, and lookup rows (and, under the sharing of §10.2.2, polynomials), so each doubling of w halves the witness while multiplying off-chain state exponentially. $w = 8$ is where the trade balances, the same signature-size-versus-exponential-work trade as Winternitz (Remark 7.6).

9.8 Application: Atomic Swap

Example 9.17 (Cross-chain atomic swap)**.** Alice holds coin A and Bob holds coin B on a second chain, and they wish to exchange atomically: either both transfers happen or neither does. Bob samples a witness t and publishes the statement $T = [t]G$. On each chain a spending transaction is prepared, and each party produces a *pre-signature* on the other's transaction bound to the same T; neither pre-signature is yet a valid signature. Bob, who knows t, adapts the pre-signature that pays him coin A and broadcasts the completed signature, claiming A. The instant it is on-chain, Alice computes $t = s - \widetilde{s}$ from the broadcast signature and her pre-signature (witness extractability); with t she adapts the pre-signature that pays her coin B. One broadcast reveals the witness and the second transfer follows atomically, with no hash-timelock or on-chain contract: the coordination lives entirely in the signatures.[28]

Remark 9.18 (Staggered refund timelocks). Each locked coin also carries a refund branch, spendable by its original owner after a timeout, in case the counterparty goes silent. These two timeouts must *not* share a deadline. The party who claims *second*, using the just-revealed witness t (Alice, on coin B), needs a strictly later refund timeout than the coin claimed *first* (Bob, on coin A). Otherwise a malicious first mover reveals t at the last moment and races his own refund against the honest second claim: if both refunds opened together, Bob could claim A and refund B, taking both coins. Staggering guarantees a window in which only the honest second claim, not a refund, can succeed. This is the same asymmetric-timelock requirement as in hash-timelocked contracts (HTLCs), and the refund branch is the timeout leaf of the Assert/Timeout templates in §11.6.

[28] Adaptor signatures were first formalized as a standalone primitive, with the channel constructions they enable, in Aumayr et al., *Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures* (ePrint 2020/476).

Part III: Protocol and On-Chain Realization

10 Setup Verification

The authenticity property of §2.2 stops a malicious *evaluator* from forging output labels, but says nothing about a malicious *garbler*. Since the operator garbles its own circuit, nothing so far prevents it from garbling one that computes the *wrong* verdict, for instance a circuit that outputs True on every proof. Setup verification closes this gap: it convinces the verifiers that the GC, the committed labels, and the Assert transaction are a faithful garbling of the agreed `SNARK.Vrfy` public circuit. This integrity is achieved through one of the following methods: a *cryptographic* route (garble one copy, prove it correct; §10.1), a *statistical* route (garble many copies, open a random subset; §10.2),[29], and a *hardware* route (garble one copy inside an attested enclave; §10.3).

Correctness (Definition 2.2) states that a garbling (GC, e, d) of a function f is *correct* if $\mathtt{De}\big(d, \mathtt{Ev}(GC, \mathtt{En}(e, x))\big) = f(x)$ for every input x. This is the property the protocol needs of the garbled verifier: that it computes the agreed public $f = \mathtt{SNARK.Vrfy}$, and in particular yields the false-output label exactly on invalid proofs. Correctness holds automatically for an honest garbler (Bellare–Hoang–Rogaway); the difficulty is that the operator garbles its own circuit and could instead commit a corrupt GC which outputs True on every proof.

Additionally, correctness itself cannot be checked directly: it quantifies over all 2^{1024} inputs, and would involve inspecting a garbled circuit by opening its seed, which exposes every wire's label pair and thus destroys its usability for binding evaluation, since a circuit whose full encoding info e is public is no longer secure (see § 12.2, point 4.).

Definition 10.1 (Soundness of setup verification)**.** A setup-verification procedure has *soundness error* ε if, for every (possibly adversarial) garbler, a setup that is *not* correct (Definition 2.2) is accepted with probability at most ε. Informally, an accepted setup is correct except with probability ε.

A verification procedure therefore certifies correctness only *indirectly*: it accepts or rejects a committed garbling without evaluating it on all inputs, and its guarantee is probabilistic, that an incorrect garbling is accepted with probability at most ε. This bound ε is the procedure's *soundness error* (Definition 10.1), and certifying correctness reduces to making it negligible.

Correctness is thus what we want; yet only soundness is what we can aim to obtain, and it is the axis on which the three routes of §10.4 differ: cut-and-choose attains $\varepsilon \approx 2^{-40}$ statistically (tunable via n), a SNARK-of-setup attains a cryptographic (or computational) $\varepsilon \approx 2^{-100}$ to 2^{-120} depending on the proving curve, and a TEE replaces ε with a trust assumption of hardware attestation. Throughout, *integrity* and *faithfulness* are informal synonyms for correctness; we reserve *soundness* for this catch-probability guarantee.

10.1 SNARK or STARK of Correct Setup

This is one of the two methods proposed in the BitVM3 paper. It is a hypothetical cryptographic alternative to the more feasible statistical guarantee of soundness explored in section §10.2.[30]

[29] BitVM3 paper, §4.2, `https://bitvm.org/bitvm3.pdf` (ePrint 2026/933). The deployed cut-and-choose line is developed in Glock (ePrint 2025/1485) and Mosaic (ePrint 2026/812).

[30] Alternative here in regards to the scope of statistical usage rather than mutually exclusive paradigms, given SNARK soundness is itself partly statistical. Groth16's QAP divisibility check is verified at a secret random point, sound by the Schwartz–Zippel lemma, statistical error at most d/q for polynomial degree d and field size $q \approx 2^{255}$, and by a discrete-logarithm-type assumption on the pairing groups, whose $\approx 2^{-100}$ term sets the security level (§10.4); STARKs likewise draw random challenges for FRI's low-degree test. The practical gap is the *size* of

The operator garbles a *single* instance and produces a SNARK (or STARK) attesting that the setup was performed correctly: that GC is a faithful garbling of the agreed circuit under the committed labels, and that pk_{GS} commits to those labels. One proof then certifies the execution of the entire garbling computation that produced GC. Formally, the setup SNARK proves membership in the relation

$$\mathcal{R}_{\text{setup}} = \Big\{ \big(\underbrace{GC,\ \{C_i\}_i,\ h,\ f}_{\text{statement}} ;\ \underbrace{r}_{\text{witness}} \big) \ :\ (GC, e, d) = \texttt{Gb}(1^\kappa, f; r) \wedge C_i = \text{commit}(e_i)\ \forall i \wedge h = H(L_{\text{out}}^{\text{False}}) \Big\},$$

where $L_{\text{out}}^{\text{False}}$ is the false output label of the decoding information d returned by the same `Gb` call, so the published Disprove hashlock h (§11.6) is certified to open exactly for the genuine false-output label; without this conjunct a faithful GC could be proven while an unrelated h makes Disprove unsatisfiable (Remark 13.3, step 3). Here $f = \texttt{Circuit}(\texttt{SNARK.Vrfy})$ is the public verifier circuit (§12), r is the garbling randomness that seeded the labels (and exists within the private part of the witness, from which the operator computes a proof that the SNARK-of-setup verifier accepts, without r itself ever being revealed), and $\{C_i\}$ are the published label commitments (to which pk_{GS} is also bound). A valid proof certifies that GC really decrypts to f under exactly those committed labels.

Concretely, this is a *different* SNARK from the one being garbled (§12): here Groth16 proves the garbling correct, rather than being the circuit that is garbled.

Remark 10.1 (Why this is out of reach today). Note what the witness of $\mathcal{R}_{\text{setup}}$ is: the *entire garbling computation*. The prover must arithmetize `Gb` itself and, inside that arithmetization, re-execute one cipher call per garbled row and hash the full output to match the commitments. The BitVM Alliance verifier circuit has 2.7×10^9 non-free gates and its tables total ≈ 41.2 GB; garbling it *natively* takes 353.7 s (Ryzen 7 7840U, 16 CPU; BitVM3 §8.2). Arithmetizing `Gb` turns each of those rows into $\sim 10^3$–10^4 constraints (arithmetization-dependent; lookup-based systems sit at the low end), a setup circuit of $\sim 10^{12}$–10^{13} constraints; at the $\sim 10^4$–10^5 constraints per second a large prover sustains that is months to years, and the 10^{12}-plus-element witness is terabytes past a single prover's memory. The obstruction is feasibility, not speed. Below, the Cut-and-choose paradigm replaces all this with native re-garbling plus a byte comparison, at the price of 181 instances, 288.4 GB ($= 7 \times 41.2$) kept-set storage, and a 3 h 24 m parallelized setup[31]. That asymmetry, not preference, is why the deployed line (GSV, Glock, Mosaic) chose cut-and-choose.

10.2 Multiple Garblings and Statistical Soundness

Rather than prove one enormous circuit correct, this approach garbles n copies of it and certifies them *statistically*: a random subset is opened and re-derived (cut-and-choose, §10.2.1), and the confidence gained transfers to the copies left closed. The price is that the surviving copies' labels must still reach the chain; naively that is one reveal per survivor, so a second mechanism (verifiable secret sharing, §10.2.2) collapses all of them into a single on-chain value. The two subsubsections are the two halves of one pipeline: *verify* the garblings, then *reveal* the survivors' labels cheaply.

10.2.1 Cut-and-Choose

The operator garbles n independent instances of the same circuit, each from a fresh seed, gates, wiring and topology remain identical, but every label pair for each wire (inputs, outputs and

the error (roughly 2^{-40} for the paper's cut-and-choose parameters versus $\approx 2^{-100}$ here), plus the computational assumptions (pairings and knowledge assumptions for Groth16, hash security for STARKs) that cut-and-choose does without.

[31] BitVM3 §8.2 measured benchmark, `https://bitvm.org/bitvm3.pdf`.

intermediary) is sampled anew, and commits to all of them. A verifier then draws a uniformly random partition

$$[n] = C \sqcup E, \qquad |E| = f, \quad |C| = n - f,$$

one of the $\binom{n}{f}$ equally likely choices: C is the *check set*, E the *evaluation set*.[32] For each instance in C the operator reveals its seed; a challenger deterministically re-garbles it and checks the result against the commitment (Remark 10.2). The instances in E stay closed and carry the real evaluation.

Remark 10.2 (Checking an opened instance). Write the garbler as $\texttt{Gb}(1^\kappa, f; \text{seed})$: all internal randomness is drawn from the seed, so the same seed on the same public circuit reproduces the instance byte for byte, every label and every ciphertext. The garbling code is public. To check an opened instance $i \in C$, a challenger runs $\texttt{Gb}(1^\kappa, f; \text{seed}_i)$ themselves and compares the output against instance i's setup commitments (the ciphertext-hash digest foremost, alongside the label and table commitments).[33] A match proves the instance was honestly garbled; any mismatch aborts the setup. The check needs no semantic understanding of the circuit, not even a notion of what the true and false output labels are: it is reproduction and comparison, which is what makes this verification cheap relative to *proving* the same statement (Remark 10.1).

Remark 10.3 (Soundness of cut-and-choose). A malicious garbler's attack vector requires all the garbled instances in the check set to be honest and the bad instances selected by the verifier to be in the evaluation set. Challengers will evaluate *all* f instances in the garbled circuit after an operator's assert tx is broadcast as the one on-chain reveal extends to every kept instance's labels by the secret sharing of §10.2.2. Therefore a single *correct* garbled circuit in the evaluated set is sufficient and establishes the verdict honestly, and achieves setup soundness and, thus, the attack only succeeds if *every* evaluated instance is bad while the check set stays clean, which forces the bad set to coincide exactly with the evaluation set: probability at most $1/\binom{n}{f}$ (see Prop. 10.4 for a more rigorous argument beyond the intuition of "possible choices of success from all the ways a verifier can build the evaluation set".).

The paper's concrete choice $(n, f) = (181, 7)$ gives $1/\binom{181}{7} \approx 2^{-40}$. (A looser per-instance estimate $(f/n)^k$ is sometimes quoted; Remark 10.5 derives the exact product and shows the heuristic errs in the attacker's favour.) The parameters n and f are derived from the single chosen input, the target $\varepsilon = 2^{-40}$: a conventional *statistical* security level (Remark 10.7). The paper calls $(181, 7)$ a standard parameter choice without stating a derivation; a natural reconstruction: $f = 7$ is fixed by storage (each kept instance costs 41.2 GB forever, and one honest one suffices), and 181 is the smallest n with $\binom{n}{7} \geq 2^{40}$: $\binom{181}{7} \approx 2^{40.03}$ while $\binom{180}{7} \approx 2^{39.97}$.

A tempting intuition runs the other way: open as many instances as possible to "annul the risk". But checking has no protective power of its own; opened instances are burned and contribute nothing at claim time, and the only thing standing between a false claim and the bridge is that at least one *kept* instance is honest, an event of probability $1 - 1/\binom{n}{f}$. Decreasing the ratio of $f : n$ improves an attacker's probability of success: $(181, 7)$ gives 2^{-40}, while opening 180 of 181 gives $1/181 \approx 2^{-7.5}$. The soundness error is exactly the probability of the event $E = B$ under the uniform draw (Prop. 10.4), so it is set by the number of equally likely partitions, $\binom{n}{f}$, and by nothing else. Opening the check set contributes no probability of its own; its role is structural: it makes any bad instance outside E detectable, which is what collapses the attacker's options to the single event $E = B$. Without checking, a bad instance anywhere

[32] The BitVM3 paper states its concrete choice $(181, 7)$ without naming the parameters; we write n for the instance count and f for the evaluation-set size. This f is unrelated to the function f being garbled in the garbling sections of this document. Context disambiguates.

[33] Concretely, in GSV (the BitVM Alliance's `garbled-snark-verifier` reference implementation, §14.2): an instance's 41.2 GB of tables are committed as one digest, an accumulating hash computed over the ciphertext stream as it is generated, and never stored; checking re-streams the regenerated ciphertexts through the same accumulating hash and compares digests. Protocol steps in `docs/gsv_vsss.md` of that repository, `https://github.com/BitVM/garbled-snark-verifier`.

would survive; with it, only a bad set equal to E does. Less risk is bought by garbling a larger n or by moving the split toward the optimum $f \approx n/2$, never by opening more of a fixed n beyond it. As the protocol's requirements include storing the kept instances, f is kept to the minimum meeting the security target (40 bits as per Rmk. 10.7).

Proposition 10.4 (Cut-and-choose soundness, formally)**.** Let $B \subseteq [n]$ be the set of corrupted instances, fixed at commitment time, and let the evaluation set E be drawn uniformly from the f-subsets of $[n]$, independently of B. The attack succeeds iff $E = B$, hence

$$\Pr[\text{undetected false claim}] \;=\; \Pr[E = B] \;=\; \begin{cases} 1/\binom{n}{f} & \text{if } |B| = f, \\ 0 & \text{otherwise.} \end{cases}$$

Proof. If $B \not\subseteq E$, then there exists an index $i \in B$ with $i \in C = [n] \setminus E$. As all instances in the check set C are re-garbled from its own revealed seed and compared against its commitment (Remark 10.2), instance i's mismatch is detected and the setup aborts. If $E \not\subseteq B$, then there exists an index $j \in E \setminus B$. The on-chain reveal extends to instance j's input labels by the secret sharing of §10.2.2, so any challenger computes

$$\mathtt{De}\big(d_j,\, \mathtt{Ev}(GC_j,\, \mathtt{En}(e_j, \pi))\big) = \mathtt{SNARK.Vrfy}(\pi) = \text{False}$$

by correctness (Definition 2.2), recovering instance j's false-output label, and the Disprove witness.

Therefore, given success requires $E \subseteq B$ and $B \subseteq E$, then $E = B$. Since E is uniform over the $\binom{n}{f}$ subset of size f, and B one such a subset when $|B| = f$, $\Pr[E = B] = 1/\binom{n}{f}$; when $|B| \neq f$ the event $E = B$ is impossible and the probability is 0.

The independence of B from E is the commit-then-challenge ordering: the commitments bind B before the partition randomness is drawn: without it the bound is vacuous, since a malicious garbler who knows in advance which instances will be selected into E corrupts exactly those instances and succeeds with probability 1. □

Remark 10.5 (The $(f/n)^k$ heuristic, and its issue). A rough estimate sometimes used for the above Proposition 10.4 treats each of k malformed garbled circuits as independently "hiding" in the evaluation set with probability f/n, giving $(f/n)^k$. By Prop. 10.4, if $k \neq f$ an undetected false claim succeeds with probability 0.

The exact probability that all f bad instances land in E is the without-replacement product

$$\frac{f}{n} \cdot \frac{f-1}{n-1} \cdots \frac{1}{n-f+1} \;=\; \frac{f!\,(n-f)!}{n!} \;=\; \frac{1}{\binom{n}{f}},$$

read as placing the bad instances one at a time: the first must land in one of f evaluation slots among n positions, and each success *consumes* a slot, until the last factor is $1/(n-f+1)$, returning the originally derived result.

Were this heuristic instead to sample with replacement, every factor stays f/n, i.e., as if all f slots were fresh each time, and hence the product is $(7/181)^7 \approx 2^{-32.9}$ against the true $1/\binom{181}{7} \approx 2^{-40.03}$. Resultingly, the heuristic's error overstates the success probability and is thus in the attacker's favour.

Remark 10.6 (More survivors, harder lottery: monotonicity in f). The claim decreasing the ratio $f : n$ improves the attackers success rate in Rmk. 10.3 can be made exact. Writing $P_f = 1/\binom{n}{f}$ for the success probability at survivor count f, first we compare the two splits directly, expanding each as its without-replacement product,

$$P_1 = \frac{1}{181}, \qquad P_7 = \frac{7}{181} \cdot \frac{6}{180} \cdot \frac{5}{179} \cdot \frac{4}{178} \cdot \frac{3}{177} \cdot \frac{2}{176} \cdot \frac{1}{175},$$

$$\frac{P_7}{P_1} = 7 \cdot \frac{6}{180} \cdot \frac{5}{179} \cdot \frac{4}{178} \cdot \frac{3}{177} \cdot \frac{2}{176} \cdot \frac{1}{175} \approx 2^{-32.5};$$

increasing the evaluation set from 1 to 7 multiplies the attacker's odds by $\approx 2^{-32.5}$, a suppression that grows with each extra malformed garbled circuit, which must land in a shrinking evaluation set.

We can generalize this comparison to every pair of splits at once:

$$\frac{P_{f+k}}{P_f} \;=\; \frac{\binom{n}{f}}{\binom{n}{f+k}} \;=\; \prod_{i=1}^{k} \frac{f+i}{n-f-i+1},$$

noting that terms in the product increase in i. Thus every term in the product lies below 1 iff $f+k < n-f-k+1$, i.e. $f+k \le \frac{n-1}{2}$; and when analysing an incremental increase of the evaluation set by $k=1$, we obtain $f+1 < n-f$, i.e. $f < \frac{n-1}{2}$.

The case $f=1$, $k=6$ reproduces the result above. So P_f is strictly decreasing in f until $f = n/2$, where the binomial coefficient peaks. For fixed f, P_f also falls as n grows. The theoretical optimum $f \approx n/2$ (here 90) is nonetheless never deployed: each survivor costs 41.2 GB of permanent storage, so the concrete $(181, 7)$ trades security bits against storage, see the "derived, not chosen" arithmetic of Remark 10.3.

Remark 10.7 (Why 2^{-40}: statistical vs. computational security). The target is a *statistical* soundness level: it bounds a single unrepeatable random event and therefore holds even against computationally unbounded adversaries, whereas a *computational* parameter such as 2^{-128} bounds PPT adversaries (Remark 2.4) limited only by their number of offline computations ($\sim 2^{128}$ hash evaluations), attempts that are repeatable, unobservable, and parallelizable. Forty bits is the benchmark at which the cut-and-choose literature compares protocols,[34] and it suffices here because each cheating attempt is one *Bernoulli trial*: a single experiment with two outcomes, success with probability p and failure with $1-p$, where success is the event $E = B$ of Proposition 10.4 and $p = 1/\binom{n}{f}$. The trial is online (the partition randomness is drawn fresh, after commitment), costs a full n-instance setup, and is publicly attributable on failure, so attempts cannot be repeated offline or concealed. Even k setups succeed at least once with probability $1-(1-p)^k \approx kp$, and the *profit of corruption* $p \cdot V$, for V the bridged value a successful claim would steal, loses to the *cost of corruption* for any plausible V. The per-attempt cost is purely computational, garbling and committing all n instances, and a protocol can raise it further by escrow: an operator bond forfeited on any detected attempt. At $p = 2^{-40}$, even a nominal per-attempt cost puts the break-even V in the trillions.

Because challengers are permissionless, the check/evaluation split cannot be chosen by any single party; a colluding operator would simply steer its bad instances into E. The partition must instead come from a public randomness beacon or a coin-tossing protocol among a committee of verifiers.

Remark 10.8 (Per-instance artefacts; one slot key serves all). Each evaluation instance carries its own artefacts: its own label commitments, hashlock $H(L_i^*)$, pre-signature set, and presigned Assert/Disprove/Timeout templates, so the transaction graph grows linearly in f (the on-chain label reveal itself is collapsed to one by the secret sharing of §10.2.2). The slot key (Remark 9.8), however, can be one key across all instances: instance i's Assert commits instance i's hashlock, so each instance's Assert is a different transaction with a different sighash, and a pre-signature completion for one instance fails `CHECKSIGVERIFY` in any other. The same message-binding argument that pins a signature to its slot (Remark 13.7) pins it to its instance.

[34] Y. Lindell, *Fast Cut-and-Choose Based Protocols for Malicious and Covert Adversaries*, CRYPTO 2013. ePrint 2013/079, `https://eprint.iacr.org/2013/079`: prior works send 125–128 circuits for cheating probability 2^{-40}; Lindell achieves 2^{-s} with s circuits, so 40 suffice. His mechanism ("cheating recovery") engineers the property BitVM3's setting has natively: cheating must corrupt *every* evaluated circuit, not a majority. Pairing $\kappa = 128$ computational with 40 statistical bits is the standard MPC convention (Evans, Kolesnikov, Rosulek, *A Pragmatic Introduction to Secure MPC*, Ch. 2).

10.2.2 Verifiable Secret Sharing of Labels

Cut-and-choose leaves f evaluated instances, each with its own input labels; revealing all f label sets on-chain would cost f separate reveals. Verifiable secret sharing collapses this to one. The primitive is Shamir secret sharing made checkable; the BitVM3 paper writes *VSSS* for it, expanded either as "verifiable Shamir secret sharing" or generically as "verifiable secret sharing scheme."

Definition 10.2 (Shamir secret sharing)**.** To share a secret s so that any $t+1$ shares reconstruct it while t or fewer reveal nothing, the dealer samples a degree-t polynomial over the scalar field $\mathbb{Z}_q$ (q here denotes the group order, the n of §9.2),

$$P(z) = s + a_1 z + \cdots + a_t z^t, \qquad a_1, \ldots, a_t \xleftarrow{\$} \mathbb{Z}_q,$$

so that $P(0) = s$, and gives party i the *share* $P(i)$. Any $t+1$ shares determine P by Lagrange interpolation, hence recover $s = P(0)$; any t shares leave s information-theoretically uniform.

Remark 10.9 (Verifiability, after Feldman). On its own, secret sharing trusts the dealer to hand out genuine shares. Feldman's variant makes them checkable: the dealer also publishes commitments to the coefficients, $C_k = [a_k]G$ for $k = 0, \ldots, t$ (so $C_0 = [s]G$). A holder verifies its share against the committed polynomial by checking $[P(i)]G = \sum_{k=0}^{t} [\, i^k \,] C_k$. A share that passes is a genuine point on the committed P, so a cheating dealer cannot slip in inconsistent shares.[35] The check is not limited to one's own share: the right-hand side is publicly computable for *every* index i, so anyone can verify any share that is later revealed (an opened check-set share, the on-chain reveal), while an unrevealed share is checkable only by its holder, since the check consumes the share value itself.

Certification is route-agnostic about the reveal mechanism (§9.7); in the statistical route the two couple, because one reveal must serve f kept instances. The claim reveals wide labels for only one instance, yet challengers must evaluate all f kept instances. VSSS closes this gap: the labels are arranged so that the one on-chain reveal is the final missing share of a committed polynomial system, from which challengers compute every other kept instance's labels themselves. The construction below is the implemented one (GSV, as cited in Remark 10.2); the Shamir polynomial is instantiated at degree $t = n - f$, tying the reconstruction threshold to the check-set size.

Definition 10.3 (Wide-label sharing)**.** For each digit d and value v of §9.7, the garbler samples a polynomial of degree $n - f$ over $\mathbb{Z}_q$,

$$P_{d,v}(z) = a_0 + a_1 z + \cdots + a_{n-f}\, z^{n-f}, \qquad a_k \xleftarrow{\$} \mathbb{Z}_q,$$

defines the instance keys $W_{d,v}^{(j)} := P_{d,v}(j)$ for every $j \in [n]$, and publishes Feldman commitments $C_k = [a_k]G$ to the coefficients. Instance j's lookup row (Definition 9.3) is keyed by $W_{d,v}^{(j)}$; each instance carries its own rows, since each was garbled with fresh labels. The $128 \times 256 = 32{,}768$ polynomials and their commitments are the entire sharing state; the bit labels themselves appear only inside rows. Of the three certification routes, this is the only one where the published tuple shrinks: the statements are derivable from the coefficient commitments, so the quadruple of Definition 9.3 becomes the triple $(R, \widetilde{s}, \mathrm{row})$, whereas the single-instance routes (§10.1, §10.3) publish all four elements.

Definition 10.4 (Shares and their three acquisition modes)**.** The *share for instance j* is the point $(j, P_{d,v}(j))$. A challenger comes to hold shares three ways:

[35] P. Feldman, *A Practical Scheme for Non-interactive Verifiable Secret Sharing*, FOCS 1987. Overview: `https://en.wikipedia.org/wiki/Verifiable_secret_sharing`.

1. *Opened* (check set): for $i \in C$ the garbler sends $P_{d,v}(i)$ directly; it is accepted only if it lies on the committed polynomial,

$$[P_{d,v}(i)]G = \sum_{k=0}^{n-f} i^k \, C_k.$$

2. *Extracted* (asserted instance j^*): the completed adaptor signature releases $W_{d,v}^{(j^*)} = \mathtt{Ext}(\widetilde{\sigma}, \sigma) = P_{d,v}(j^*)$ (Definition 9.1). The adaptor statement is itself derivable from the coefficient commitments, $T_{d,v} = [P_{d,v}(j^*)]G = \sum_k [\,(j^*)^k\,]\, C_k$, which is what binds each pre-signature to the committed polynomial.

3. *Interpolated* (all remaining kept instances): holding $n - f + 1$ points $\{(x_i, y_i)\}$, Lagrange interpolation

$$P_{d,v}(z) = \sum_i y_i \prod_{m \neq i} \frac{z - x_m}{x_i - x_m}$$

yields the polynomial, evaluated at the remaining indices of E and verified against the same commitment equation before use.

Remark 10.10 (Why degree $n - f$). Opening the check set necessarily publishes $n - f$ shares of every polynomial. Hiding requires that these do not determine the kept instances' labels, forcing degree $\geq n - f$; the single on-chain reveal must then complete reconstruction, forcing degree $\leq n - f$. The degree is exactly the check-set size: the reveal is the tipping point by construction.

Example 10.11 (One digit, five instances)**.** Take $n = 5$, $f = 2$, $C = \{1, 2, 3\}$, $E = \{4, 5\}$, and fix one (d, v), so $\deg P_{d,v} = 3$ and four points determine it. Opening C gives $P(1), P(2), P(3)$: one short, and instances $4, 5$'s wide labels stay hidden only computationally: the Feldman commitments expose $[P(4)]G$ and $[P(5)]G$, so hiding rests on discrete log. The Assert on instance 4 releases $W^{(4)} = s - \tilde{s} = P(4)$; the challenger checks $[P(4)]G = \sum_k 4^k C_k$ and now holds four points. The interpolation is explicit: evaluating the Lagrange basis of $\{1, 2, 3, 4\}$ at $z = 5$,

$$\ell_1(5) = \tfrac{(5-2)(5-3)(5-4)}{(1-2)(1-3)(1-4)} = -1, \quad \ell_2(5) = 4, \quad \ell_3(5) = -6, \quad \ell_4(5) = 4,$$

so

$$P(5) = -P(1) + 4\,P(2) - 6\,P(3) + 4\,P(4) \pmod{q} :$$

a fixed integer combination of known shares, computed without ever writing down P's coefficients. Concretely, $W^{(4)} = P(4)$ and $W^{(5)} = P(5)$ are the two kept instances' keys: deriving $H(W^{(j)})$ and decrypting $\mathrm{row}^{(j)}$ (Definition 9.3, item (iii)) yields each instance's own 8 bit-labels for this (d, v). The on-chain label is itself a share; no distinguished index is involved.

Remark 10.12 (The coding-theoretic view: Shamir as Reed–Solomon). An *error-correcting code* adds redundancy to a message so that the original is recoverable even when some transmitted symbols are lost or corrupted. Reed–Solomon codes realize this with polynomials: the message fixes a low-degree polynomial, the transmitted symbols are its evaluations at fixed points, and since a degree-$(k-1)$ polynomial is determined by any k of its evaluations, the surplus evaluations are the redundancy. The sharing of Definition 10.3 is such a codeword. Encoding a polynomial of degree at most $n - f$ by its evaluations at n fixed points is the Reed–Solomon code of length n and dimension $k = n - f + 1$; the share vector $(P_{d,v}(1), \ldots, P_{d,v}(n))$ is one codeword, and Shamir sharing *is* this encoding (McEliece and Sarwate[36]). The code is maximum-distance-separable: any k symbols determine the codeword, since the $k \times k$ Vandermonde system

[36] R. J. McEliece, D. V. Sarwate, *On Sharing Secrets and Reed-Solomon Codes*, Communications of the ACM 24(9), 1981. `https://dl.acm.org/doi/10.1145/358746.358762`

$P(x_i) = \sum_{j=0}^{k-1} a_j x_i^j$ has determinant $\prod_{i<j}(x_j - x_i) \neq 0$ for distinct evaluation points; and any $k-1$ symbols leave every unseen symbol uniform, since for each candidate secret s exactly one polynomial passes through the $k-1$ points and $(0, s)$. The two halves of Remark 10.10 are these two facts read in protocol order: the opened check set is exactly $k-1$ symbols (hiding), and the on-chain reveal is symbol k (reconstruction), as Example 10.11 works concretely. The kept instances' unrevealed shares are *erasures*, missing symbols at known positions, and Lagrange interpolation is the erasure decoder; their count, $f-1$, is exactly the code's erasure-correction limit $n-k$. Corrupted shares would be *errors*, wrong symbols at unknown positions. These are correctable by the Berlekamp–Welch method: find an error-locator polynomial E of degree e and a polynomial Q of degree at most $n-f+e$ satisfying $Q(x_i) = y_i\,E(x_i)$ at every received point, whence $P = Q/E$; the linear system is determined only while $n \geq k+2e$, bounding correction at $e \leq \lfloor (f-1)/2 \rfloor$ (three at $(n, f) = (181, 7)$). Against a dishonest dealer even that fails: error decoding presumes the symbols came from *some* codeword, which is what a dealer distributing inconsistent shares denies. The Feldman commitments close this gap by *detection* rather than correction: every share is verifiable against the committed polynomial (Remark 10.9), so a bad share is identified and discarded, and the challenger faces only erasures, never errors. The same polynomial toolbox recurs across this stack: FROST-style threshold Schnorr generates its signing key by Feldman-verified Shamir sharing, the very code above; and Groth16 compiles R1CS constraints into a QAP by interpolation, then checks polynomial equality at one random point, sound by Schwartz–Zippel, the same count-the-roots fact that lets k evaluations pin a degree-$(k-1)$ polynomial.

The chain in full: the Assert reveals the asserted instance's wide labels; each is one share, and interpolation as above converts them into the corresponding wide labels of *every* kept instance; each instance's wide labels open that instance's own lookup rows, yielding its own bit-labels for the *same* committed π; challengers then evaluate all f kept circuits, each with its own genuine keys. One on-chain reveal, f evaluations.

Remark 10.13 (Protocol flow). Commit (coefficient, share, row, and circuit commitments) $\rightarrow$ partition $[n]$ and pick the assert instance $j^* \in E$ (in GSV's two-party protocol the evaluator chooses, sound because the chooser is the party the check protects; the permissionless bridge instead draws the partition from the public randomness of §10.2.1) $\rightarrow$ the garbler opens C's seeds and shares, and everything is rechecked against the commitments $\rightarrow$ the Assert releases j^*'s wide labels on-chain $\rightarrow$ extract, interpolate, decrypt rows, evaluate every kept instance; any invalid-output label opens the Disprove hashlock.

10.3 Trusted Execution Environment

A third route replaces cryptographic or statistical certification with hardware. A single instance is garbled *inside* a trusted execution environment (an enclave, e.g. an AWS Nitro instance), and the enclave's remote *attestation*, a signed measurement of the exact code that ran, stands in for cut-and-choose or a setup proof: verifiers accept the garbling as honest because they trust the measured code and the hardware root of trust that signed the measurement, not because they re-derived or proved anything. This removes both the n-fold storage of the statistical branch and the prover cost of the cryptographic one, at the price of a hardware trust assumption (enclave integrity, no side channels, an uncompromised manufacturer key). The Lombard BSA redesign (§14.3) is a concrete instance, pairing enclave attestation with ECIES-sealed labels; it is a genuinely different trust model, not an instantiation of BitVM3-core.

10.4 Comparison of the Three Routes

A garbled circuit can be *checked* or *used*, but not both: verifying it means opening it, which exposes both labels of every wire and burns it for any binding evaluation. The three routes escape this differently: the cryptographic route *proves* correctness without opening, the statistical

route opens *copies* it can afford to destroy, and the hardware route has an enclave *attest* that the single unopened instance was honestly produced. All three routes exist to protect *depositors and challengers* from a malicious *garbler*; the garbler itself needs none of them, since it knows its own garbling is honest.

	cryptographic (SNARK-of-setup)	**statistical** (cut-and-choose)	**hardware** (TEE)
garbled circuits	1	n copies (e.g. 181)	1
off-chain cost	one garbling + one proof	$n\times$ garbling and storage	one garbling
certified by	a proof over the garbling relation	opening a random subset	enclave attestation
soundness error ε	computational; DL security of the proving curve: $\approx 2^{-120}$ (BLS12-381) or $\approx 2^{-100}$ (BN254)	statistical, $1/\binom{n}{f} \approx 2^{-40}$	none: conditional trust
trust assumptions	the SNARK's (setup, hardness)	the garbling scheme only	enclave integrity, manufacturer key
status	infeasible today (Remark 10.1)	deployed (GSV, Glock, Mosaic)	alternative model (§14.3)

The three soundness errors are not of one kind, and each is derived differently. Cut-and-choose's is *statistical*: it is exactly $1/\binom{n}{f}$, the chance the random check set misses every planted bad instance, which for $(n, f) = (181, 7)$ is $\approx 2^{-40}$ (Remark 10.3); it is tunable, shrinking as n grows. The SNARK's is *computational*: Groth16 knowledge-soundness reduces (in the algebraic group model) to a q-type discrete-log assumption in the pairing groups, so ε is the cost of the best attack on the curve, $\approx 2^{-120}$ for BLS12-381 ($\approx 2^{-100}$ for BN254). It is *not* the Schwartz–Zippel error of the QAP divisibility check, whose statistical component ($\approx d/q \approx 2^{-225}$ for degree $d \approx 2^{30}$ and field size $q \approx 2^{255}$) is far smaller and not the binding term. A TEE gives no probabilistic ε: cheating probability is 0 conditional on honest hardware and an uncompromised attestation key, a trust assumption rather than a bound. The ordering inverts the deployment choice: the statistical route is the *weakest* (2^{-40}) yet is what ships, because a cheap 2^{-40} check beats a 2^{-100} proof that is infeasible to produce (Remark 10.1).

11 Bitcoin Transactions and the BitVM3 Transaction Graph

Everything so far has been off-chain cryptography. This section covers the on-chain half: the Bitcoin transactions that lock the operator's bond and either release it on an unchallenged claim or slash it on a disproof. It is also the theory backing the Rust implementation of the Assert/Disprove graph (`rust-bitcoin`).

11.1 Transaction Anatomy

A Bitcoin transaction consumes *inputs*, each referencing a previous unspent transaction output (UTXO) by its txid and output index (*vout*), and creates new *outputs*, each locking an amount under a script.[37] Since SegWit,[38] the unlocking data (signatures, script preimages) lives in

[37] A Bitcoin *address* is just a locking script: it encodes, in a standardised human-transferable form, the variable data of one of several accepted script templates (a public-key hash for P2PKH/P2WPKH, a script hash for P2SH, or a witness program for P2WSH/P2TR), which a wallet expands back into the actual `scriptPubKey` opcodes. The encoding (Base58Check for legacy, Bech32/Bech32m for SegWit and Taproot) folds in a version prefix and a checksum, so a mistyped address fails locally rather than sending coins astray. The address itself never appears on-chain; only the expanded script does, and it is that script the network's consensus rules enforce.

[38] Segregated Witness, the 2017 Bitcoin soft-fork upgrade that moved the unlocking data out of the input into a separate witness field committed outside the txid, fixing transaction malleability and pricing witness bytes at

a separate *witness* field rather than in the input's `scriptSig`; this removes transaction malleability and discounts witness bytes, which is what makes BitVM3's large witnesses (a revealed garbled label, a per-digit adaptor signature) affordable. A transaction is named by its *txid*, a double-SHA-256 over its non-witness serialization. The txid is handled internally in little-endian byte order but displayed by block explorers big-endian; the two are hex-byte reversals of one another.[39]

11.2 Taproot

A Taproot output (BIP341) locks coins to a single tweaked public key Q, built from an internal key P and the Merkle root m of a *taptree* of alternative spending scripts, where each script is a *leaf* and m the single hash they all compress into (§B), via the tweak

$$Q = P + t \cdot G \text{ where tweak } t = H(P \,\|\, m).$$

The scalar t is *public*, recomputable by anyone from P and m and commits both the public key and the entire script tree into one point Q, so a single output *multiplexes* many spending conditions yet reveals only the branch taken, i.e. the many scripts sit as leaves under the one root m, not many tweaks t satisfying the relation; for fixed P the root fixes a single t and hence a single Q (see Rmk. 11.2).

A UTXO locked to a Taproot public key Q can be spent in one of two ways (and not with two separate keys). Firstly, the *key path* produces one BIP340 Schnorr signature verifying under Q: this needs the discrete log $q = p + t$ (the map $k \mapsto [k]G$ is a group homomorphism, so $[p + t]G = [p]G + [t]G = P + tG = Q$), and since t is public it needs only the internal secret p. It reveals nothing about the scripts, on-chain indistinguishable from an ordinary payment. Secondly, the *script path* signs nothing under Q: it reveals one *tapleaf* (a script and its leaf version) together with a *control block* (the internal key P and a Merkle authentication path to get from a leaf to the root), then the network recomputes m from the leaf and path and checks the tweak equation to confirm the leaf was committed, i.e., checking the elliptic curve point addition of these two points ($P + tG$) is Q, and not a discrete-log proof of Q, after which all validating nodes then execute the leaf, whose own condition (a signature under some *other* key, a hashlock, a timelock, or in general any predicate expressible in Bitcoin Script opcodes) is what actually authorises the spend. BitVM3 uses the script path for the challenge and timeout branches, and the cheaper key path for cooperative settlement.

Remark 11.1 (The control block). The control block is a Merkle *proof of membership* for one leaf: the sibling hashes needed to climb from that leaf to the root, plus the key material to confirm that root is the one Q commits to. It is *per leaf*, not shared and not the root itself. BIP341 fixes its length at $33 + 32m$ bytes, where m is the leaf's depth: the first byte holds the leaf version and the parity of Q's Y-coordinate, the next 32 bytes are the internal key P, and the remaining $32m$ bytes are the Merkle authentication path: one sibling hash per level from the leaf up to the root. The verifier hashes the revealed leaf, folds in the sibling hashes to recompute the root (children are sorted before each `TapBranch` hash, so no left/right flags are needed), and accepts iff $Q = P + \text{hash}(P \,\|\, \text{root})\, G$. Because the path differs per leaf, each spendable leaf carries its own control block. A taptree may commit up to 2^{128} leaves (depth 128), a vast space of alternative spending scripts locked under one Q at setup; a spend still opens exactly *one* of them, and the control-block size tracks the tree *depth* ($\log_2$ of the leaf

one weight unit against four for non-witness bytes, so witness data counts only a quarter toward the block-weight limit.

[39] Little-endian is the native word order of the x86 hardware Satoshi built on, so a 256-bit hash could be read straight off as an integer for the proof-of-work target comparison with no byte-swap. The big-endian display was layered on afterwards only for human readability (largest byte leftmost); see `https://learnmeabitcoin.com/technical/general/byte-order/`.

count), not the count itself. BitVM3's Assert output is deliberately minimal, a two-leaf tree ($m = 1$): each control block is 65 bytes with its single path element the *other* leaf's hash, and a spend takes one of just two branches (Disprove or Timeout). The large combinatorial space in BitVM3 lives not in this output taptree but in the Assert *input*, whose witness commits the garbled proof labels across many per-digit signatures (§13.2).

Remark 11.2 (Why the tweak binds). At a glance, one can misread $Q = P + H(P \,\|\, \text{root}) \cdot G$ as if many roots (hence many script trees) could satisfy it for one fixed Q, so the output commits to nothing in particular. The opposite holds: once a spender reveals (P, root), the tweak $t = H(P \,\|\, \text{root})$ is *deterministic* and $Q = P + t\,G$ is a single point. As group elements this is rigid: $tG = Q - P$ is the *unique* curve point solving the equation, and since G generates a prime-order group $t \mapsto tG$ is a bijection, so exactly one scalar t matches it. For fixed P the map $\text{root} \mapsto Q$ is therefore a *function*, so distinct roots give distinct Q (bar a negligible tweak hash collision). The many taptrees that exist in the abstract are not many trees openable against *one* Q; only the committed one opens.

Binding closes the two ways an attacker might spend Q through a script s that was never committed. *(i) Fix P, hunt a root.* Find a $\text{root}' \neq \text{root}$ whose tree contains s yet still yields Q; this needs $H(P \,\|\, \text{root}') = H(P \,\|\, \text{root})$, a tap-hash collision (or second preimage). *(ii) Vary the key too.* Pick a scalar t' freely and set $P' = Q - t'\,G$: the equation $Q = P' + t'\,G$ holds by construction, but a script-path spend must also exhibit a root' whose tree contains s with $H(P' \,\|\, \text{root}') = t'$, a hash preimage problem; and no discrete log of P' is known, so the *key path* is dead. Either route is a hash collision or preimage, not a spend. (The Merkle machinery the root relies on is in §B; removing route (ii) outright via a NUMS key is Remark 11.3.)

Remark 11.3 (NUMS internal keys). A *nothing-up-my-sleeve* (NUMS) internal key turns route (ii) of Remark 11.2 into a feature. Picking P with no known discrete log (e.g. a hash-to-curve point) makes the key path permanently unspendable, forcing every spend through a committed leaf. This does not loosen the commitment: it *removes* the key-path escape, leaving the collision barrier as the only attack surface, so NUMS makes Q strictly harder to spend illegitimately. Variants of BitVM3 puts a NUMS key on the Assert output so its two leaves (Disprove, Timeout) are the *only* ways it can move.

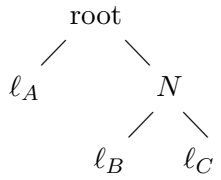
For secp256k1, the curve Bitcoin uses, the group order $|E(\mathbb{F}_p)| = n$ is prime, so Lagrange's theorem gives every point other than the identity order exactly n and any such point generates the whole group: the cofactor is 1 and every point on the curve is a multiple of G. An $x \in \mathbb{F}_p$ for which $x^3 + 7$ is a square in $\mathbb{F}_p$ therefore lifts to a point of $E(\mathbb{F}_p)$, the finite group of $\mathbb{F}_p$-rational points, that requires no further check. This is a property of that curve, not of elliptic curves in general. When the curve order is composite, say $N = hr$ with r prime and $h > 1$, this argument fails: point orders may be any divisor of N, so points of small order exist, and a hash-derived point generically carries a small-order component alongside its component in the order-r subgroup. Constructions on such curves, for instance sect233k1[40] must therefore multiply the lifted point by h or otherwise verify subgroup membership.

The lift succeeds for about half of all $x \in \mathbb{F}_p$, and the count is exact rather than heuristic. The question is whether $a := x^3 + 7$ has a square root in $\mathbb{F}_p$, which depends only on which field elements are squares. If two field elements satisfy $u^2 = v^2$ then $(u - v)(u + v) = 0$, and a field has no zero divisors, so $v = \pm u$; for $u \neq 0$ and odd p these are two distinct roots. The $p - 1$ nonzero elements of $\mathbb{F}_p$ therefore pair up as $\{u, -u\}$, each pair squaring to one value, so exactly $(p - 1)/2$ nonzero elements are squares, the *quadratic residues*; the subtracted 1 is the element 0, whose only square root is itself. Euler's criterion, $a^{(p-1)/2} = 1$, decides whether a given a is a square, and when it is, the equation $y^2 = a$ has exactly the two solutions $\pm y$, the two points

[40] A Koblitz curve over $\mathbb{F}_{2^{233}}$ with $h = 4$, used by Alpen Labs' Glock construction as the SNARK curve: binary-field multiplication garbles at a few AND gates per bit and the Frobenius endomorphism accelerates scalar multiplication, at the price of the subgroup check discussed here.

$(x, \pm y)$. On the curve side, $E(\mathbb{F}_p)$ has n elements, one of which, the point at infinity, is the identity and has no coordinates; the remaining $n-1$ affine points all have $y \neq 0$, since a point with $y = 0$ would have order 2 and n is odd, so they pair up as $(x, \pm y)$ and the number of x with a lift is $(n-1)/2$.

Example 11.4 (Every leaf of an unbalanced taptree reaches the same root)**.** Let a taptree commit three scripts A, B, C with A at depth 1 and B, C at depth 2:



Write $\ell_A = H_{\texttt{TapLeaf}}(A)$ and likewise ℓ_B, ℓ_C; the internal node and root are

$$N = H_{\texttt{TapBranch}}\big(\mathrm{sort}(\ell_B, \ell_C)\big), \qquad \mathrm{root} = H_{\texttt{TapBranch}}\big(\mathrm{sort}(\ell_A, N)\big),$$

nested hashing: each leaf hash digests a script, each branch hash digests the sorted pair of child hashes below it. Here sort compares the two children byte by byte from the first byte at which they differ (equivalently, as 256-bit integers) and concatenates the smaller first, making the branch hash independent of child order; the control block therefore carries bare sibling hashes with no per-level direction flag, since which side a sibling enters on is decided by its bytes. A spend of A reveals the script A and a control block carrying the single sibling N; the verifier recomputes ℓ_A and folds once. A spend of B reveals B with siblings (ℓ_C, ℓ_A), one per level: the verifier recomputes ℓ_B, folds to N, folds again to the root. A spend of C carries (ℓ_B, ℓ_A) and, since children are sorted before each `TapBranch` hash, reaches the identical N and hence the identical root. All three paths terminate in the same 32 bytes, so the single tweak equation $Q = P + \mathrm{hash}(P \,\|\, \mathrm{root})\, G$ covers all three spends; were any path to yield a different root, its leaf would fail the equation and be unspendable, so agreement of every path on one root is what makes a multi-leaf taptree spendable at all. Depth varies only the proof length ($33 + 32m$ bytes, §11.2): the spender of A never learns B or C, their entire subtree standing in as the one hash N.

11.3 Sighash: What a Signature Commits To

A Bitcoin spend has three layers. The *transaction data* is the set of input UTXOs to be destroyed and output UTXOs to be created; it contains no authorization of its own. The *sighash* is a commitment to that data: a tagged SHA-256 digest (tag `TapSighash`, BIP341) over the fields the one-byte sighash flag selects. The *signature* is the authorization: it signs the sighash, and it verifies only under the public key that the UTXO being spent named in its locking script, so validity of the signature under the demanded key is what authorizes exactly that destruction and creation. The flag is the policy, the sighash is the bytes actually signed. The default, `SIGHASH_ALL`, covers every input and output, so the signature authorises exactly one transaction paying exactly those outputs; change any output and the sighash changes, invalidating the signature.

For a Taproot key-path spend the digest commits, in order, to: the sighash flag; the transaction version and locktime; the inputs being spent, i.e. their outpoints (which coins), amounts, scripts, and sequences; and the outputs (who is paid). Other flags relax this: `SIGHASH_ANYONECANPAY` drops all inputs but the one being signed, `SIGHASH_NONE` drops the outputs, and `SIGHASH_SINGLE` commits only to the output at the signed input's index. BitVM3 designs generally sign with

SIGHASH_ALL, which is what "binds the signature to all inputs and outputs" means in Remark 11.6.[41]

Remark 11.5 (Sighash flags as a fee lever). Bitcoin circuit-style protocols exploit these flags deliberately: the signed *outputs* are kept a fixed set (sometimes plus one output returning fees) so the covenant emulation holds, while the *input* side may be left open, e.g. via ANYONECANPAY, so that a fee-paying input can be attached at broadcast time. Pre-signed graphs thereby gain a dynamic fee mechanism without precommitting fees at setup for every state transition.

Two consequences of *what* the digest covers matter for the graph. The digest commits to each input's nSequence, so a pre-signed timeout transaction has its relative timelock frozen in: lowering it afterwards would change the sighash and invalidate the committee's signatures. Witness data, by contrast, is *not* covered by any sighash: this is why the disprover can attach the preimage L^* to an already-pre-signed Disprove transaction (Remark 11.6) at dispute time without invalidating anything.

Remark 11.6 (Covenant emulation by pre-signing). A *covenant* would let an output constrain *where* it may be spent, restricting the spending transaction to a specific output or locking script. Bitcoin Script has no such opcode, so a script cannot bind the coins it locks to a designated next output. BitVM3 emulates one by pre-signing: at setup the N-of-N committee uses Partially Signed Bitcoin Transactions (PSBTs) to collaboratively sign every transaction of the dispute graph (Assert, Disprove, timeout, payout), each under SIGHASH_ALL, committing the graph's possible end states and binding each signature to that transaction's exact inputs and outputs. Two facts make the graph constructible in advance. Signing requires the funding outpoint fixed first, since each signature commits to its input outpoints; and because a txid covers only the non-witness serialization (§11.1), every txid in the graph is computable at setup, each child referencing its parent's txid before anything is broadcast, while witnesses attached at claim time (adaptor completions, revealed labels) alter no txid. Pre-SegWit malleability would have broken this chain. Once at least one honest signer deletes its key share, no alternative transaction can ever be signed, so the pre-signed graph is the only way the bonded output can move. This is what pins the off-chain verdict to a fixed on-chain outcome.

11.4 Spending Conditions

Bitcoin Script defines around a hundred opcodes (many disabled or reserved), interpreted in interpreter.cpp; the tapleaf scripts in BitVM3 use only the handful below:

- **Hashlock** (OP_SHA256 $\langle h \rangle$ OP_EQUALVERIFY): spendable only by revealing a preimage hashing to h.[42] This gates Disprove on the false-output label: $h = H(L^*)$, and the witness must supply L^*.
- **Relative timelock** ($\langle \Delta \rangle$ OP_CHECKSEQUENCEVERIFY): the output cannot be spent until Δ blocks after its confirmation, giving challengers a window before the operator's timeout path opens.
- **Signature check** ($\langle pk \rangle$ OP_CHECKSIGVERIFY; as a leaf's *final* opcode, OP_CHECKSIG, which leaves true on the stack, cf. the hashlock footnote): a BIP340 Schnorr signature under pk; e.g. the Assert timeout leaf pairs this with the relative timelock above, so only the operator holding pk_P can reclaim the bond, and only once the challenge window has elapsed.

[41] Field ordering follows BIP341 (§11.2). In Bitcoin Core it is SignatureHashSchnorr in src/script/interpreter.cpp: the function opens at line 1493, and lines 1521–1552 serialise the message in this order: epoch byte, hash_type, nVersion/nLockTime, the four input hashes (skipped under ANYONECANPAY), sha_outputs (under SIGHASH_ALL), spend_type, and the input index. Accessible walkthrough of flag versus signed digest: learn me a bitcoin, https://learnmeabitcoin.com/technical/keys/signature/#sighash.

[42] As a *standalone* tapleaf this must end in OP_EQUAL (which leaves true on the stack), not bare OP_EQUALVERIFY (which consumes both operands and leaves the stack empty, failing the spend); OP_EQUALVERIFY is correct only when followed by further script that leaves a truthy result.

- **Code separator** (`OP_CODESEPARATOR`): records its position in the script, and that position enters the next signature's sighash. BitVM3 uses it to encode the SNARK proof, split into 128 digits of 8 bits, one 65-byte Schnorr adaptor signature per digit, with `OP_CODESEPARATOR` between successive `OP_CHECKSIGVERIFY` operations so a single public key can authenticate every digit.

Script execution is a stack machine. The interpreter holds five pieces of state between opcodes: a main stack, an alternate stack, the program counter, the conditional-execution flags of `OP_IF`, and the variable `m_codeseparator_pos`. All five are declared or initialized at the head of `EvalScript`, the interpreter's entry point (Bitcoin Core v28.0, `src/script/interpreter.cpp`, L406–L433):

```
bool EvalScript(
    std::vector<std::vector<unsigned char>>& stack, // (1) main stack
    const CScript& script,   // the opcodes being executed
    unsigned int flags, const BaseSignatureChecker& checker,
    SigVersion sigversion,
    ScriptExecutionData& execdata, // holds (5) m_codeseparator_pos
    ScriptError* serror)
{
    CScript::const_iterator pc = script.begin(); // (2) program counter
    ConditionStack vfExec;                       // (3) OP_IF flags
    std::vector<valtype> altstack;               // (4) alternate stack
    execdata.m_codeseparator_pos = 0xFFFFFFFFUL; // (5) init: none
       executed
    ...
    case OP_CODESEPARATOR:
        execdata.m_codeseparator_pos = opcode_pos; // the opcode's
           entire effect (L1054)
    ...
}

// In a separate function, the sighash serializer SignatureHashSchnorr
// (L1555-1560), for tapscript spends:
ss << execdata.m_tapleaf_hash;        // which leaf is executing
ss << key_version;
ss << execdata.m_codeseparator_pos;   // where within it: committed
   into the digest
```

Witness items are pushed onto the stack first; opcodes then execute strictly left to right. The sentinel `0xFFFFFFFF` (the suffix `UL` in the listing is C++ compiler syntax, unsigned long, not part of the value) is set *once*, at script start; only the first signature check ever reads it, and each executed separator overwrites the single variable with its own position. Exactly one opcode writes `m_codeseparator_pos` and only signature checks read it, serializing its current value into the sighash preimage. Three slots of the CheckGS leaf:

idx	opcode	`m_codeseparator_pos`	effect
0	$\langle pk_{slot} \rangle$	`0xFFFFFFFF`	push key
1	`CHECKSIGVERIFY`	`0xFFFFFFFF`	verify σ_1 against m_1
2	`OP_CODESEPARATOR`	$\to 2$	update the variable
3	$\langle pk_{slot} \rangle$	2	push key
4	`CHECKSIGVERIFY`	2	verify σ_2 against m_2
5	`OP_CODESEPARATOR`	$\to 5$	update

Each signature committed to a position at signing time, the position being a field of the message it signed; at verification each slot recomputes its message from the variable's current value and compares. Agreement per slot, checked locally and independently, is what makes only the correctly ordered witness pass, and the 128-signature requirement itself is enforced by the leaf

containing 128 `CHECKSIGVERIFY` opcodes, each aborting the script on failure. The position value is committed, never stored: it appears nowhere in the transaction bytes and exists only as a field of the hash input at verification time.

11.5 Signatures and Adaptors in the Witness

Two kinds of signature populate the witnesses. Plain BIP340 Schnorr signatures authorize the cooperative key-path settle and the timeout-leaf spend (though some BitVM variants disable the cooperative key path entirely, forcing every spend through a committed script leaf; cf. the NUMS key of Remark 11.3). Schnorr *adaptor* signatures (§9) carry the proof encoding: the operator posts a pre-signature bound to the proof digits, and completing it on-chain simultaneously authorizes the Assert transaction and reveals the encoded labels to challengers, with no separate label-delivery step (worked one wire at a time in §13.2).

11.6 The Assert, Disprove, and Timeout Templates

The dispute reduces to three transaction templates (BitVM3 paper Tables 3–4).

- **Assert** spends the bonded output. Its input is gated by the on-chain check $\texttt{CheckGS}_{pk}(\sigma)$, so posting it publishes the encoded proof. Its output is a taptree with two leaves: a challenge leaf $\texttt{Hashlock}(H(L^*))$ and a timeout leaf $\texttt{CheckSig}_{pk_P} \wedge \texttt{RelTimelock}(\Delta)$.
- **Disprove** spends the Assert output through the challenge leaf. Its witness is the recovered false-output label L^*; the script checks $H(L^*)$ against the committed hash and, on success, burns the operator's bond (the slashing; the paper's Disprove template carries a single `OP_RETURN` output).
- **Timeout** spends the Assert output through the timeout leaf once Δ blocks have elapsed with no valid Disprove, letting the operator reclaim the bond. This is the honest-operator path.

Remark 11.7 (`CheckGS` is a script gadget, not an opcode). The two checks in the Assert template are gated differently. $\texttt{CheckSig}_{pk_P}$ in the timeout leaf is a single real opcode (`OP_CHECKSIGVERIFY`, §11.4) verifying one BIP340 signature under the operator's key pk_P. $\texttt{CheckGS}_{pk}(\sigma)$ has no opcode of its own: it is shorthand for the per-digit gadget of §13.2, a chain of $\langle pk_{slot} \rangle$ `OP_CHECKSIGVERIFY` separated by `OP_CODESEPARATOR`, one slot per proof digit (pk_{slot}: a joint key no party can sign under after setup, not the operator's own; §13.2). Consensus checks only those ordinary signatures under pk_{slot}; the "GS" semantics, that passing releases $\texttt{En}(e, \pi)$, come from the adaptor pre-signatures each bound to a label statement $T_i^b = [L_i^b]G$ (Remark 13.6), not from any special validation rule. So both checks reduce to `OP_CHECKSIGVERIFY` under a fixed key; `CheckGS` differs only in that its signatures are adaptor completions that leak the labels. Completing them requires the wire labels L_i^b themselves, the garbling secret $e = sk_{GS}$ that only the operator (the garbler) holds, so only the operator can produce a valid Assert witness, and posting it necessarily releases the active labels $\texttt{En}(e, \pi)$. Because pk_{slot} is fixed at setup, the whole Assert is bound to one operator's instance, while the audited verifier circuit is reused across operators (Remark 12.3).

Remark 11.8 (Multiple operators under one deposit). Each eligible operator carries its own setup artefacts: slot key, pre-signature set, and CheckGS leaf. A single deposit output hosts them as sibling leaves of one taptree, one claim path per operator. Every party can construct this tree because its leaves contain only public data, public keys and hash commitments, so all participants derive the identical Merkle root and address; no secret enters the tree image. Which operator can claim through a given leaf is decided not by script but by possession: that leaf's pre-signatures complete only with the key scalars W held by its operator (cf. Remark 12.3).

Remark 11.9 (Committee compromise does not reach the labels). The signer committee contributes only slot-key signing shares; `PreSign` consumes the public statements $T_{d,v}$ and never the scalars $W_{d,v}$, which are sampled and held solely by the operator, as are the plaintext labels. A colluding committee therefore cannot extract a key, decrypt a lookup row, or derive the Disprove preimage to slash an honest operator. What committee compromise does break is the covenant layer: with the slot key reconstructed before deletion, fresh signatures and a re-signed graph become possible, which is why the protocol assumes at least one honest signer deletes its share (§13.1).

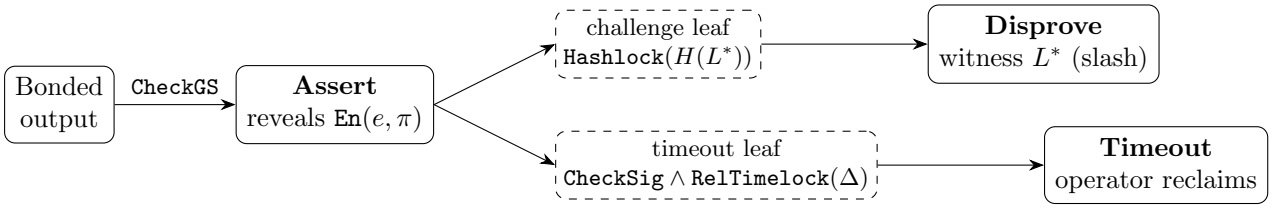


Figure 1: The Assert output carries two tapleaves; the verdict decides which is spendable. A false verdict lets a challenger open the hashlock (Disprove, slash); otherwise the timelock lets the operator reclaim the bond (Timeout).

12 The SNARK Being Garbled

The circuit the operator garbles is the *verifier* of the application proof, $\texttt{SNARK.Vrfy}(vk, \varphi, \pi)$, instantiated here with Groth16. This section fixes what that verifier consumes and, crucially, how its *public inputs* bind a proof to one specific claim. It is distinct from the *setup* SNARK of §10.1: there the SNARK proves the garbling honest; here the SNARK is the thing *inside* the garbled circuit.

12.1 Components

A Groth16 instance is fixed by a *common reference string* $\text{CRS} = (pk, vk)$ from a trusted setup:

- the *proving key* pk lets the operator turn a witness w into a proof $\pi = (A, B, C) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$ (the proof's size is independent of the statement being proved: always exactly three group elements, regardless of circuit size).[43] pk is consumed entirely at proving time and never appears in verification.
- the *verifying key* vk is the circuit-specific subset of the CRS that, with π and the public inputs, drives the bilinear pairing check of elliptic curve group (ECG) elements. It is vk that is compiled into the garbled verifier: verification uses only vk, never pk, which is what lets one audited vk (Remark 12.2) check proofs from any operator holding the corresponding pk.

The CRS entries are group elements, commitments to the setup's secret scalars (the "toxic waste"): usable for verification, but revealing nothing that would let a party forge a proof.

Remark 12.1 (Why the CRS entries are group elements). Encoding the setup scalars as points $[\tau]G, [\tau^2]G, [\tau^3]G, \ldots$ (the powers of τ of §12.4)[44] rather than as the scalars themselves does

[43] Here w is the *SNARK witness*: the prover's private input (the destination-chain burn transaction, block header, and Merkle proofs). It is unrelated to the *Bitcoin transaction witness*, the on-chain unlocking field (§11.5) that carries signatures and released labels.

[44] $[k]G$ is scalar multiplication: the point G added to itself k times under the elliptic-curve group law (§9.2). See `https://en.wikipedia.org/wiki/Elliptic_curve_point_multiplication`.

two jobs at once. First, *hiding*: the scalars are the toxic waste, and publishing any of them would let a party forge proofs, so they are locked behind the discrete-log problem, one can compute with the point but not recover the scalar, which is what makes the CRS safe to publish. Second, *computability*: proving and verifying are linear combinations of CRS points followed by a bilinear pairing over them (§12.6), and the scalar-to-point homomorphism (Remark 9.1) lets the prover form $\pi = \sum_i [w_i]$ (CRS point) without knowing the underlying scalars. A plain hash commitment could hide the scalars but not support this arithmetic; a group point is one-way *and* algebraically usable (the same distinction as Remark 9.5). The proof π is itself group elements built from the CRS, but that is a consequence of the encoding, not its reason.

Remark 12.2 (Why the vk is fixed at setup and audited once). The verifying key vk is *compact*: a handful of group elements (the images of $\alpha, \beta, \gamma, \delta$ and one public-input base ψ_i per public input). The bulk of the ceremony output, the powers of τ whose length scales with the circuit's constraint count, sits in the *proving key*, not vk. Because the circuit `SNARK.Vrfy` is fixed, its vk is fixed too, so it can be audited once and reused across every instance: this is the "one audited verifier" of §12.3, with only the per-UTXO public inputs distinguishing deposits (Remark 12.3).

12.2 Garbled Verifier Overview

Four facts fix the shape of everything in §13; they are collected here once, before the protocol uses them.

1. Input side. The GC is itself the SNARK verifier. It takes a proof $\pi = (A, B, C)$, with A, C of size 32 bytes and B of size 64 bytes, so π totals 128 bytes = 1024 bits, meaning the GC has 1024 input wires, each with two labels: 2048 labels total, 2^{1024} possible input combinations.[45] The verifier's public inputs are hard-coded into the circuit at setup (§12.3), which is why the proof bits are the only input wires.

2. Output side. The GC has one output wire with two labels, $L_{\text{out}}^{\text{True}}$ and $L_{\text{out}}^{\text{False}}$: either the evaluation result is the preimage of the on-chain hash $h = H(L_{\text{out}}^{\text{False}})$, or it is not; looking it up in the decoding info d also reveals the plaintext verdict. Only the False label is ever hash-committed on-chain (developed in Remark 13.3).

3. Commitment power, not forging power. Holding the input labels gives the operator the ability to *commit* any proof π to the garbled verifier, not the ability to *forge* one: an accepting π without a valid witness happens with negligible probability, as for any other verifier. The map input $\to$ output is the deterministic public `SNARK.Vrfy`, so freedom over inputs is worthless when no accepting input exists.

4. Why e stays secret. The encoding info e (the input label pairs) must nonetheless be kept secret from challengers: anyone holding e could encode a fake proof, run it through the GC, and evaluate their way to the preimage of the hash, stealing the bond from an honest operator's Assert. Authenticity (Definition 2.3) protects the honest operator only while challengers hold nothing beyond the one released encoding $\texttt{En}(e, \pi)$.

12.3 Public Inputs and Binding

The statement φ is a vector of *public input values*, supplied per proof, that bind π to a specific assertion; the verifier must hold them, or a proof would verify without attesting to *anything*. In a bridge these encode circuit-format constants plus a commitment to the destination-chain event (peg-out txid, a light-client/SPV inclusion proof, accumulated chain work).

The binding is mechanical, i.e. it is established automatically as a consequence of the verification arithmetic, not by any separate check or assumption. The circuit-specific phase of setup

[45] B lives in $\mathbb{G}_2$, whose points have coordinates in the extension field $\mathbb{F}_{p^2}$ (the sextic twist allows $\mathbb{G}_2$, natively in $\mathbb{F}_{p^{12}}$, to be represented in $\mathbb{F}_{p^2}$: an efficiency win), hence twice the size of a $\mathbb{G}_1$ point. A and B must sit in different groups for the pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$; symmetric variants exist but are insecure or impractical at useful parameter sizes.

fixes, for each public-input wire i, a group element $\psi_i \in vk$. At verification the public-input scalar x_i multiplies its ψ_i, and the term $\sum_i [x_i]\,\psi_i$ enters the pairing equation, so a proof verifies only for the exact x_i it was made against. In BitVM3 the ψ_i are identical across all UTXOs (one audited verifier), while the x_i are *per-UTXO scalars hard-coded at garble time*, which is what pins a garbled instance to one deposit.

Remark 12.3 (Pooled-UTXO hazard). Given the verifier circuit must be identical across UTXOs to be audited once, the *public inputs* are the only thing distinguishing instances. If they do not bind per-UTXO, an operator holding a valid proof for one deposit could replay it against another, draining a co-depositor. The per-UTXO scalar x_i, bound through ψ_i, is what prevents this cross-claim.

12.4 Trusted Setup and Toxic Waste

Groth16's setup runs in two phases. *Powers of Tau* encodes powers of a secret τ as group elements, producing a *structured reference string* (SRS) that is *universal*, reusable across circuits. The *circuit-specific* phase takes the SRS and the circuit and produces the CRS (pk, vk), reusable across all proofs for that circuit. Both phases generate secret scalars (the toxic waste) that must be destroyed: whoever retains them can forge proofs. In practice the universal phase is a public ceremony reused by many protocols (e.g. the one SP1 relies on), so only the circuit-specific phase is protocol-specific (a fuller treatment of Groth16 is in the companion notes, `https://github.com/CLGAP/Crypto/blob/main/docs/theory.pdf`).

12.5 How the Proof Reaches the Chain

Bitcoin never runs `SNARK.Vrfy`. The operator garbles it, and the proof π enters as the garbled input: its encoding $\mathtt{En}(e, \pi)$ is committed at garble time and released on-chain only by signing π through the garbled-encoding extractable / adaptor signatures of §9.6. So the on-chain object committing the proof is the adaptor signature, and the public-input binding above is what ties that commitment to a specific claim.

Remark 12.4 (What a "trace" is, and why BitVM2 commits it). An *execution trace* of `SNARK.Vrfy` is the full step-by-step record of running it: every intermediate value the computation produces, gate by gate (or instruction by instruction), not just the final accept/reject bit. It is the same object as the complete wire assignment $\{\beta_v\}$ of Definition 1.2, taken over *every* vertex rather than the output wires alone. BitVM2 settles disputes over this trace directly: the verifier's computation is compiled to a circuit, the whole trace is bit-committed on-chain, and a challenger who disagrees points at a single faulty transition and has Bitcoin Script re-execute just that one low-level operation, cheap enough to fit the script budget; the $O(\log C)$-round interactive bisection this replaces is the original BitVM's (the same single-instruction idea underlies BitVMX, Remark 6.1). The cost is that committing and disputing the trace is on-chain-heavy. BitVM3 removes the trace from the chain entirely: the challenger evaluates the garbled verifier off-chain (§13) and the only on-chain operation is a single hash equality, which is where the $\sim 1000\times$ reduction over BitVM2 comes from.

12.6 Structuring the Verifier Circuit

As it is the *verifier* that is garbled, the sole design pressure is the verifier's gate count: every AND gate becomes a table the garbler commits and the challenger evaluates, so a smaller verifier means a smaller garbled circuit, a smaller setup, and a cheaper reveal. This is why the garbled verifier is Groth16's; the choice is comparative, not a claim that the verifier is cheap.

Groth16's verifier is a fixed set of elliptic-curve operations: the pairing check $e(A, B) = e(\alpha, \beta) \cdot e(vk_\varphi, \gamma) \cdot e(C, \delta)$ with $vk_\varphi = \sum_j [x_j]\,\psi_j$: the verifying key specialized to the statement φ, i.e. the fixed bases $\psi_j \in vk$ weighted by the claim's public-input scalars x_j, and the only

statement-dependent term in the check. In total, four pairings and one multi-scalar multiplication over BN254. As Boolean gates this is still *billions* of gates (2.7×10^9 non-free gates, Remark 10.1): a single BN254 pairing is a Miller loop and final exponentiation over $\mathbb{F}_{p^{12}}$, thousands of ∼254-bit field multiplications deep.[46] Fewer than four are actually garbled: $e(\alpha, \beta)$ is a setup constant, and for the hard-coded public inputs of §12.3 so is $e(vk_\varphi, \gamma)$, leaving exactly the two proof-dependent pairings, $e(A, B)$ and $e(C, \delta)$.

Two properties make Groth16 the least-bad target rather than a small one: the gate count is *constant in the statement* being proved, and it is far smaller than any competing verifier.[47] A STARK or PLONK verifier runs tens to hundreds of thousands of hash and field operations, larger still as a Boolean circuit. This comparison is specific to garbling: when the verifier is instead *executed* in script (the BitVM2 route of Remark 12.4), Bitcoin's native hash opcodes may favour hash-heavy verifiers if the verifier's hash function is among those Bitcoin Script supports natively; a garbled verifier is never executed on-chain, so only Boolean gate count matters, and a single hash invocation there is tens of thousands of gates.

The standard mitigation is *recursion*. A fast, transparent inner system (a STARK from Risc0 or SP1) proves the application statement; a single outer Groth16 proof then attests "I checked that inner proof", and it is the outer Groth16 verifier, constant-size, that is compiled into the garbled circuit.[48] The garbled object then never grows with the application, only with the outer verifier. The division of labour: the STARK carries the application claim, the Groth16 wrap attests that the STARK verifies, and the garbled circuit is the Groth16 verifier, whose input is that wrap proof $\pi = (A, B, C)$, never the STARK itself.

Curve choice is the remaining lever. BN254 is the de facto target because Ethereum's precompiles fix it and because its ∼ 254-bit field keeps the pairing circuit smaller than BLS12-381's ∼ 381-bit field; the trade is BN254's thinner security margin (∼ 100 bits under recent tower-field-NFS estimates) against BLS12-381's ∼ 120.[49] Either way the garbled Groth16 verifier remains tens of GB even after the optimizations of §4 (the ≈ 40 GB of Remark 4.12 already assumes half-gates), which is what motivates the designs of §14: Glock and BABE keep the optimized Boolean garbler and restructure *what* is garbled, while Argo (§**??**) replaces the Boolean primitive itself with arithmetic-circuit garbling.

12.7 Hazards and Constraints

Garbling a verifier, rather than running one, brings its own hazards and constraints:

- **Public-input count, order, and encoding.** The term $\sum_j [x_j]\, \psi_j$ fixes one $\psi_j \in vk$ per public input, and the garbled circuit bakes in that count and order. A statement re-parameterized after garble time (an extra input, a reordering, a different field encoding of x_j) no longer matches the committed ψ_j and simply fails. For example, a 256-bit event digest does not fit in one ∼ 254-bit BN254 field element, so it enters as two limbs (two x_j); switching to a truncated single-element encoding, or batching several withdrawals under one Merkle root where a single digest stood before, changes the constraint system interpreting the inputs even when the input count is unchanged. Either change means a new circuit-specific (pk, vk): a fresh ceremony, a fresh audit, a re-garbling, and a re-signed graph, not a configuration change.

[46] The deployed BitVM line lowers this cost by *verifying* the pairing rather than computing it: the final exponentiation is replaced by a residue witness check and the Miller loop shrinks under precomputed line evaluations (Novakovic and Eagen, *On Proving Pairings*, ePrint 2024/640, `https://eprint.iacr.org/2024/640`).

[47] J. Groth, *On the Size of Pairing-based Non-interactive Arguments*, EUROCRYPT 2016. ePrint 2016/260, `https://eprint.iacr.org/2016/260`.

[48] Risc0, `https://dev.risczero.com`; SP1 (Succinct), `https://docs.succinct.xyz`. Both expose a Groth16 "wrap" of an inner STARK for cheap on-chain verification.

[49] Curve comparison and the tower-NFS re-estimate: `https://hackmd.io/@benjaminion/bls12-381`.

- **Static vs. dynamic public inputs.** Garbling freezes the statement at setup: the efficient construction folds vk_φ into the circuit as a constant (the folding that lets §12.6 drop $e(vk_\varphi, \gamma)$ from the garbled pairings), so every x_j must be known at garble time. A bridge violates this: the peg-out output digest exists only once a user withdraws, long after peg-in. The BitVM3 paper meets this with enumeration and commitment: a garbled instance is prepared per anticipated claim slot (per checkpoint and peg-out instance, its statement folded in), and genuinely runtime values, such as the light client's checkpoint block hash and timestamp, are bound by on-chain Winternitz commitments the relation is written against (§5). The witness-encryption variant instead rebinds at runtime: its ciphertext bakes the statement term in at setup, and the *deferred binding* split of §**??** binds the dynamic scalar at evaluation.
- **Proof non-malleability.** A Groth16 proof is not unique: negating both A and B satisfies the verification equation unchanged, since $e(-A, -B) = e(A, B)$ by bilinearity, so $(-A, -B, C)$ is a second valid proof of the same statement (and full re-randomization with fresh scalars yields infinitely many more, so no canonical-form check closes the gap[50]). The hazard is not soundness (a mauled proof still proves the same statement) but any surrounding logic that uses proof bytes as an identifier: replay protection keyed on a hash of the proof, for instance, reads a mauled proof as a fresh claim and processes the same event twice. The graph must therefore bind the *statement* (via the public inputs and the covenant), never the proof bytes.
- **Verifying-key binding.** The audited object is one vk; a garbled instance is only as sound as the guarantee that it garbles *that* vk, which is what setup verification certifies (§10). The ceremony behind (pk, vk) remains a trust root with toxic waste to destroy (§12.4), but that hazard attaches to any Groth16 deployment, not to garbling specifically.
- **Cross-instance replay.** The public inputs are the only per-instance binding across UTXOs; without a per-UTXO scalar, a proof for one deposit replays against another (Remark 12.3).

13 BitVM3-core Protocol

The garbling primitive of §3 ($\mathtt{Gb}, \mathtt{En}, \mathtt{Ev}, \mathtt{De}$) and the extractable signature scheme of §8 form elements which can be composed into a Bitcoin bridge. The operator garbles the SNARK verifier circuit, proves the garbling well-formed (§10), and binds a signature scheme to its input encoding; posting a signature on a SNARK proof on-chain then releases input labels which can be evaluated on that garbled circuit. Any challenger can then run the garbled verifier off-chain, and if valid, the proof will produce an output wire which corresponds to a label which which will decode to true, and if invalid the proof the output label will return a label which the decoder will identify as false, and that label can then be posted in a fraud-proof to slash the operator's bond. Bitcoin script itself never executes the SNARK verifier: it only checks a hash equality, which is what keeps the construction within Bitcoin's size (scripting) constraints..

Throughout this section the label reveal is the Schnorr adaptor realization of §9: labels committed as curve points $T = [\ell]G$, released by completing pre-signatures (one 65-byte adaptor signature per proof digit), chosen for on-chain cost: about 8 witness bytes per proof bit against roughly 25 for Winternitz and 94 for Lamport (BitVM3 §8.1). The notation `GS.Sign`/`GS.Extract`

[50]Concretely: for a valid (A, B, C) and fresh $r, s \xleftarrow{\$} \mathbb{Z}_q^*$, the tuple $([r]A,\ [1/r]B + [s]\delta,\ C + [rs]A)$ verifies for the same statement: the left side becomes $e(A, B) \cdot e(A, \delta)^{rs}$ and the shifted C contributes the matching factor $e(A, \delta)^{rs}$ on the right. Only the public proof and $\delta \in vk$ are needed, so anyone can maul; negation is the case $r = -1$, $s = 0$. The output is distributed exactly as a fresh honest proof, since the prover's own internal randomness is what is being re-sampled, which is why mauled proofs cannot be filtered.

below is shorthand for the GES interface of §8, whose security properties (extractability, unforgeability) the realization must meet; it does not imply the hash-based instantiation.

Six artefacts are produced once at setup, per eligible operator (only the audited verifier circuit is shared across operators; Remark 11.7), and referenced throughout:

1. Garbled circuit GC of $\texttt{SNARK.Vrfy}$, a public list of encrypted truth tables (§3.2).
2. Consistency proof that GC and the committed labels are a faithful garbling of the public circuit, and that pk_{GS} commits the same encoding e (§10), so the self-garbling operator cannot encode a wrong verdict.
3. Decoding info d, mapping output-wire labels back to True/False (§2).
4. GES keypair (pk_{GS}, sk_{GS}), extractable signatures keyed to the garbled circuit's input encoding e (§8). (pk_{GS} is the public table of statements $T_{d,v} = [W_{d,v}]G$ and rows $\text{row}_{d,v}$ against which challengers decode released scalars back into labels and digit values; $sk_{GS} = e$ itself, the encoding info returned by the same $\texttt{Gb}$ call that produced GC and d above.)
5. Transaction graph $\{\text{Assert}, \text{Disprove}, \ldots\}$, pre-signed at setup (§11.6).
6. Signer committee $S_1, \ldots, S_N$, which presigns the transaction graph once, then deletes setup-time secrets (§13.1). (After deletion no one, the operator included, can sign fresh under the slot key: a valid Assert signature can only arise by scalar-adding a label onto a setup pre-signature; Remark 9.8.)

The protocol then runs in three phases, each expanded in the subsections that follow:

1. **Setup** (off-chain, once): operator runs $\texttt{Gb}$ and $\texttt{GS.KeyGen}$, and proves GC well-formed against the agreed verifier circuit, i.e. the public vk and hard-coded public inputs (§12), publishing the certification in whichever form the chosen variant of §10 dictates (instance commitments and opened seeds, a setup proof, or an attestation); committee N-of-N presigns the transaction graph over a funding outpoint fixed in advance (Remark 11.6); $(GC, d, pk_{GS}, \text{Assert})$ and all pre-signatures are published: everyone sees them, but only the label-holder can complete one.
2. **Proving** (on-chain, per claim): operator builds the SNARK proof π and signs it into the label-revealing signature $\sigma_\pi \leftarrow \texttt{GS.Sign}(sk_{GS}, \pi)$: for each bit π_i, select that wire's matching label, so $\sigma_\pi = (L_{1,\pi_1}, L_{2,\pi_2}, \ldots, L_{\ell,\pi_\ell}) = \texttt{En}(e, \pi)$; the signature *is* the garbled input. The operator posts the Assert transaction with σ_π as its witness.
3. **Disprove** (on-chain, conditional): a challenger extracts the proof's input labels $L_\pi \leftarrow \texttt{GS.Extract}(pk_{GS}, \sigma_\pi)$ from the posted signature, evaluates them through the garbled circuit to the output label $L_y \leftarrow \texttt{Ev}(GC, L_\pi)$, and decodes that label to the plaintext verdict $y \leftarrow \texttt{De}(d, L_y)$; if $y =$ False they post Disprove with the output label L_y as the fraud-proof witness: this $L_y = L^*$ *is* the preimage of the hash $H(L^*)$ committed in the challenge leaf, so the script check is a single hash equality.

13.1 Role Assignments

The protocol partitions roles cleanly across three party classes (per §2.2 and Fig. 6 of the BitVM3 paper). The *operator* (the prover P) plays the garbler: they run $\texttt{Gb}(1^\kappa, \texttt{Circuit}(\texttt{SNARK.Vrfy}))$ to obtain (GC, e, d), run $\texttt{GS.KeyGen}(1^\kappa, e)$ to obtain (pk_{GS}, sk_{GS}), and send $(GC, d, pk_{GS}, \text{Assert})$ to the verifiers. When claiming, the operator signs the SNARK proof $\sigma_\pi \leftarrow \texttt{GS.Sign}(sk_{GS}, \pi)$ and posts Assert on-chain with σ_π as witness.

The *challenger* (any verifier $V \in \mathcal{V}$) plays the evaluator: extracts $L_\pi \leftarrow \texttt{GS.Extract}(pk_{GS}, \sigma_\pi)$ from the on-chain signature, computes $L_y \leftarrow \texttt{Ev}(GC, L_\pi)$ and $y \leftarrow \texttt{De}(d, L_y)$ off-chain, and posts

Disprove with L_y as fraud-proof witness if $y = \text{False}$. The role is permissionless: any signer, operator, or user may act as a challenger, and the protocol assumes at least one honest challenger is online at all times.

Remark 13.1 (No challenge bond). A spurious Disprove is not economically deterred but cryptographically impossible: its witness L^* is obtainable only when the proof actually evaluates False (Definition 2.3), so challenging carries no bond and costs only the transaction fee. Challenge bonds as anti-spam belong to interactive dispute designs (BitVM2), which the one-shot Disprove eliminates.

The *signer committee* $(S_1, \ldots, S_N)$ does *not* garble; it N-of-N presigns the transaction graph at setup using SIGHASH_ALL, providing covenant emulation that prevents the operator from substituting different transactions. Setup integrity holds as long as one honest signer deletes their setup-time secret afterwards.

Finally, *Bitcoin* plays neither garbler nor evaluator: it only adjudicates the on-chain hash check $H(L_y) = H(L^*)$, never executing `Ev` or `De` itself.

13.2 Setup, Proving, Disprove

This subsection follows the data through the three phases of Fig. 6; the enumerate above is the summary, here we name the artefact each step produces or consumes.

13.2.1 Setup (off-chain, once)

The operator fixes the circuit $\texttt{Circuit}(\texttt{SNARK.Vrfy})$ for the statement to be proved and runs $(GC, e, d) \leftarrow \texttt{Gb}(1^\kappa, \cdot)$, then $(pk_{GS}, sk_{GS}) \leftarrow \texttt{GS.KeyGen}(1^\kappa, e)$, binding the signature scheme to the input encoding. The public bundle (GC, d, pk_{GS}) goes to the verifiers: pk_{GS} commits to every input label, in the deployed wide-label realization as the table of statements $T_{d,v} = [W_{d,v}]G$ and encrypted rows $\text{row}_{d,v}$, one per digit $d \in [128]$ and value $v \in \{0, \ldots, 255\}$ (Definition 9.3), and d commits to the output-wire labels, so the dispute can later reduce to a hash equality. Since the function $f = \texttt{SNARK.Vrfy}$ is public, the operator also posts a one-time consistency proof that GC and the committed labels are a faithful garbling of it; this is what stops a malicious operator from garbling a circuit that computes the wrong verdict, a guarantee the garbling *authenticity* property alone does not provide. In parallel the signer committee $S_1, \ldots, S_N$ presigns the transaction graph (Assert, Disprove, payout, and timeout transactions) N-of-N under SIGHASH_ALL, emulating a covenant that pins which transaction may spend which output; at least one honest signer then deletes their share, so the graph can never be re-signed. In the deployed adaptor realization, setup also *publishes* the per-digit pre-signatures, each bound to one candidate statement $T_{d,v}$ (Example 13.5). This is where spendability is fixed: the pre-signatures are public, but completing one requires its key $W_{d,v}$, and once the slot key's shares are deleted (Remark 9.8) completions are the *only* valid signatures that can ever exist. Holding the keys, and through them the labels e, is what it means to be able to spend the Assert.

Remark 13.2 (What pk_{GS} is for). Three consumers, none of them Bitcoin consensus (the script sees only pk_{slot}):

1. *Setup verification*: the consistency proof binds the table to the circuit, showing that the rows $\text{row}_{d,v}$ encrypt the encoding e that GC was garbled with, under the keys committed by the $T_{d,v}$ (§10).
2. *Extraction*: a challenger decodes a posted signature against the table: the recovered scalar $W = s - \widetilde{s}_{d,v}$ must satisfy $[W]G = T_{d,v}$, *which* of the slot's 256 statements matches determines the digit value v, and decrypting $\text{row}_{d,v}$ under W yields the digit's 8 labels. Without the table, released scalars are meaningless random numbers; with it they become "digit d, value v", a well-defined garbled input.

3. *Equivocation evidence*: all 256 statements of every slot are committed upfront, so a second opening on any slot is provable cheating, punished through evaluation (Remark 9.11).

Remark 13.3 (Recipe: constructing the Disprove gadget). The Disprove path (§11.6) is assembled from pieces defined separately; collected here as one construction, then read off as the three behaviours it guarantees.

1. *Garble the public verifier.* Take $f = \texttt{SNARK.Vrfy}$, a single-output circuit ($m = 1$; Definition 1.2), and run $(GC, e, d) \leftarrow \texttt{Gb}(1^\kappa, f)$. Its one output wire carries a label pair $(L_{\text{out}}^{\text{False}}, L_{\text{out}}^{\text{True}})$, two uniform κ-bit strings (Definition 3.1). Nothing about the gates is special: the "gadget" lives entirely in *which output label is committed where.*
2. *Commit the boundary asymmetrically.* pk_{GS} commits *both* labels of every *input* wire, so a signature on π releases exactly $\texttt{En}(e, \pi)$ (§8). At the *output*, publish only $h := H(L_{\text{out}}^{\text{False}})$ into the Disprove challenge leaf $\texttt{Hashlock}(h)$ (§11.6). The True label is never hash-committed on-chain: d decodes it off-chain so an honest challenger knows *not* to disprove, but no Bitcoin script ever needs its preimage. Only the reject outcome arms a spend.
3. *Certify well-formedness.* Prove GC, the committed labels, and d are a faithful garbling of the *agreed public* f (§10); otherwise a malicious garbler could commit $H(L_{\text{out}}^{\text{False}})$ of a circuit that outputs True on every input. Garbling authenticity alone does not cover a dishonest *garbler.*

The behaviour this produces, matching the three requirements:

- *The accepting outcome yields nothing spendable.* A π with verdict True evaluates to $L_{\text{out}}^{\text{True}}$, whose hash is not h; by authenticity (Definition 2.3) no evaluator can produce $L_{\text{out}}^{\text{False}}$ from it. No Disprove exists, and the operator times out and reclaims.
- *Every rejecting outcome hands over the false preimage.* Any π with verdict False evaluates *deterministically* to $L_{\text{out}}^{\text{False}} = L^*$, the exact preimage of h; the challenger drops it in the Disprove witness and the hashlock opens (slash).
- *Undesignated labels return nothing.* Feed any label not from a committed pair and, in the half-gate scheme of §4, which has no marker and no trial decryption, evaluation still produces a label, but one matching no hash in d, so `De` yields $\perp$ (§3.2, Definition 2.3). On the input side, a slot's `CHECKSIGVERIFY` completes *only* with its committed label (Remark 13.6), so the operator cannot Assert without releasing genuine $\texttt{En}(e, \pi)$.

A caveat on framing: `SNARK.Vrfy` is not "one input True, all others False" – it accepts *every* valid proof and rejects the rest. The gadget pins a single committed π per Assert, and the trap fires on *that* π's output-wire verdict, not on a unique accepting input.

13.2.2 Proving (on-chain, per claim)

To assert a claim φ (for instance a destination-chain burn that should release pegged funds), the operator builds a SNARK proof π with $\texttt{SNARK.Vrfy}(\pi) = \text{True}$, signs it as $\sigma_\pi \leftarrow \texttt{GS.Sign}(sk_{GS}, \pi)$, and posts the Assert transaction with σ_π in its witness. By extractability (§8) this single act publishes $L_\pi = \texttt{En}(e, \pi)$, exactly one label per input wire, the active half of e selected by π. In the deployed adaptor realization this "signing" involves no key at all: per digit, the operator performs one scalar addition on a published pre-signature, $s = \tilde{s} + L$. Completing with any scalar other than the committed label fails the Schnorr verification equation (Remark 13.6), so only genuine input labels of the garbled circuit produce a spendable Assert. The operator's bond sits in the output that Disprove or the timeout will spend. The bond is distinct from the deposit: the depositor's pegged funds sit in the bridge reserve and move only via the withdrawal

transaction, which requires the surviving Assert as a connector input, while the slashable output is the operator's own collateral, so lying costs the operator its stake, never the depositor's funds.
What a claim commits, and where:

- *Public inputs* ϕ: an anchor commitment to destination-chain state (the light-client root the relation verifies inclusion against), the peg-out txid (pinning which deposit is reimbursed; Remark 12.3), and a session identifier binding the proof to this garbled instance.
- *SNARK witness* (private): the burn transaction, its block header, and Merkle inclusion proofs, establishing that the claimed event actually occurred under the anchor.
- *On-chain Winternitz commitments*: the light client's checkpoint block hash and timestamp, advancing destination-chain state accountably, since committing two conflicting values is slashable equivocation (Remark 6.1). A hash-based OTS is the carrier because Bitcoin Script is stateless and `CHECKSIG` signs only transactions: with the OTS public key embedded in the script, hash opcodes verify the revealed preimages, making committed *data* script-checkable.
- *The pre-signed graph*: the payout destination itself, fixed at setup so a valid claim cannot redirect funds (Remark 11.6).

The relation itself is the paper's Algorithm 4 ($\mathcal{R}_{\text{eth-bridge}}$): verify a chain of sync-committee handoffs from C_{anchor} to a committee finalizing block B, verify the burn's inclusion in B, and check the burn references the deposit (txid $\|$ sid); the Bitcoin-side checkpoint chain is a separate relation (Algorithm 3, $\mathcal{R}_{\text{btc-lc}}$).

13.2.3 Disprove (on-chain, conditional, within the challenge window)

Any challenger reads σ_π off chain, extracts $L_\pi \leftarrow \texttt{GS.Extract}(pk_{GS}, \sigma_\pi)$, and evaluates the garbled verifier off chain: $L_y \leftarrow \texttt{Ev}(GC, L_\pi)$, then $y \leftarrow \texttt{De}(d, L_y)$. If $y =$ False the proof was invalid, so the challenger posts Disprove carrying the recovered output label L_y as witness; the script checks only the hash equality $H(L_y) = H(L^{\text{False}})$ against the committed false-output label, and on success slashes the operator's bond. If no valid Disprove appears before the timeout expires, the claim finalises and the operator collects. Bitcoin script therefore runs one hash comparison, never `Ev` or `De`.

13.2.4 Anatomy of the Assert Broadcast

The Assert template is fixed at setup: its input spends the bonded output, its outputs (the challenge/timeout taptree) are pinned by the committee's N-of-N SIGHASH_ALL signatures. What varies at broadcast time is only the witness.

Remark 13.4 (How one locking script accepts 2^{1024} witnesses). The bonded output's tapleaf is, schematically,

$$\langle pk_{slot}\rangle \texttt{ CHECKSIGVERIFY CODESEPARATOR } \langle pk_{slot}\rangle \texttt{ CHECKSIGVERIFY } \cdots \quad \text{(128 repetitions)}.$$

We write pk_{slot}, not pk_{op}: the operator must never hold this key's secret, or they could fresh-sign a consensus-valid Assert that completes no pre-signature and reveals no labels. It is a joint key whose shares are deleted after the pre-signatures are made (Remark 9.8); the paper writes it as the operator's key, leaving this arrangement unspecified. Script execution, performed by every validating node's interpreter when the transaction is relayed or mined, pops one item off the witness stack per `CHECKSIGVERIFY` and demands a valid BIP340 signature under pk_{slot}; any failure invalidates the transaction: this is what *enforces* that all 128 slots are present and genuine. The count derives from the proof: $|\pi| = 128$ bytes $= 1024$ bits, one garbled input

wire per bit, grouped 8 bits per slot, one signature per digit. Two facts make the same script accept many different fillings. (i) A signature check verifies against a *message*, not against a listed set of allowed signatures: any (R, s) satisfying the verification equation (9.1) for the slot's sighash passes. Both completions $\tilde{\sigma}_i^0 + L_i^0$ and $\tilde{\sigma}_i^1 + L_i^1$ are such valid signatures. The script cannot and need not distinguish them; the *pre-signatures* (off-chain artefacts) are what tie each acceptable signature to one label. (ii) The sighash never commits to witness data (§11.3), so the committee's covenant signatures pin inputs and outputs while leaving the witness free: one pre-signed template, $(2^8)^{128} = 2^{1024}$ admissible witnesses, one per possible proof value $\pi \in \{0,1\}^{1024}$, chosen at broadcast. For that freedom to be *sound*, each slot must sign a *distinct* message, so no signature can be moved between slots; making the 128 messages distinct under a single key is exactly the job of `OP_CODESEPARATOR` (Remark 13.7). Cost note: 128×65 signature bytes ≈ 8.3 kB; the full Assert witness is ≈ 9.2 kB, ≈ 2.4 kvB after the segwit discount; at 8 bits per 65-byte signature (≈ 8 bytes per proof bit) this is the $\sim 3\times$ saving over Winternitz's ~ 25 bytes per bit (§7). The single key also keeps the *script* small: it is pushed once, and the optimized leaf is $33 + 3(N-1) + 1$ bytes (BitVM3 §8.1), each additional slot costing 3 bytes rather than a fresh 33-byte key push.

The remark above explains the *flexibility*: why one fixed template admits a filling per proof value (the sighash commits to no witness data, and a signature check accepts any signature valid for its message, not a listed set). The next three pieces explain the *binding*: how the filling actually chosen is pinned down, first per slot (which signature releases which digit's labels), then per value (only the committed key completes a slot), then per position (a completed signature is spendable only in its own slot).

Example 13.5 (One-slot reveal)**.** Fix slot d of the Assert, covering the 8 input wires of digit d of π, the slot key x with $P = [x]G = pk_{slot}$ (its shares deleted after setup, Remark 9.8), and the slot's message m_d.

1. *Commit* (setup): for every value $v \in \{0, \ldots, 255\}$ publish the statement $T_{d,v} = [W_{d,v}]G$, the pre-signature $\widetilde{\sigma}_{d,v} \leftarrow \mathtt{PreSign}(x, m_d, T_{d,v})$ under a fresh nonce, and the row $\mathrm{row}_{d,v}$ encrypting the digit's 8 labels for value v (Definition 9.3). No pre-signature verifies as a signature; each becomes one only by adding the discrete log of its statement, $s = \widetilde{s}_{d,v} + W_{d,v}$.
2. *Prove* (claim time): the SNARK prover outputs π; let v be the value of digit d of π.
3. *Complete*: $\sigma_d \leftarrow \mathtt{Adapt}(\widetilde{\sigma}_{d,v}, W_{d,v}) = (R_{d,v} + T_{d,v},\ \widetilde{s}_{d,v} + W_{d,v})$, possible only for the operator since $W_{d,v}$ is secret; place σ_d in witness slot d. All 128 slots filled this way form the one witness consensus accepts.
4. *Extract* (any challenger): match the posted nonce point against the slot's 256 candidates $R_{d,v} + T_{d,v}$ to find v, compute $W_{d,v} = s - \widetilde{s}_{d,v}$, and decrypt $\mathrm{row}_{d,v}$ to the digit's 8 labels; collected over all slots this is exactly $\mathtt{En}(e, \pi)$, every other row untouched.

Completing two pre-signatures of a slot is equivocation, punished through evaluation (Remark 9.11); every pre-signature carries its own nonce, since a shared nonce would leak the slot key from the published pre-signatures alone (Remark 9.4).

Remark 13.6 (Only the committed label completes the signature). The binding runs the other way too: a slot spends *only* with one of its committed keys. The pre-signature $\widetilde{\sigma}_{d,v}$ fixes the statement $T_{d,v} = [W_{d,v}]G$ at setup, and completing it adds a scalar t with $s = \widetilde{s}_{d,v} + t$; by Proposition 9.3 the result $(R_{d,v} + T_{d,v}, s)$ satisfies the Schnorr equation $[s]G = (R_{d,v} + T_{d,v}) + [e_{d,v}]P$ *iff* $[t]G = T_{d,v}$, that is $t = W_{d,v}$. Any other scalar yields an (R, s) that fails verification, so the slot's `OP_CHECKSIGVERIFY` rejects (clause (i) above) and the Assert does not spend. The key is carried inside the tweak $T_{d,v}$, which enters the challenge $e_{d,v} = H(R_{d,v} + T_{d,v} \,\|\, P \,\|\, m_d)$,

so a wrong key is a wrong tweak and a wrong challenge; the operator cannot post a valid Assert without releasing genuine keys, hence the genuine $\texttt{En}(e, \pi)$.

Remark 13.7 (Why slots need distinct messages: reordering and replay). Remark 13.6 pins which *value* completes a slot; nothing so far pins which *position* a completed signature may occupy. A Schnorr check binds a signature to a *message*, never to a position in the witness. That message is the slot's sighash, and consensus accepts *any* (R, s) satisfying the verification equation (9.1) for it; the interpreter has no notion of "this signature was produced for slot i". The witness is attacker-supplied data, and `CHECKSIGVERIFY` merely pops the next stack item and checks it against the current message: reading the slots *in order* is not the same as checking that item k belongs to slot k. Nothing in the script ties a signature to a slot except which message it verifies against.

Suppose the 128 slots were *not* separated by `OP_CODESEPARATOR`. Each is $\langle pk_{slot} \rangle$ `CHECKSIGVERIFY` under the same key over the same transaction, so all 128 sighash messages are byte-for-byte identical; call the common value m. Every completed digit-signature verifies against m, hence against *every* slot. Whoever assembles the witness (the operator) can then *reorder* (place the 128 signatures in a permuted order) or *replay* (drop one digit's signature into many slots), and consensus still accepts: every check passes, since every signature verifies against the one common m.

Why a cheating operator wants this freedom: suppose no valid proof exists (the claimed event never happened). Filling the slots honestly is self-defeating, since evaluation would hand challengers the false-output label L^* (the paragraph below). A scrambled witness escapes this. Extraction is positional: the challenger recovers slot i's key as $s_i - \widetilde{s}_{i,v}$ against slot i's own pre-signatures, so a misplaced signature (one that actually completes slot j) yields a scalar that is no label at all, and a replayed signature releases one wire's label while other wires' labels never surface. Either way the challenger holds no well-formed $\texttt{En}(e, \pi)$: the recovered non-labels match no ciphertext row, trial decryption fails (§3.2), `Ev` aborts before any output label, and in particular L^*, is recovered. No Disprove can be posted, the timeout passes, and the operator collects on an invalid claim.

`OP_CODESEPARATOR` closes this by making the message position-specific. The opcode's standard description reads "ignore this and everything preceding when deciding what to sign when signature-checking": in legacy script the signed script truncates to what follows the last executed separator. Tapscript refines this: the *whole* leaf stays committed via its leaf hash, and BIP342 instead folds the opcode position of the last executed separator into the sighash, so slot i, sitting after the i-th separator, signs a message m_i with the m_i pairwise distinct. A signature for m_i now verifies *only* at slot i: reordering or replay fails the very check it was trying to pass, and a consensus-valid Assert necessarily releases the genuine, complete $\texttt{En}(e, \pi)$ for a single well-defined π. Valid-on-chain then implies evaluatable-off-chain, hence disprovable. One key still suffices: it is the message that is made position-specific, not the key, which is why 128 distinct keys would be the redundant alternative.

Proposition 13.8 (Position-binding is necessary and sufficient for extractability)**.** Write $m_1, \ldots, m_{128}$ for the slot sighashes of the Assert.

1. If the slots are not separated, so $m_1 = \cdots = m_{128} = m$, there exists a consensus-valid Assert witness whose extraction returns $\texttt{En}(e, \pi)$ for *no* π.
2. If the m_d are pairwise distinct and no fresh signature under the slot key can be produced after setup (Remark 9.8), every consensus-valid Assert witness extracts to $\texttt{En}(e, \pi)$ for a unique π.

Proof. (i) Let σ be any single completed signature, valid for the common message m, and fill all 128 witness slots with copies of σ. Every `CHECKSIGVERIFY` checks its popped item against m and passes, so the transaction is consensus-valid. Extraction at slot d is positional: the challenger

matches the posted nonce point against slot d's published candidates $R_{d,v} + T_{d,v}$. For d other than σ's slot of origin no candidate matches (nonces are fresh per pair, so a cross-slot coincidence has negligible probability), and subtracting any $\widetilde{s}_{d,v}$ yields a scalar t with $[t]G \neq T_{d,v}$: no key, no decrypted row, and `Ev` never runs. A valid Assert has revealed no encoding, so extractability fails.

(ii) With distinct messages, the signature at slot d verifies only against m_d. By the setup assumption the only producible signatures for m_d are completions of slot d's published pre-signatures, so the nonce-point match identifies a unique v_d, and subtraction yields W with $[W]G = T_{d,v_d}$, a genuine key (Remark 13.6). All 128 slots extract, and the recovered values assemble $\texttt{En}(e, \pi)$ for exactly the π whose digits are $(v_1, \ldots, v_{128})$. □

Remark 13.9 (What separates an honest Assert from a dishonest one). Nothing on-chain: both are the same template, valid by consensus, and extraction works identically on both. The separation happens off-chain, at evaluation: the challenger feeds the extracted labels into the public garbled tables and runs `Ev` gate by gate, obtaining the output label L^{v}_{out} for $v = \texttt{SNARK.Vrfy}(\varphi, \pi)$. *Ending 1, honest:* the destination event occurred, π verifies, $v = \text{True}$. The evaluator holds $L^{\text{True}}_{\text{out}}$, but the challenge leaf's hashlock demands the preimage of $h = H(L^{\text{False}}_{\text{out}})$, and authenticity (§2.2) says no evaluator can produce the label of the *other* output value. No Disprove exists; after Δ blocks the operator's timeout path opens. *Ending 2, dishonest:* the event never happened, so by SNARK soundness no valid proof exists: whatever 128 digits the operator committed, $v = \text{False}$, and evaluation *hands the challenger* $L^{\text{False}}_{\text{out}} = L^{*}$: the hashlock preimage. One Disprove later, the bond is slashed. The operator chooses its ending at completion time; the circuit merely executes the consequence. Lying is never prevented, only made losing: the same broadcast that authorises the claim deterministically arms its own refutation.

13.3 Why the Pieces Compose

Four properties established earlier line up with the four things the protocol needs; it is their conjunction that makes the dispute sound.

1. *Extractability gives availability.* The GES guarantee (§8) is that a signature on π *is* $\texttt{En}(e, \pi)$, so σ_π, the one on-chain object the operator must post anyway, already carries the input labels every challenger needs, with no separate label-delivery step and no oblivious transfer (Remark 3.9).

2. *Correctness, with the consistency proof, catches false claims.* The setup consistency proof fixes GC as a faithful garbling of the public `SNARK.Vrfy`, and correctness then gives $\texttt{De}(d, \texttt{Ev}(GC, \texttt{En}(e, \pi))) = \texttt{SNARK.Vrfy}(\pi)$. So an invalid π evaluates to False, an honest challenger recovers the False label and disproves it, while a valid π evaluates to True and survives to the timeout.

3. *Authenticity blocks false slashing.* By authenticity (Definition 2.3), an evaluator holding (GC, L_π) cannot produce any output label other than the genuine $\texttt{Ev}(GC, L_\pi)$. So no malicious challenger can forge the False label for a valid proof: the check $H(L_y) = H(L^{\text{False}})$ fires only when the proof really is invalid, and an honest operator cannot be framed.

4. *Non-equivocation gives enforceability.* The labels double as the one-time-signature secrets (§6), so the operator can open each input wire to only one bit; revealing both labels of a wire is equivocation, which is itself the slashable evidence (§6.2). In the clean model this non-equivocation is that Lamport preimage leak; under the deployed Schnorr adaptor reveal (§9) the same single-opening guarantee instead comes from the N-of-N pre-signed graph binding one pre-signature per proof. The committee's N-of-N presigning and SIGHASH_ALL covenant emulation stop the operator from swapping in a different

circuit or redirecting the bonded output, binding the off-chain verdict to a fixed on-chain transaction graph.

In sum: extractability makes the verdict computable by anyone, correctness-with-consistency makes a false claim catchable, authenticity stops an honest claim being slashed, and non-equivocation plus the covenant bind the verdict to a fixed transaction graph. Bitcoin only ever adjudicates a hash equality, yet the dispute it settles is the full SNARK-verification result.

14 Related Designs and Security History

The clean model of §13 is the paper's abstraction. The systems being built diverge from it, partly for efficiency and partly because the original garbling primitive was broken. This section records the security history and the deployed lineages; the pairing-based designs that replace the Boolean verifier (Glock, BABE, Argo MAC, Duty-Free Bits) are treated in Part **??**.

14.1 The RSA Garbling Scheme and Its Break

The first BitVM3 whitepaper instantiated the garbling scheme with RSA: the two labels on a wire are related by modular exponentiation $x \mapsto x^a \bmod N$, and RSA's multiplicative homomorphism replaces the symmetric encryption of Yao's tables. This was broken by Futoransky, Larotonda, and Barbàra (ePrint 2025/1291): because a wire's outgoing exponents a, b can be coprime, a malicious evaluator runs the extended Euclidean algorithm to find k_1, k_2 with $ak_1 + bk_2 = 1$ and recovers the secret label as $x = (x^a)^{k_1}(x^b)^{k_2} \bmod N$, *without* knowing $\varphi(N)$. This forges wire labels the evaluator was never given, breaking authenticity (Definition 2.3) on a minimal two-AND-gate counterexample. The scheme was retracted; GOAT's "label forward propagation" optimization built over it inherits the same break.

14.2 The GSV Lineage

The public `garbled-snark-verifier` reference implementation stays close to the deployed stack this document teaches: half-gate + Free-XOR garbling of the Groth16 verifier, cut-and-choose for setup verification, VSSS to reveal labels cheaply, and Schnorr adaptor signatures on-chain. Its own addition is *soldering*: a base evaluated instance carries the real input labels, and a succinct proof binds the per-wire deltas of every other evaluated instance to that base, so all evaluated instances provably receive the same logical input. It is a two-party "conditional revealing secret scheme" rather than BitVM3-core verbatim, but it composes the same primitives.

14.3 Lombard BSA: TEE-Attested, Sealed Labels

A more distant relative replaces the cryptographic honesty machinery with hardware. The Lombard BSA redesign garbles a *single* instance inside a trusted execution environment (an AWS Nitro enclave) and relies on remote *attestation* instead of cut-and-choose to convince verifiers the garbling was honest. Input labels are *ECIES-sealed* to a recipient public key, so the enclave operator that persists the circuit never learns them; the recipient later decrypts and reveals the label matching its proof on each wire. There is no hash-based one-time signature and no adaptor signature in the reveal path. This trades a cryptographic assumption for a hardware trust assumption, and is a genuinely different protocol, not an instantiation of BitVM3-core.

14.4 Reality Check

Three summary points. First, the garbling primitive is the least settled component: RSA is broken, Argo is the frontier but nascent, and no consensus winner has emerged. Second, the

on-chain label-reveal has largely converged on Schnorr adaptor signatures, for the ∼3–10× on-chain saving of §9. Third, BitVM3 is still research and testnet: the Bitcoin bridges live today (Citrea, GOAT, BOB) run on BitVM2, and the BitVM3 line is being hardened (Glock, Mosaic, and the BOB cost-reduction work) rather than deployed.

(optional) Further Reading

A Bitwise XOR as an Encryption Primitive

Bitwise XOR is its own inverse: for bit strings A, B of equal length, $A \oplus A = 0^{|A|}$ and $(B \oplus A) \oplus A = B$. Given a message M and a key K of the same length, $C := M \oplus K$ is an encryption: whoever knows K recovers $M = C \oplus K$, and if K is uniformly random and used once, C alone reveals nothing about M (the *one-time pad*).

For example, with $M = 1010$ and $K = 1100$, chosen so that the four positions cover all four (m_i, k_i) combinations, $C = 1010 \oplus 1100 = 0110$ and $C \oplus K = 0110 \oplus 1100 = 1010 = M$.

In the garbling scheme of Definition 3.2 the key is the hash $H(K_1 \,\|\, K_2 \,\|\, \nu)$ and the message is the padded label $M \,\|\, 0^t$; the evaluator recomputes the hash and XORs it off. The pad is pseudorandom, not uniform: its secrecy rests on H behaving as a random function and holds only against a bounded adversary, whereas a true one-time pad is secure against an unbounded one.

A.1 Wide-label rows

The lookup rows of §9.7 have the same shape at a larger size. A digit of w bits spans w wires, one label each, so the plaintext is the concatenation $L_1 \,\|\, \cdots \,\|\, L_w$, longer than one hash output. A *keystream* bridges the gap: the key $W_{d,v}$, serialized in little-endian byte order as the implementation does, is expanded block by block as $H(W_{d,v} \,\|\, 0) \,\|\, H(W_{d,v} \,\|\, 1) \,\|\, \cdots$ with H instantiated by SHA-256, and

$$\mathrm{row}_{d,v} = (L_1 \,\|\, \cdots \,\|\, L_w) \oplus \mathrm{keystream}(W_{d,v}). \tag{A.1}$$

The counters are load-bearing. Were one block repeated, the pad would cancel between any two label-sized segments of a published row, $(L_i \oplus P) \oplus (L_j \oplus P) = L_i \oplus L_j$, and such differences, combined with the one row opened at claim time, would yield unrevealed complementary labels. Under distinct counters the segment XOR is $L_i \oplus L_j \oplus H(W \,\|\, i) \oplus H(W \,\|\, j)$, from which nothing cancels. The challenger, holding W from extraction, regenerates the keystream, XORs it off, and splits the result into labels at fixed offsets; the labels were sampled independently of W and merely locked under it.

B Merkle Commitments

A *Merkle tree* commits to an ordered list of values $v_1, \ldots, v_n$ with a single hash, the *root*, such that any one v_i can later be proved a member without revealing the rest. Place the values at the leaves as $H(v_i)$ and hash pairwise up the tree: each internal node is $H(\mathrm{left} \,\|\, \mathrm{right})$, up to one root R. The root is a binding commitment to the whole list (changing any v_i changes R except with negligible collision probability), yet is only 32 bytes.

Membership is proved with an *authentication path*: to show v_i is in the tree, supply the sibling hash at each level from leaf to root. The verifier recomputes $H(v_i)$, folds in each sibling in turn, and accepts iff it arrives back at R. The path is $\log_2 n$ hashes, so proofs are logarithmic in the list size, the property that makes Merkle roots the workhorse of blockchain inclusion proofs (an operator's Ethereum burn proof is one such) and of Taproot script trees (§11.2).

Two design points bear on the security arguments in the body. First, *order and domain separation.* A bare $H(\text{left} \,\|\, \text{right})$ tree is malleable unless leaf and internal hashes are separated, since otherwise an internal node can be reinterpreted as a leaf (a second-preimage, "node-as-leaf" attack). BIP341 separates them with tagged hashes (`TapLeaf` versus `TapBranch`) and, unusually, *sorts* the two children before hashing each branch (lexicographically by byte value, the alphabetical order of their hex encodings), so no left/right flag need be transmitted, the reason the control block of Remark 11.1 carries only sibling hashes, not directions. Second, *binding but not hiding.* A Merkle root pins down its committed set but conceals nothing once a leaf is opened, which is what a Taproot script-path spend does when it reveals one tapleaf. This is why the taptree argument of Remark 11.2 rests on collision-resistance alone: there is nothing to hide, only a unique committed set to pin down.